

vmlinux: Anatomy of bzimage and how x86_64 processor is booted

Adrian Huang | May, 2021

- * Based on kernel 5.11 (x86_64) – QEMU
- * Legacy BIOS

Agenda

- bzimage: high-level overview
- Layout of bzImage
 - ELF layout
 - setup.bin and compressed vmlinux
 - Physical memory layout
- Entry point of Linux – ‘start_of_setup’@0x10200 (physical memory)
 - From viewpoint of GRUB and QEMU loader
 - Initialization flow
- Compressed vmlinux
 - ELF layout
 - Physical memory layout
 - Initialization flow

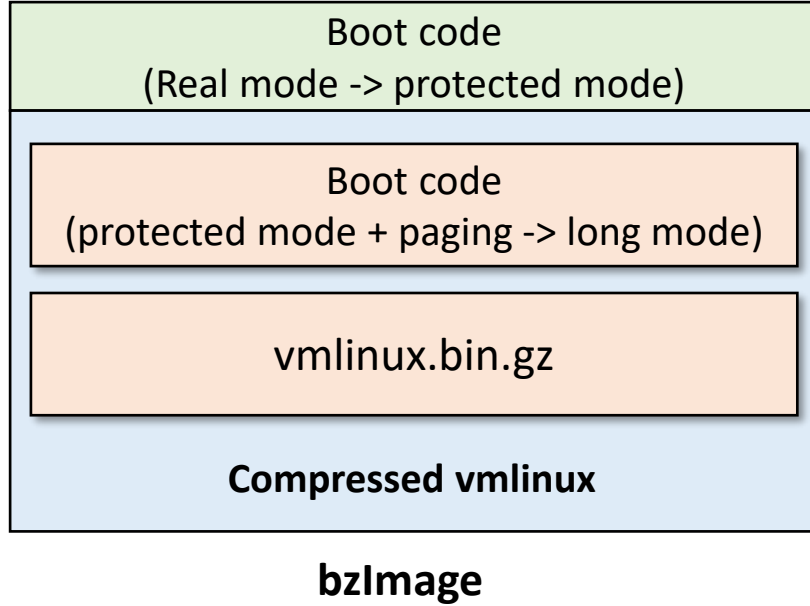
Agenda

- Layout of bzImage
 - ELF layout
 - setup.bin and compressed vmlinux
 - Physical memory layout
- Entry point of Linux – ‘start_of_setup’@0x10200 (physical memory)
 - From viewpoint of GRUB and QEMU loader
 - Initialization flow
- Compressed vmlinux
 - ELF layout
 - Physical memory layout
 - Initialization flow

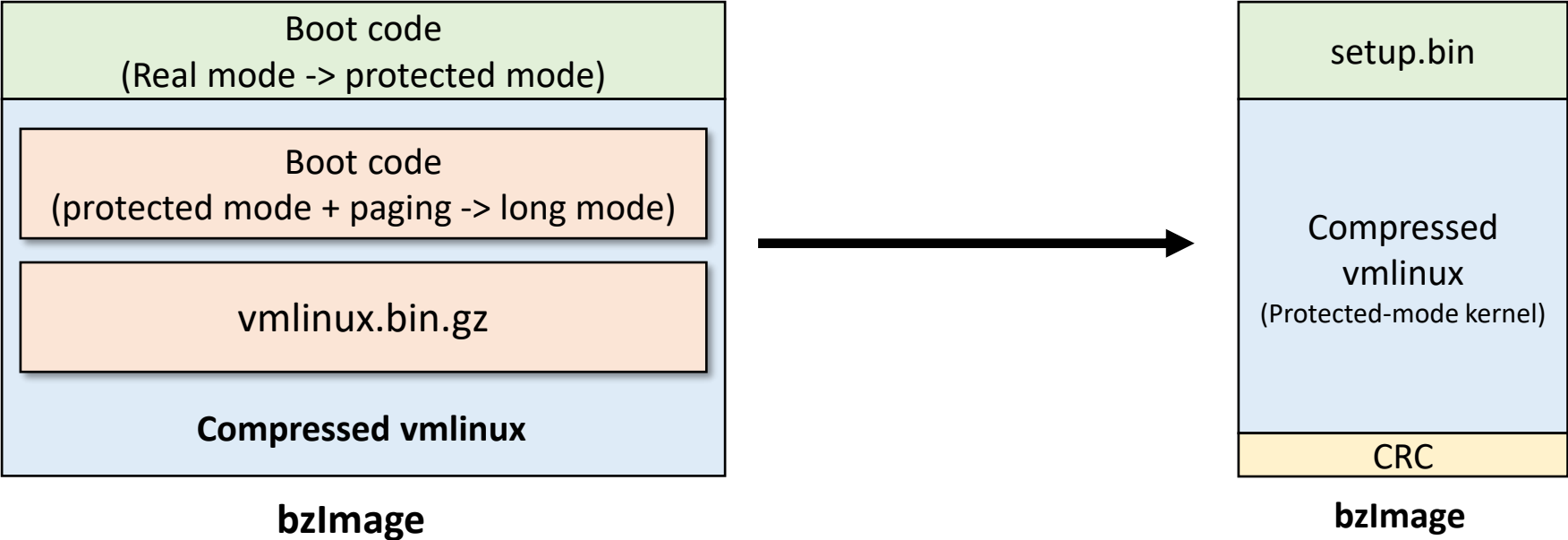
Requisite Knowledge

- CPU architecture knowledge
 - ✓ Near call and far call
 - ✓ Near jump and far jump
 - ✓ Instruction opcode
- CPU Operation Mode
 - ✓ Real mode, protected mode and long mode (64-bit mode)
 - Memory addressing
- ELF
 - ✓ Relocation, program header,...
- GNU assembly

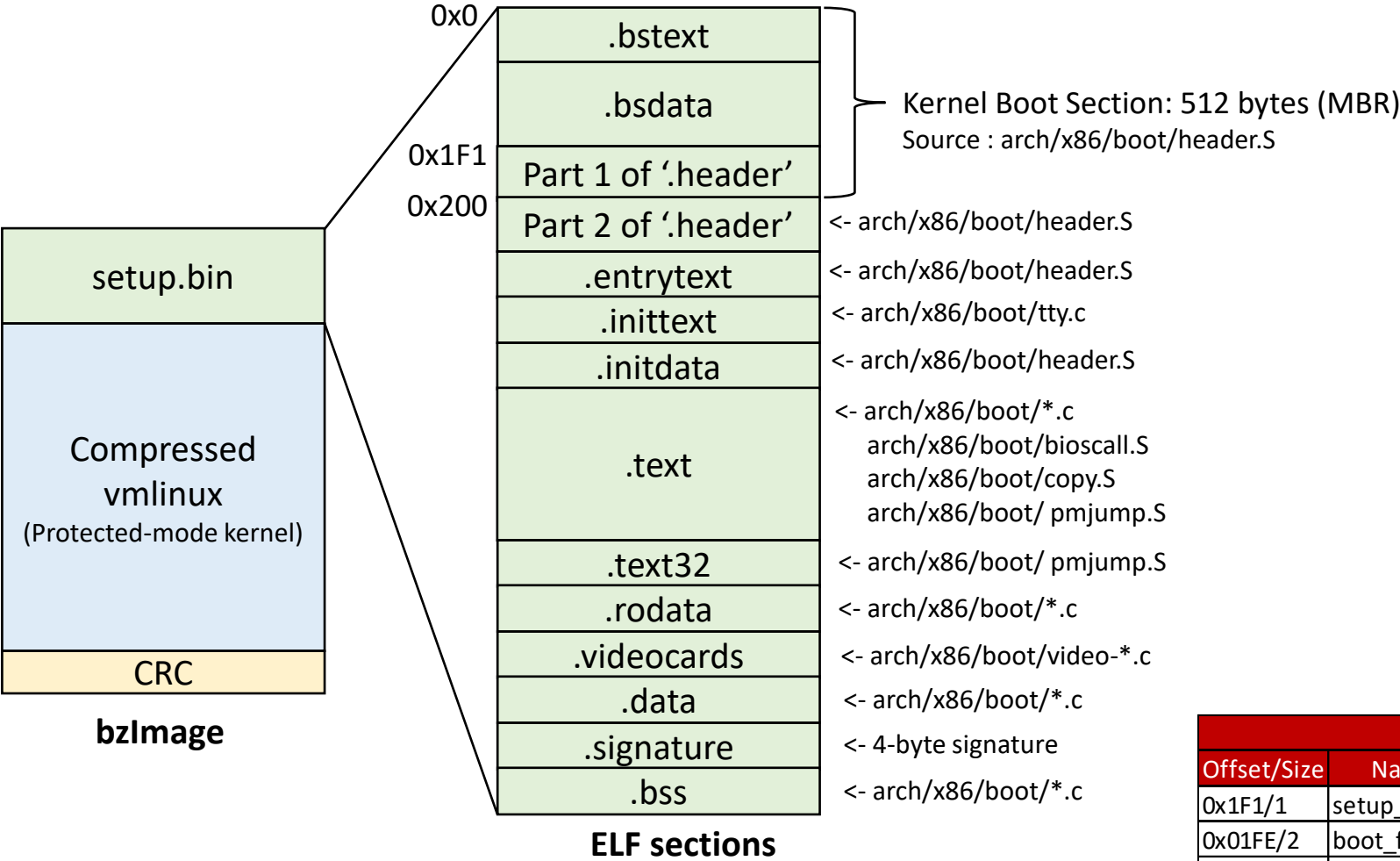
bzImage: High-level Overview (1/2)



bzImage: High-level Overview (2/2)



Layout of bzImage – setup.bin



```
struct setup_header {
    __u8    setup_sects;
    __u16   root_flags;
    __u32   syssize;
    __u16   ram_size;
    __u16   vid_mode;
    __u16   root_dev;
    __u16   boot_flag;
    __u16   jump;
    __u32   header;
    ...
    __u8    loadflags;
    __u16   setup_move_size;
    __u32   code32_start;
    __u32   ramdisk_image;
    __u32   ramdisk_size;
    ...
    __u16   heap_end_ptr;
    __u32   cmd_line_ptr;
    __u32   initrd_addr_max;
    __u32   kernel_alignment;
    __u8    relocatable_kernel;
    ...
} __attribute__((packed));
```

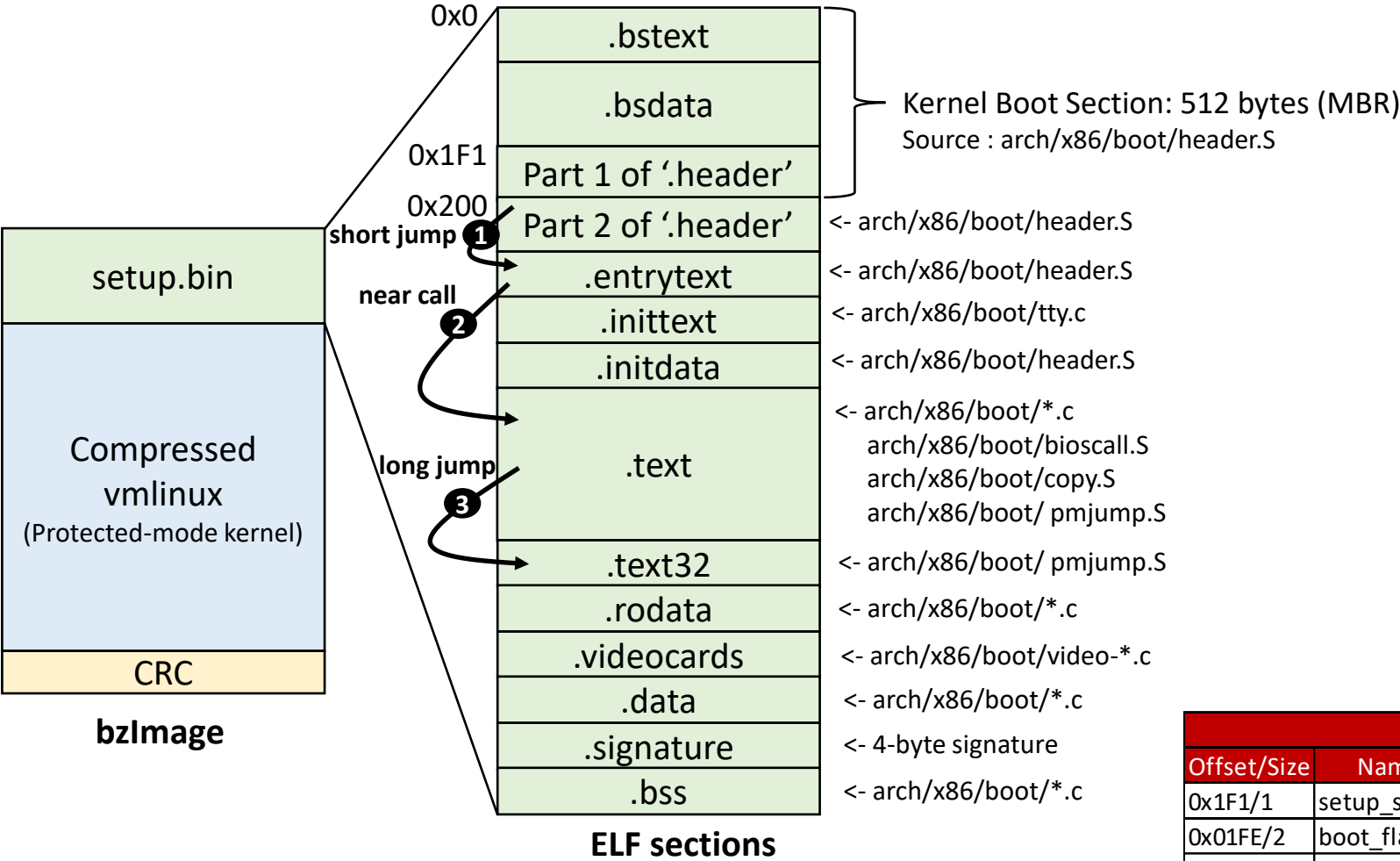
Part 1 of 'header'

Part 2 of 'header'

arch/x86/include/uapi/asm/bootparam.h

".header": Real-mode kernel header		
Offset/Size	Name	Description
0x1F1/1	setup_sects	The size of the setup in sectors
0x01FE/2	boot_flag	magic number: 0xAA55
0x200/2	jump	Jump instruction
0x214/4	code32_start	Boot loader hook: The address to jump to in protected mode. Default: 0x100000

Layout of bzImage – setup.bin



```
struct setup_header {
    __u8    setup_sects;
    __u16   root_flags;
    __u32   syssize;
    __u16   ram_size;
    __u16   vid_mode;
    __u16   root_dev;
    __u16   boot_flag;
    __u16   jump;
    __u32   header;
    ...
    __u8    loadflags;
    __u16   setup_move_size;
    __u32   code32_start;
    __u32   ramdisk_image;
    __u32   ramdisk_size;
    ...
    __u16   heap_end_ptr;
    __u32   cmd_line_ptr;
    __u32   initrd_addr_max;
    __u32   kernel_alignment;
    __u8    relocatable_kernel;
    ...
} __attribute__((packed));
```

Part 1 of 'header'

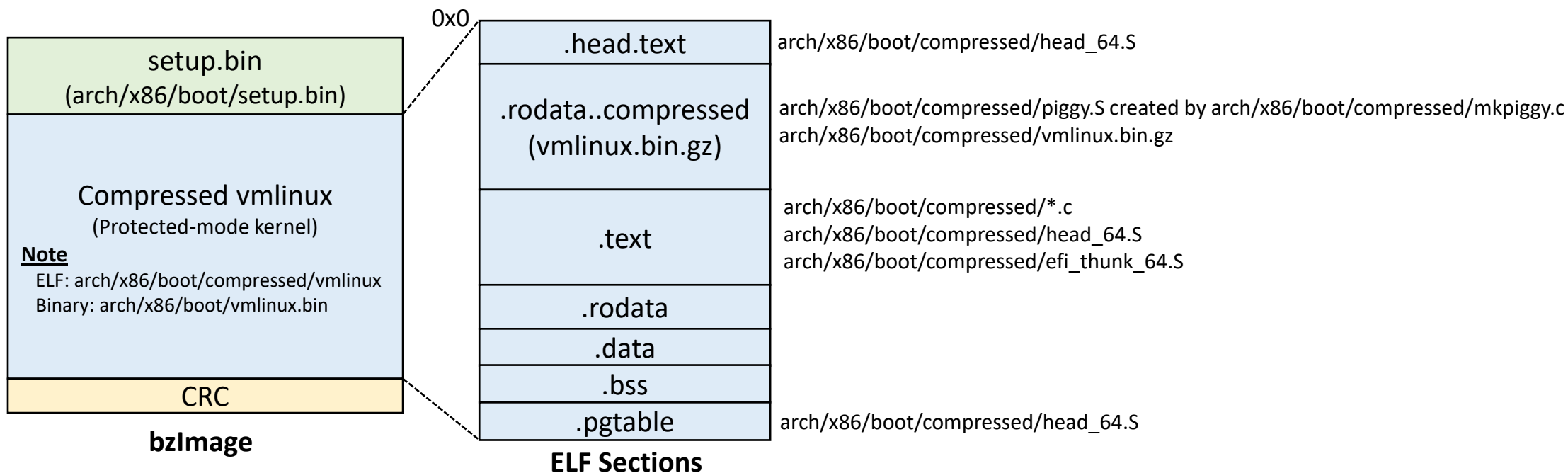
Part 2 of 'header'

arch/x86/include/uapi/asm/bootparam.h

".header": Real-mode kernel header		
Offset/Size	Name	Description
0x1F1/1	setup_sects	The size of the setup in sectors
0x01FE/2	boot_flag	magic number: 0xAA55
0x200/2	jump	Jump instruction
0x214/4	code32_start	Boot loader hook: The address to jump to in protected mode. Default: 0x100000

- ① CPU Real Mode (16 bits)
- ② CPU Real Mode
- ③ CPU Real Mode -> CPU Protected Mode (32 bits)

Layout of bzImage – compressed vmlinux



arch/x86/boot/compressed

```
adrian@adrian-ubuntu:linux$ ls arch/x86/boot/compressed/
cmdline.o          idt_64.o          mkpiggy           vmlinux.bin
cpuflags.o         idt_handlers_64.o pgtable_64.o      vmlinux.bin.gz
early_serial_console.o kaslr.o          piggy.o           vmlinux.lds
error.o            kernel_info.o    piggy.S           vmlinux.relocs
head_64.o          mem_encrypt.o    string.o
ident_map_64.o     misc.o           vmlinux
```

vmlinux.bin

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.bin.cmd
cmd_arch/x86/boot/compressed/vmlinux.bin := objcopy -R .comment -S vmlinux arch/
/x86/boot/compressed/vmlinux.bin
```

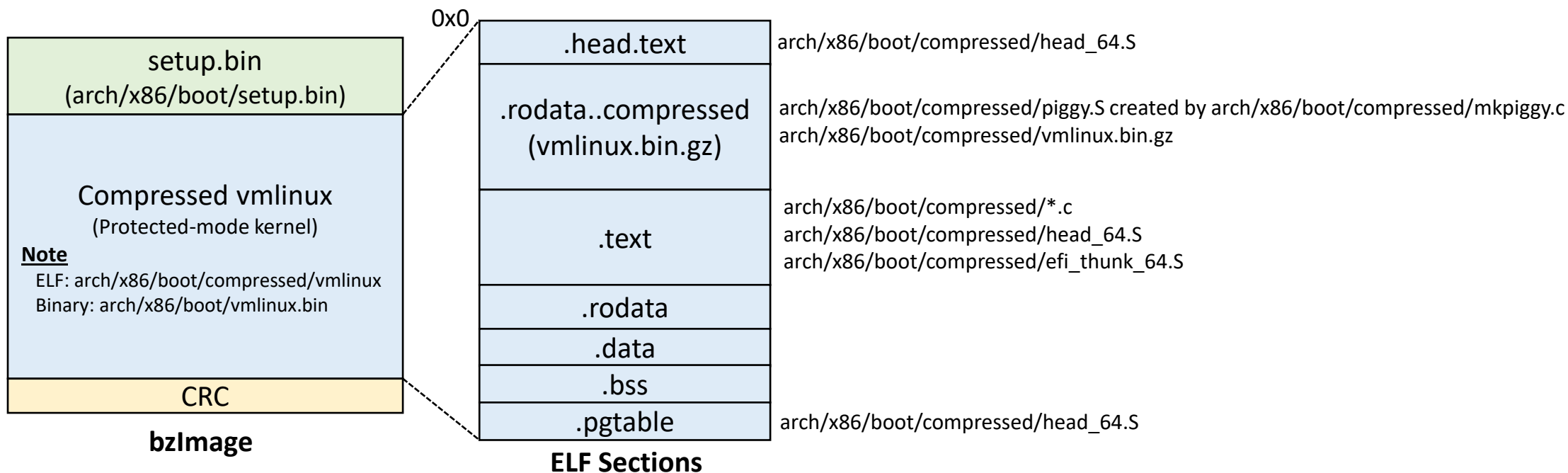
vmlinux.bin.gz

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.bin.gz.cmd
cmd_arch/x86/boot/compressed/vmlinux.bin.gz := cat arch/x86/boot/compressed/vml
inux.bin arch/x86/boot/compressed/vmlinux.relocs | gzip -n -f -9 > arch/x86/boot/
compressed/vmlinux.bin.gz
```

How to pack vmlinux.bin.gz?

```
adrian@adrian-ubuntu:linux$ vimcat arch/x86/boot/compressed/piggy.S
.section ".rodata..compressed", "a", @progbits
.globl z_input_len
z_input_len = 1322905
.globl z_output_len
z_output_len = 11884712
.globl input_data, input_data_end
input_data:
.incbin "arch/x86/boot/compressed/vmlinux.bin.gz"
input_data_end:
.section ".rodata", "a", @progbits
.globl input_len
input_len:
        .long 1322905
.globl output_len
output_len:
        .long 11884712
```

Layout of bzImage – compressed vmlinux



Compressed vmlinux

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.cmd
cmd_arch/x86/boot/compressed/vmlinux := ld -m elf_x86_64 --no-ld-generated-unwind-info -pie --no-dynamic-linker --orphan-handling=warn -T arch/x86/boot/compressed/vmlinux.lds arch/x86/boot/compressed/kernel_info.o arch/x86/boot/compressed/head_64.o arch/x86/boot/compressed/misc.o arch/x86/boot/compressed/string.o arch/x86/boot/compressed/cmdline.o arch/x86/boot/compressed/error.o arch/x86/boot/compressed/piggy.o arch/x86/boot/compressed/cpuflags.o arch/x86/boot/compressed/early_serial_console.o arch/x86/boot/compressed/kaslr.o arch/x86/boot/compressed/ident_map_64.o arch/x86/boot/compressed/idt_64.o arch/x86/boot/compressed/idt_handlers_64.o arch/x86/boot/compressed/mem_encrypt.o arch/x86/boot/compressed/pgtable_64.o -o arch/x86/boot/compressed/vmlinux
```

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/.bzImage.cmd
cmd_arch/x86/boot/bzImage := arch/x86/boot/tools/build arch/x86/boot/setup.bin arch/x86/boot/vmlinux.bin arch/x86/boot/offset.h arch/x86/boot/bzImage
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/.vmlinux.bin.cmd
cmd_arch/x86/boot/vmlinux.bin := objcopy -O binary -R .note -R .comment -S arch/x86/boot/compressed/vmlinux arch/x86/boot/vmlinux.bin
```

Layout of bzImage – compressed vmlinux.bin

```
adrian@adrian-ubuntu:linux$ vimcat arch/x86/boot/compressed/piggy.S
.section ".rodata..compressed","a",@progbits
.globl z_input_len
z_input_len = 1322905 ← * Symbol: Equivalent to using 'set' directive
.globl z_output_len
z_output_len = 11884712
.globl input_data, input_data_end
input_data:
.incbin "arch/x86/boot/compressed/vmlinux.bin.gz"
input_data_end:
.section ".rodata","a",@progbits
.globl input_len
input_len:
    .long 1322905
.globl output_len
output_len:
    .long 11884712
```

```
adrian@adrian-ubuntu:linux$ readelf -s arch/x86/boot/compressed/piggy.o

Symbol table '.symtab' contains 12 entries:

```

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	2	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	SECTION	LOCAL	DEFAULT	4	
5:	0000000000000000	0	SECTION	LOCAL	DEFAULT	5	
6:	00000000000142f99	0	NOTYPE	GLOBAL	DEFAULT	ABS	z_input_len
7:	00000000000b558a8	0	NOTYPE	GLOBAL	DEFAULT	ABS	z_output_len
8:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	4	input_data
9:	00000000000142f99	0	NOTYPE	GLOBAL	DEFAULT	4	input_data_end
10:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	5	input_len
11:	00000000000000004	0	NOTYPE	GLOBAL	DEFAULT	5	output_len

```
$ objdump -D arch/x86/boot/compressed/piggy.o
arch/x86/boot/compressed/piggy.o:      file format elf64-x86-64
```

Disassembly of section .rodata..compressed:

```
0000000000000000 <input_data>:
      0:      1f                (bad)
      1:      8b 08            mov     (%rax),%ecx
...
142f97:      b5 00            mov     $0x0,%ch
```

Disassembly of section .rodata:

```
0000000000000000 <input_len>:
      0:      99                cltd
      1:      2f                (bad)
      2:      14 00            adc     $0x0,%al

0000000000000004 <output_len>:
      4:      a8 58            test    $0x58,%al
      6:      b5 00            mov     $0x0,%ch
```

Why z_input_len/input and z_output_len/output_len?

```
commit 3f086189cd3641d212949ff044d8e4486c93d55e
Author: Arvind Sankar <nivedita@alum.mit.edu>
Date:   Fri Jul 31 16:07:50 2020 -0700
```

x86/boot: Remove run-time relocations from head_{32,64}.S

The BFD linker generates run-time relocations for z_input_len and z_output_len, even though they are absolute symbols.

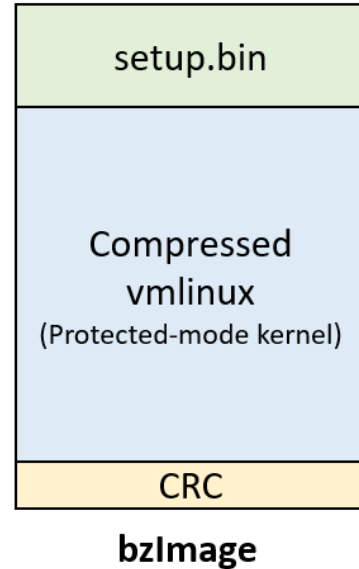
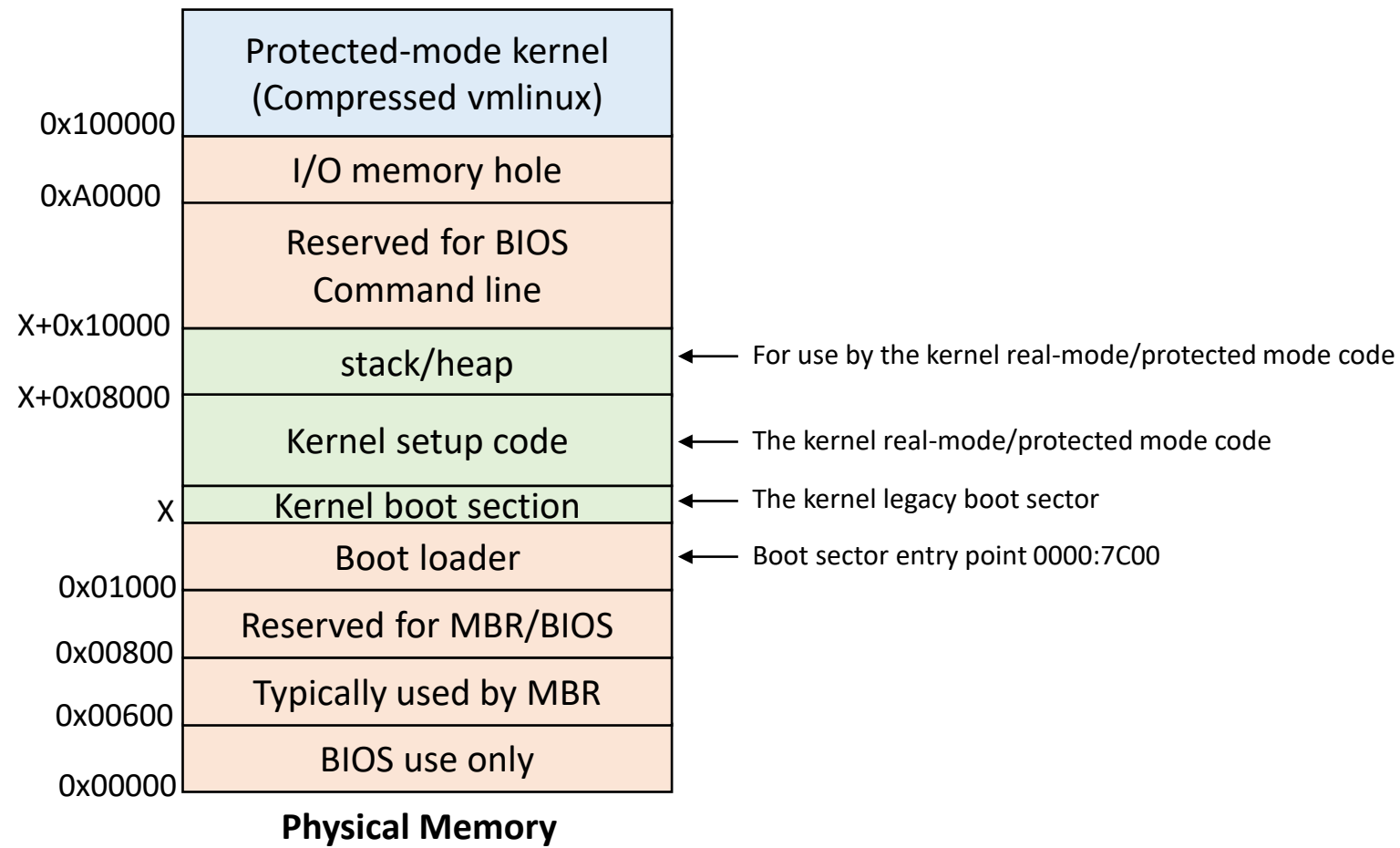
This is fixed for binutils-2.35 [1]. Work around this for earlier versions by defining two variables input_len and output_len in addition to the symbols, and use them via position-independent references.

This eliminates the last two run-time relocations in the head code and allows us to drop the -z noreloc-overflow flag to the linker.

* <https://sourceware.org/binutils/docs/as/Setting-Symbols.html>

* BFD: Binary File Descriptor library - <https://www.gnu.org/software/binutils/>

Memory layout of bzImage – Entry Point Address

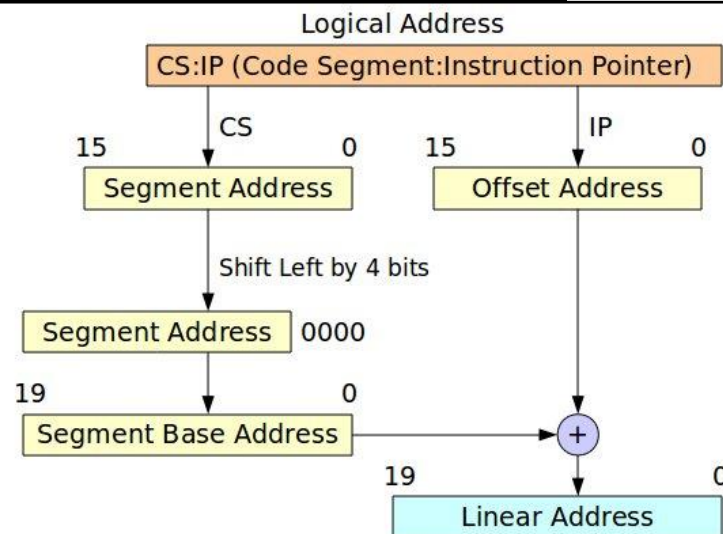


bzImage

Where is 'X'?

Entry Point of Linux - GRUB

Memory addressing in real mode



Logical Address: 0x07C0:0x0008
CS=0x07C0, IP: 0x0008

Memory Segmentation

Linear Address = 0x7C08

```
static grub_err_t
grub_linux16_boot (void)
{
    grub_uint16_t segment;
    struct grub_relocator16_state state;

    segment = grub_linux_real_target >> 4;
    state.gs = state.fs = state.es = state.ds = state.ss = segment;
    state.sp = GRUB_LINUX_SETUP_STACK;
    state.cs = segment + 0x20;
    state.ip = 0;
    state.a20 = 1;
}
```

grub-core/loader/i386/pc/linux.c

42,1

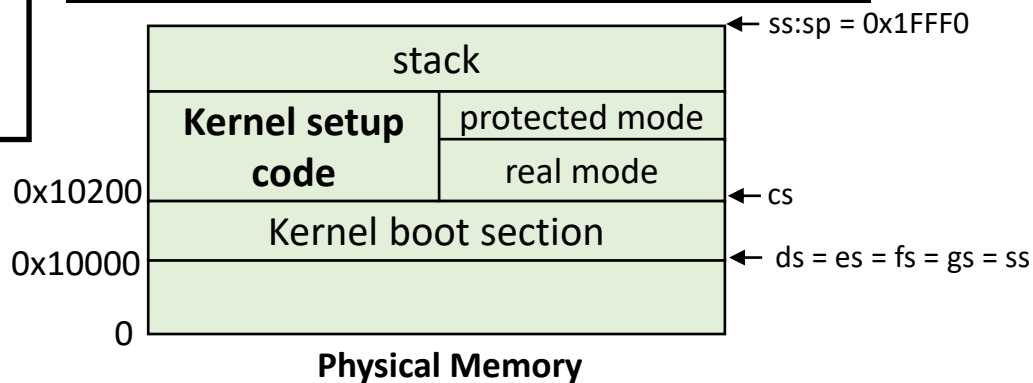
[GRUB] Get the memory address for real mode code

```
grub_cmd_linux
|- grub_linux_real_target = grub_find_real_target()
|- grub_mmap_iterate
```

Registers configured by GRUB

- gs = fs = es = ds = ss = 0x1000
- sp = GRUB_LINUX_SETUP_STACK = 0x9000
- cs = 0x1020, ip = 0

GRUB loads 'setup.bin' at address 0x10000



```
-----[ CPU ]-----
AX: 1020 BX: 0000 CX: 0000 DX: 0000
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000

IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

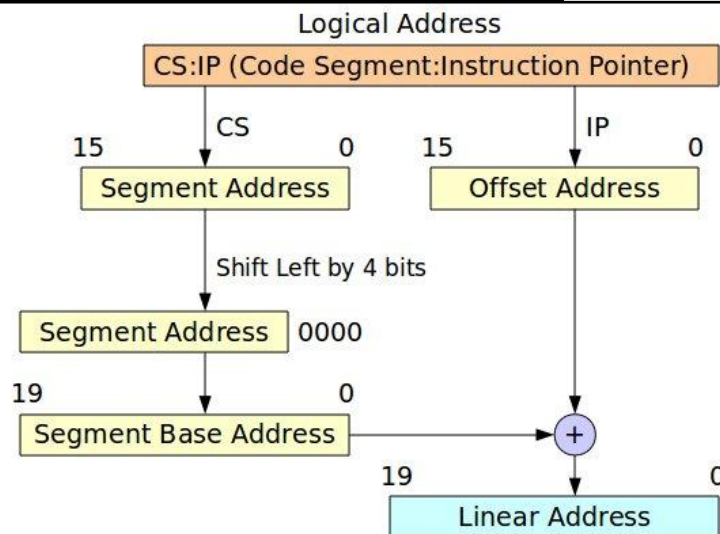
-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]

-----[ eflags]-----
OF <0> DF <0> IF <0> TF <0> SF <0> ZF <1> AF <0> PF <1> CF <0>
ID <0> VIP <0> VIF <0> AC <0> VM <0> RF <0> NT <0> IOPL <0>

-----[ CODE ]-----
0x10200: jmp 0x1026c
0x10202: rex.W
0x10203: fs jb 0x10259
```

Entry Point of Linux - GRUB

Memory addressing in real mode



Logical Address: 0x07C0:0x0008
CS=0x07C0, IP: 0x0008

Memory Segmentation

Linear Address = 0x7C08

```
static grub_err_t
grub_linux16_boot (void)
{
    grub_uint16_t segment;
    struct grub_relocator16_state state;

    segment = grub_linux_real_target >> 4;
    state.gs = state.fs = state.es = state.ds = state.ss = segment;
    state.sp = GRUB_LINUX_SETUP_STACK;
    state.cs = segment + 0x20;
    state.ip = 0;
    state.a20 = 1;
}
```

grub-core/loader/i386/pc/linux.c

42,1

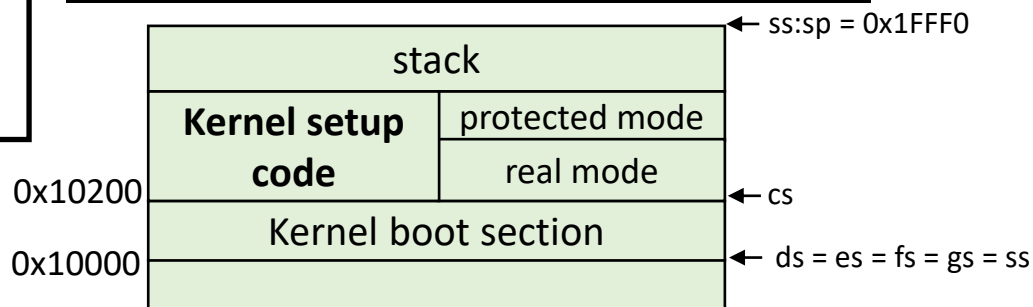
[GRUB] Get the memory address for real mode code

```
grub_cmd_linux
|- grub_linux_real_target = grub_find_real_target()
|- grub_mmap_iterate
```

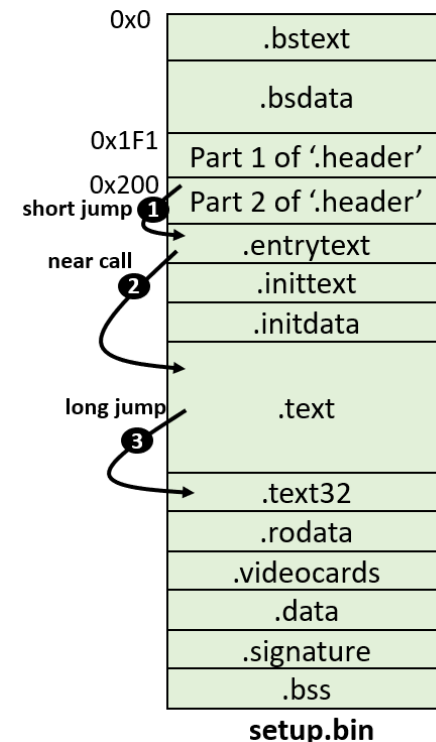
Registers configured by GRUB

- gs = fs = es = ds = ss = 0x1000
- sp = GRUB_LINUX_SETUP_STACK = 0x9000
- cs = 0x1020, ip = 0

GRUB loads 'setup.bin' at address 0x10000



- QEMU loader and GRUB load 'setup.bin' at address 0x10000
- QEMU loader sets SS:SP = 1000:FFF0 while GRUB sets SS:SP 1000:9000



```
-----[ CPU ]-----
AX: 1020 BX: 0000 CX: 0000 DX: 0000
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000

IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0           [ PDBR=0 PCID=0 ]
efer         0x0           [ ]

-----[ eflags]-----
OF <0> DF <0> IF <0> TF <0> SF <0> ZF <1> AF <0> PF <1> CF <0>
ID <0> VIP <0> VIF <0> AC <0> VM <0> RF <0> NT <0> IOPL <0>

-----[ CODE ]-----
0x10200: jmp 0x1026c
0x10202: rex.W
0x10203: fs jb 0x10259
```

Entry Point of Linux: QEMU loader

```
void load_kernel(void)
{
    ...
    3 bios_cfg_read_entry_dma(&setup_addr, FW_CFG_SETUP_ADDR, 4);

    bios_cfg_read_entry_dma(&kernel_addr, FW_CFG_KERNEL_ADDR, 4);

    ...
    /* Boot linux */
    4 segment_addr = ((uint32_t)setup_addr >> 4);
    stack_addr = (uint32_t)(cmdline_addr - setup_addr - 16);

    /* As we are changing critical registers, we cannot leave freedom to the
     * compiler.
     */
    asm("movw %%ax, %%ds\n"
        "movw %%ax, %%es\n"
        "movw %%ax, %%fs\n"
        "movw %%ax, %%gs\n"
        "movw %%ax, %%ss\n"
        "movl %%ebx, %%esp\n"
        "addw $0x20, %%ax\n"
        "pushw %%ax\n" /* CS */
        "pushw $0\n" /* IP */
        /* Clear registers and jump to Linux */
        "xor %%ebx, %%ebx\n"
        "xor %%ecx, %%ecx\n"
        "xor %%edx, %%edx\n"
        "xor %%edi, %%edi\n"
        "xor %%ebp, %%ebp\n"
        "lretw\n"
        : : "a"(segment_addr), "b"(stack_addr));
}
```

5

6

7 Prepare for far return

8 far return: change 'cs' by means of CPU arch itself

pc-bios/optionrom/linuxboot_dma.c 135,1

```
void x86_load_linux(X86MachineState *x86ms,
                  FWCfgState *fw_cfg,
                  int acpi_data_size,
                  bool pvh_enabled,
                  bool linuxboot_dma_enabled)
{
    ...
    if (protocol < 0x200 || !(header[0x211] & 0x01)) {
        /* Low kernel */
        real_addr = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr = 0x10000;
    } else if (protocol < 0x202) {
        /* High but ancient kernel */
        real_addr = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr = 0x100000;
    } else {
        /* High and recent kernel */
        1 real_addr = 0x10000;
        cmdline_addr = 0x20000;
        prot_addr = 0x100000;
    }

    ...
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_ADDR, prot_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_SIZE, kernel_size);

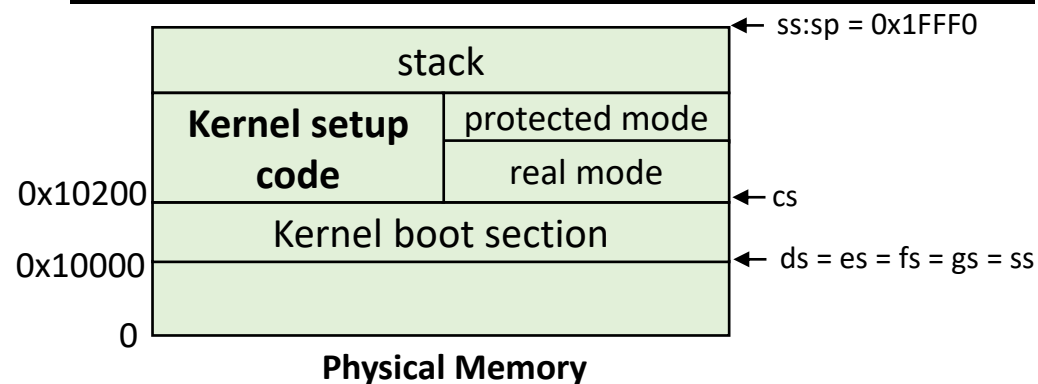
    2 fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_ADDR, real_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_SIZE, setup_size);
    ...
}
```

hw/i386/x86.c

Registers configured by QEMU loader

- 5 ds = es = fs = gs = ss = segment_addr = 0x1000
- 6 esp = stack_addr = cmdline_addr - setup_addr - 16 = 0x20000 - 0x10000 - 16 = 0x10000 - 16 = 0xfff0
- 7 cs = 0x1020, ip = 0

QEMU loader loads 'setup.bin' at address 0x10000



Entry Point of Linux: QEMU loader – Near and Far calls

```
void load_kernel(void)
{
    ...
    3 bios_cfg_read_entry_dma(&setup_addr, FW_CFG_SETUP_ADDR, 4);

    bios_cfg_read_entry_dma(&kernel_addr, FW_CFG_KERNEL_ADDR, 4);

    ...
    /* Boot linux */
    4 segment_addr = ((uint32_t)setup_addr >> 4);
    stack_addr = (uint32_t)(cmdline_addr - setup_addr - 16);

    /* As we are changing critical registers, we cannot leave freedom to the
     * compiler.
     */
    asm("movw %%ax, %%ds\n"
        "movw %%ax, %%es\n"
        "movw %%ax, %%fs\n"
        "movw %%ax, %%gs\n"
        "movw %%ax, %%ss\n"
        "movl %%ebx, %%esp\n"
        "addw $0x20, %%ax\n"
        "pushw %%ax\n" /* CS */
        "pushw $0\n" /* IP */
        /* Clear registers and jump to Linux */
        "xor %%ebx, %%ebx\n"
        "xor %%ecx, %%ecx\n"
        "xor %%edx, %%edx\n"
        "xor %%edi, %%edi\n"
        "xor %%ebp, %%ebp\n"
        "lretw\n"
        : : "a"(segment_addr), "b"(stack_addr));
}
```

5

6

7 Prepare for far return

8 far return: change 'cs' by means of CPU arch itself

pc-bios/optionrom/linuxboot dma.c 135,1

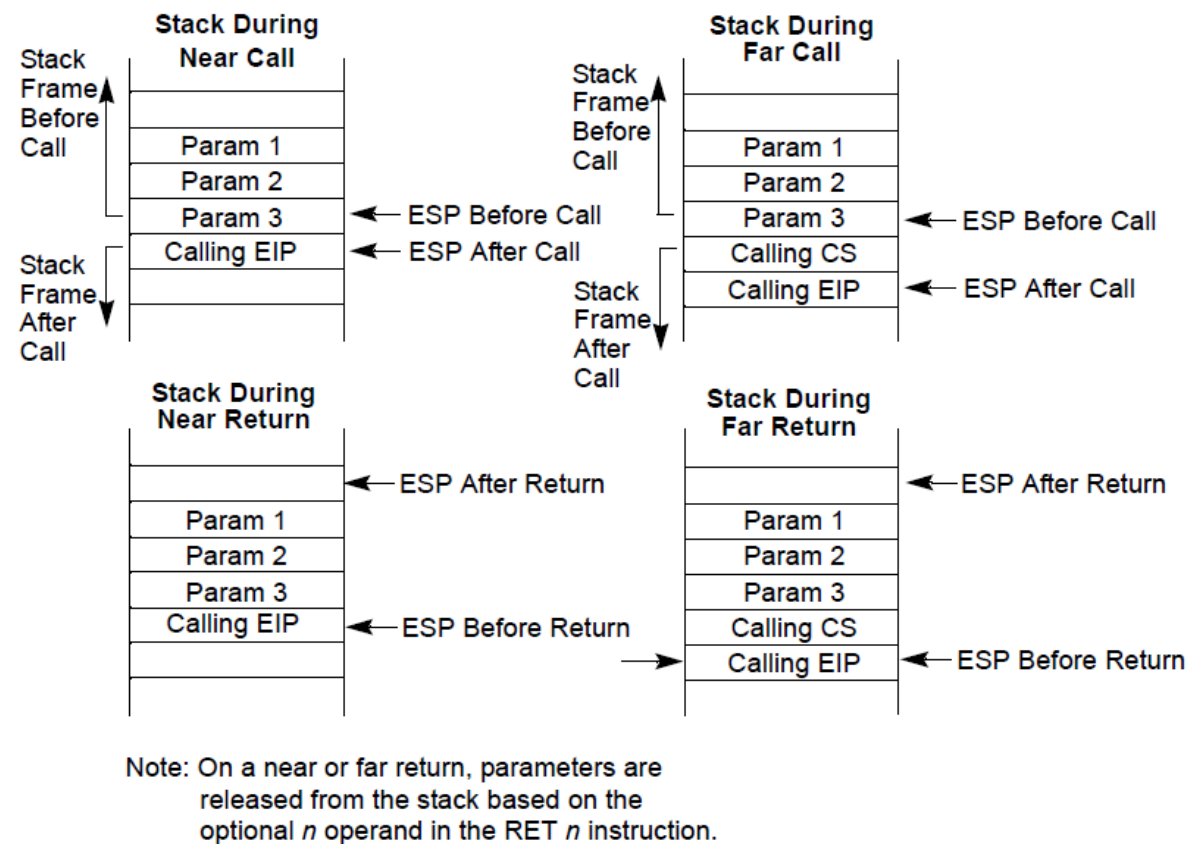
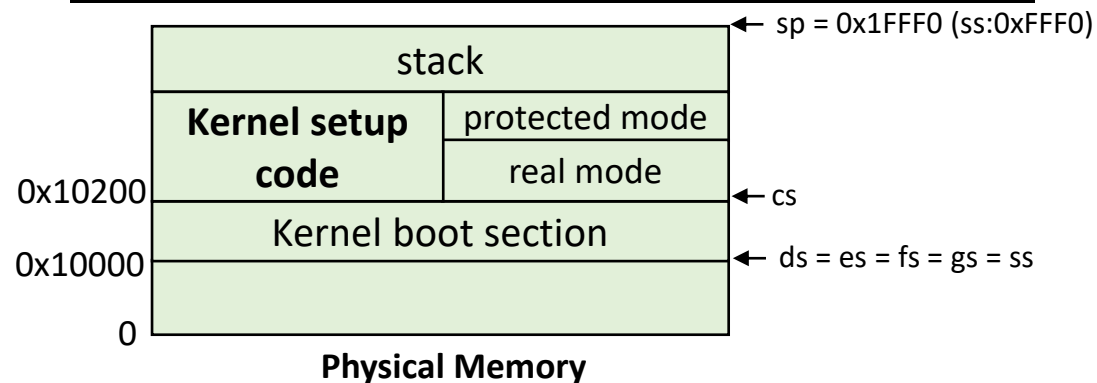


Figure 6-2. Stack on Near and Far Calls

Entry Point of Linux: QEMU loader

QEMU loader loads 'setup.bin' at address 0x10000



Make sure setup.bin is loaded at 0x10000

```
(gdb) x /4x 0x10000
```

```
0x10000: 0xc00005ea 0x8ec88c07 0x8ec08ed8 0xfbe431d0
```

```
adrian@adrian-ubuntu:linux$ xxd -e arch/x86/boot/setup.bin | head -n 1
00000000: c00005ea 8ec88c07 8ec08ed8 fbe431d0 .....1..
```

Make sure vmlinux.bin is loaded at 0x100000

```
(gdb) x /4x 0x100000
```

```
0x100000: 0xa68dfa6c 0x000001e8 0x000000e8 0xed835d00
```

```
adrian@adrian-ubuntu:linux$ xxd -e arch/x86/boot/vmlinux.bin | head -n 1
00000000: a68dfa6c 000001e8 000000e8 ed835d00 .....]
```

```
void x86_load_linux(X86MachineState *x86ms,
                   FWCfgState *fw_cfg,
                   int acpi_data_size,
                   bool pvh_enabled,
                   bool linuxboot_dma_enabled)
{
    ...
    if (protocol < 0x200 || !(header[0x211] & 0x01)) {
        /* Low kernel */
        real_addr = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr = 0x10000;
    } else if (protocol < 0x202) {
        /* High but ancient kernel */
        real_addr = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr = 0x100000;
    } else {
        /* High and recent kernel */
        real_addr = 0x10000; ← Address of setup.bin
        cmdline_addr = 0x20000;
        prot_addr = 0x100000; ← Address of vmlinux.bin
    }

    ...
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_ADDR, prot_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_SIZE, kernel_size);

    fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_ADDR, real_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_SIZE, setup_size);
    ...
}
hw/i386/x86.c
```

Entry Point of Linux: GNU Linker

```
/*
 * setup.ld
 *
 * Linker script for the i386 setup code
 */
```

```
OUTPUT_FORMAT("elf32-i386")
```

```
OUTPUT_ARCH(i386)
```

```
ENTRY(_start)
```

```
SECTIONS
```

```
{
    . = 0;
    .btext      : { *(.btext) }
    .bdata      : { *(.bdata) }

    . = 495;
    .header     : { *(.header) }
    .entrytext  : { *(.entrytext) }
    .inittext   : { *(.inittext) }
    .initdata   : { *(.initdata) }
    __end_init = .;
}
```

arch/x86/boot/setup.ld

[GNU Linker] ENTRY() command

- * First executable instruction in an output file → entry point
- * ENTRY() is one of choosing the entry point
 - the '-e' entry command-line option
 - the ENTRY(symbol) command in a linker control script
 - the value of the symbol **start**, if present
 - the address of the first byte of the .text section, if present;
 - the address 0

```
...
# Kernel attributes; used by setup. This is part 1 of the
# header, from the old boot sector.

.section ".header", "a"
.globl sentinel
sentinel: .byte 0xff, 0xff /* Used to detect broken loaders */

.globl hdr
hdr:
setup_sects: .byte 0 /* Filled in by build.c */
root_flags: .word ROOT_RDONLY /* Filled in by build.c */
syssize: .long 0 /* Filled in by build.c */
ram_size: .word 0 /* Obsolete */
vid_mode: .word SVGA_MODE
root_dev: .word 0 /* Filled in by build.c */
boot_flag: .word 0xAA55
```

```
# offset 512, entry point
```

```
.globl start
start:
```

```
# Explicitly enter this as bytes, or the assembler
# tries to generate a 3-byte jump here, which causes
# everything else to push off to the wrong offset.
.byte 0xeb # short (2-byte) jump
.byte start_of_setup-1f
```

```
1:
```

```
# Part 2 of the header, from the old setup.S
```

```
.ascii "HdrS" # header signature
.word 0x020f # header version number (>= 0x0105)
# or else old loadlin-1.5 will fail)
```

```
init_size: .long INIT_SIZE # kernel initialization size
handover_offset: .long 0 # Filled in by build.c
kernel_info_offset: .long 0 # Filled in by build.c
```

```
# End of setup header #####
```

```
.section ".entrytext", "ax"
```

```
start_of_setup:
```

```
# Force %es = %ds
```

```
movw %ds, %ax
```

```
movw %ax, %es
```

arch/x86/boot/header.S

Entry Point of Linux: GNU Linker

```
/*
 * setup.ld
 *
 * Linker script for the i386 setup code
 */
OUTPUT_FORMAT("elf32-i386")
OUTPUT_ARCH(i386)
ENTRY(_start)

SECTIONS
{
    . = 0;
    .btext      : { *(.btext) }
    .bdata      : { *(.bdata) }

    . = 495;
    .header     : { *(.header) }
    .entrytext  : { *(.entrytext) }
    .inittext   : { *(.inittext) }
    .initdata   : { *(.initdata) }
    __end_init = .;
}
```

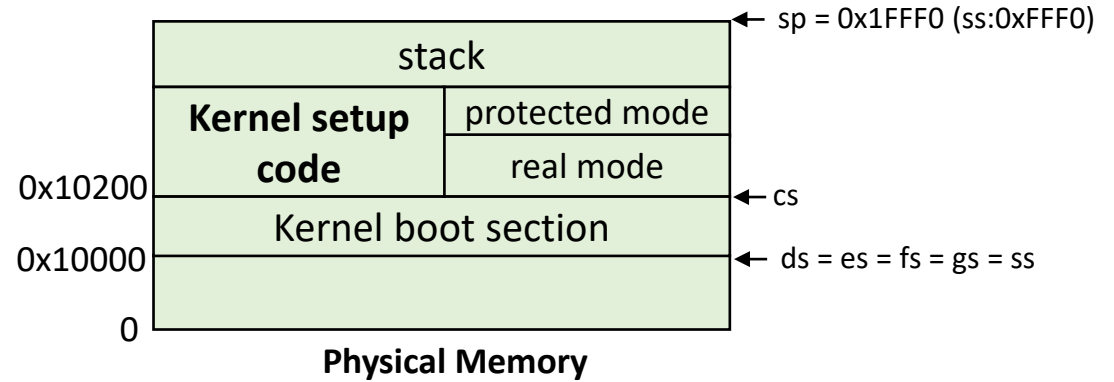
arch/x86/boot/setup.ld

[GNU Linker] ENTRY() command

- * First executable instruction in an output file → entry point
- * ENTRY() is one of choosing the entry point
 - the '-e' entry command-line option
 - the ENTRY(symbol) command in a linker control script
 - the value of the symbol **start**, if present
 - the address of the first byte of the .text section, if present;
 - the address 0

```
adrian@adrian-ubuntu:linux$ readelf -h arch/x86/boot/setup.elf
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF32
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                                Intel 80386
  Version:                                0x1
  Entry point address:                    0x200
  Start of program headers:               52 (bytes into file)
  Start of section headers:              259364 (bytes into file)
  Flags:                                  0x0
  Size of this header:                    52 (bytes)
  Size of program headers:                32 (bytes)
  Number of program headers:               2
  Size of section headers:                40 (bytes)
  Number of section headers:              26
  Section header string table index:      25
```

QEMU loader loads 'setup.bin' at address 0x10000



Entry Point of Linux: start_of_setup - GDB

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ ls
conf  gdb-files  my.log  out  README.md  scripts  src
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-gdb.sh
```

```
IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

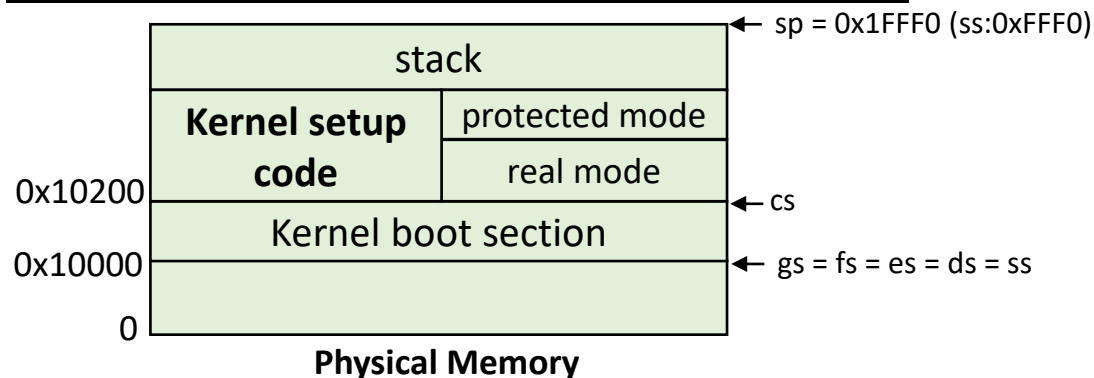
-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]

-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <1>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>

-----[ CODE ]-----
0x10200:    jmp     0x1026c
0x10202:    rex.W
0x10203:    fs jnb 0x10259
0x10206:    lar     (%rax),%eax
0x10209:    add     %al, (%rax)
0x1020b:    add     %al, (%rax)
0x1020d:    adc     %ah, 0x33(%rax)
0x10210:    mov     $0x81, %al
0x10212:    add     %al, 0x100000(%rax)
0x10218:    add     %dl, (%rax)
0x0000000000000000 in ?? ()
(gdb)
```

near jump

Boot loader loads 'setup.bin' at address 0x10000



```
(gdb) si
-----[ STACK ]-----
0000 0000 0000 0000 0000 0000 0000 0000
6165 6C72 7079 6972 746E 3D6B 6573 6972

-----[ DS:SI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D8 8E C0 8E D0 31 E4 FB .....1..
00010010: FC BE 2D 00 AC 20 C0 74 09 B4 0E BB 07 00 CD 10 ..-....t.....
00010020: EB F2 31 C0 CD 16 CD 19 EA F0 FF 00 F0 55 73 65 ..1.....Use
00010030: 20 61 20 62 6F 6F 74 20 6C 6F 61 64 65 72 2E 0D .a.boot.loader..

-----[ ES:DI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D8 8E C0 8E D0 31 E4 FB .....1..
00010010: FC BE 2D 00 AC 20 C0 74 09 B4 0E BB 07 00 CD 10 ..-....t.....
00010020: EB F2 31 C0 CD 16 CD 19 EA F0 FF 00 F0 55 73 65 ..1.....Use
00010030: 20 61 20 62 6F 6F 74 20 6C 6F 61 64 65 72 2E 0D .a.boot.loader..

-----[ CPU ]-----
AX: 1020 BX: 0000 CX: 0000 DX: 0000
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000

IP: 006C RIP:0000006C
CS:IP: 1020:006C (0x1026C)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]

-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <1>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>

-----[ CODE ]-----
0x1026c:    mov     %ds,%eax
0x1026e:    mov     %eax,%es
0x10270:    cld
0x10271:    mov     %ss,%edx
0x10273:    cmp     %eax,%edx
0x10275:    mov     %esp,%edx
0x10277:    je      0x1028f
0x10279:    mov     $0x6f64a10,%edx
0x1027e:    adc     %eax, (%rdx)
0x10280:    xorb    $0x16, -0x75(%rsp,%rax,1)
0x000000000000006c in ?? ()
(gdb)
```

Entry Point of Linux: start_of_setup - GDB

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ ls
conf  gdb-files  my.log  out  README.md  scripts  src
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-gdb.sh
```

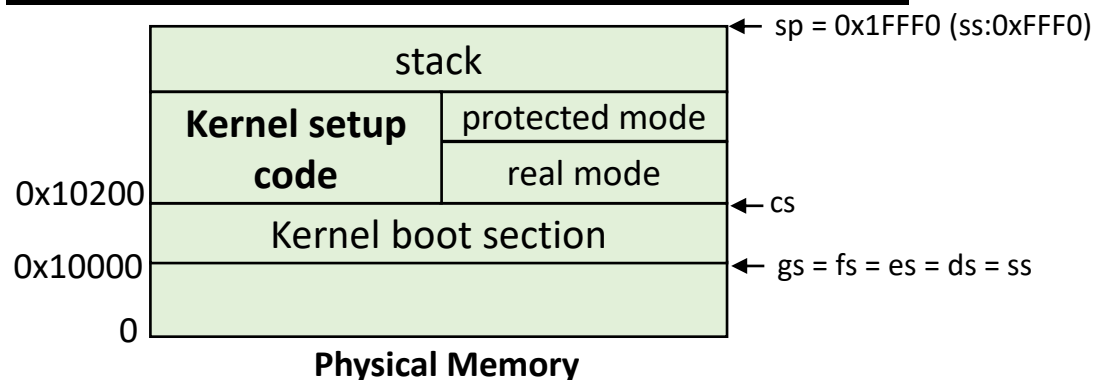
```
IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]

-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <1>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>

-----[ CODE ]-----
0x10200:  jmp     0x1026c
0x10202:  rex.W
0x10203:  fs jnb  0x10259
0x10206:  lar     (%rax),%eax
0x10209:  add     %al, (%rax)
0x1020b:  add     %al, (%rax)
0x1020d:  adc     %ah, 0x33(%rax)
0x10210:  mov     $0x81,%al
0x10212:  add     %al, 0x100000(%rax)
0x10218:  add     %dl, (%rax)
0x0000000000000000 in ?? ()
(gdb)
```

Boot loader loads 'setup.bin' at address 0x10000



```
(gdb) si
-----[ STACK ]-----
0000 0000 0000 0000 0000 0000 0000
6165 6C72 7079 6972 746E 3D6B 6573

-----[ DS:SI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D
00010010: FC BE 2D 00 AC 20 C0 74 0
00010020: EB F2 31 C0 CD 16 CD 19 E
00010030: 20 61 20 62 6F 6F 74 20 6

-----[ ES:DI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D
00010010: FC BE 2D 00 AC 20 C0 74 0
00010020: EB F2 31 C0 CD 16 CD 19 E
00010030: 20 61 20 62 6F 6F 74 20 6

-----[ CPU ]-----
AX: 1020 BX: 0000 CX: 0000 DX: 0000
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000

IP: 006C RIP:0000006C
CS:IP: 1020:006C (0x1026C)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)

-----[ CPU C ]-----
cr0          0x10
cr3          0x0
efer         0x0

-----[ eflag ]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF
ID <0>  VIP <0>  VIF <0>  AC <0>  VM

-----[ CODE ]-----
0x1026c:  mov     %ds,%eax
0x1026e:  mov     %eax,%es
0x10270:  cld
0x10271:  mov     %ss,%edx
0x10273:  cmp     %eax,%edx
0x10275:  mov     %esp,%edx
0x10277:  je      0x1028f
0x10279:  mov     $0x6f64a10,%edx
0x1027e:  adc     %eax, (%rdx)
0x10280:  xorb    $0x16, -0x75(%rsp,%rax,1)
0x0000000000000006c in ?? ()
(gdb)
```

The diagram shows the memory layout of **setup.bin** starting at address 0x0:

- .btext**: 0x0
- .bsdata**
- Part 1 of '.header'**: 0x1F1
- Part 2 of '.header'**: 0x200
- .entrytext**
- .inittext**
- .initdata**
- .text**
- .text32**
- .rodata**
- .videocards**
- .data**
- .signature**
- .bss**

Control flow annotations:

- near jump 1**: From 0x200 to .entrytext
- near call 2**: From .entrytext to .inittext
- long jump 3**: From .inittext to .text

Entry Point of Linux: start_of_setup – short jump

```
# offset 512, entry point
.globl _start
_start:
    # Explicitly enter this as bytes, or the assembler
    # tries to generate a 3-byte jump here, which causes
    # everything else to push off to the wrong offset.
    .byte 0xeb          # short (2-byte) jump
    .byte start_of_setup-1f

1:

    # Part 2 of the header, from the old setup.S

    .ascii "Hdrs"       # header signature
    .word 0x020f         # header version number (>= 0x0105)
                        # or else old loadlin-1.5 will fail)
...
# End of setup header #####

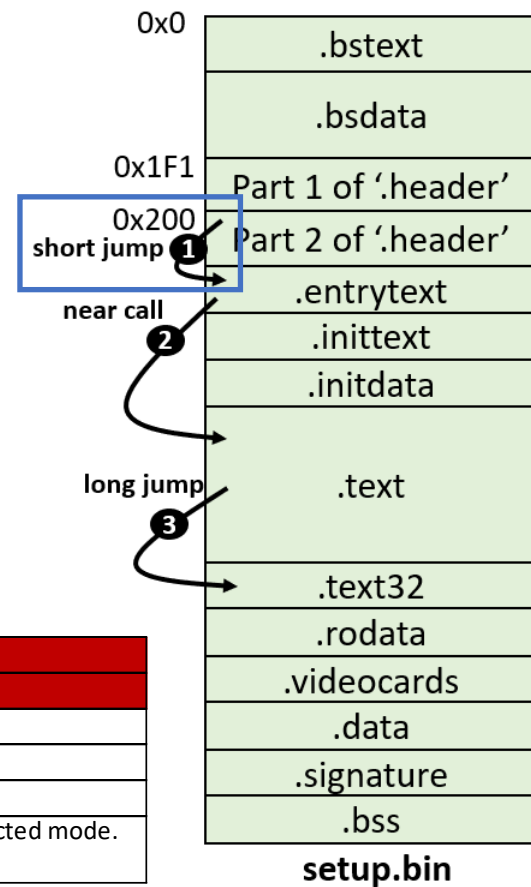
.section ".entrytext", "ax"
start_of_setup:
# Force %es = %ds
movw %ds, %ax
movw %ax, %es
cld

arch/x86/boot/header.S 289,1
```

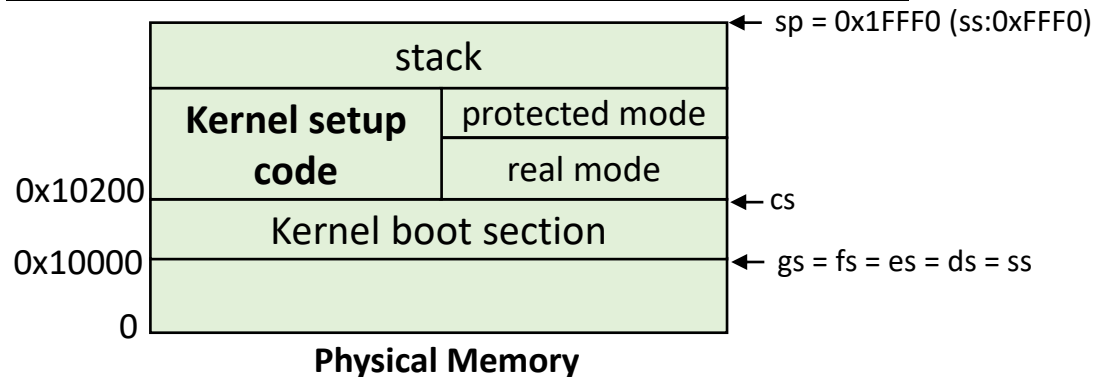
JMP—Jump

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
EB cb	JMP rel8	D	Valid	Valid	Jump short, RIP = RIP + 8-bit displacement sign extended to 64-bits
E9 cw	JMP rel16	D	N.S.	Valid	Jump near, relative, displacement relative to next instruction. Not supported in 64-bit mode.

- cb, cw, cd, cp, co, ct** — A 1-byte (cb), 2-byte (cw), 4-byte (cd), 6-byte (cp), 8-byte (co) or 10-byte (ct) value following the opcode. This value is used to specify a code offset and possibly a new value for the code segment register.



Boot loader loads 'setup.bin' at address 0x10000



".header": Real-mode kernel header		
Offset/Size	Name	Description
0x1F1/1	setup_sects	The size of the setup in sectors
0x01FE/2	boot_flag	magic number: 0xAA55
0x200/2	jump	Jump instruction
0x214/4	code32_start	Boot loader hook: The address to jump to in protected mode. Default: 0x100000

Entry Point of Linux: start_of_setup – short jump

```
# offset 512, entry point

.globl _start

_start:
    # Explicitly enter this as bytes, or the assembler
    # tries to generate a 3-byte jump here, which causes
    # everything else to push off to the wrong offset.
    .byte 0xeb          # short (2-byte) jump
    .byte start_of_setup-1f

1:

    # Part 2 of the header, from the old setup.S

    .ascii "Hdrs"       # header signature
    .word 0x020f         # header version number (>= 0x0105)
                        # or else old loadlin-1.5 will fail)
...
# End of setup header #####

.section ".entrytext", "ax"
start_of_setup:
# Force %es = %ds
movw %ds, %ax
movw %ax, %es
cld
```

arch/x86/boot/header.S

```
$ objdump -M -i8086 -D arch/x86/boot/setup.elf
arch/x86/boot/setup.elf:      file format elf32-i386
```

Disassembly of section .btext:

...

00000200 <start>:

200: eb 6a 0x26c - 0x202 = 0x6a jmp 26c <start_of_setup>

202: 48 dec %eax

...

Disassembly of section .entrytext:

0000026c <start_of_setup>:

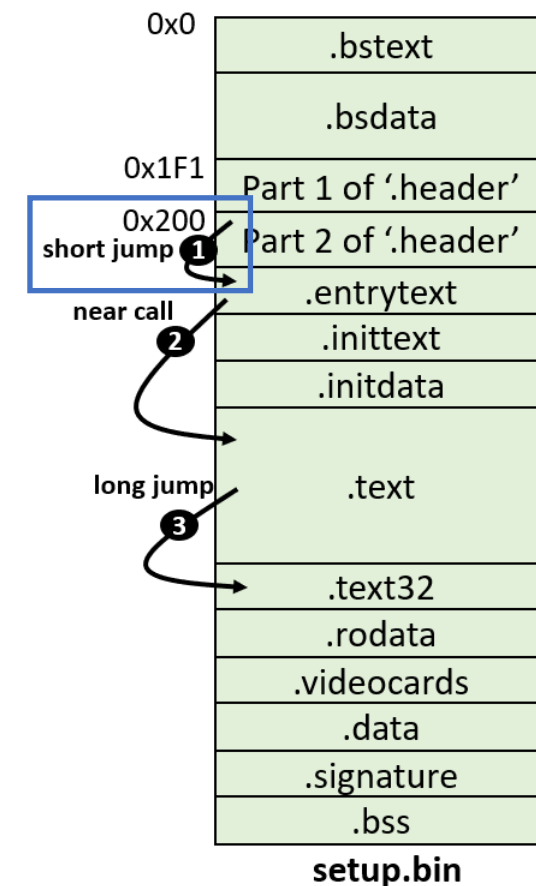
26c: 8c d8 mov %ds, %eax

...

JMP—Jump

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
EB cb	JMP rel8	D	Valid	Valid	Jump short, RIP = RIP + 8-bit displacement sign extended to 64-bits
E9 cw	JMP rel16	D	N.S.	Valid	Jump near, relative, displacement relative to next instruction. Not supported in 64-bit mode.

- cb, cw, cd, cp, co, ct** — A 1-byte (cb), 2-byte (cw), 4-byte (cd), 6-byte (cp), 8-byte (co) or 10-byte (ct) value following the opcode. This value is used to specify a code offset and possibly a new value for the code segment register.



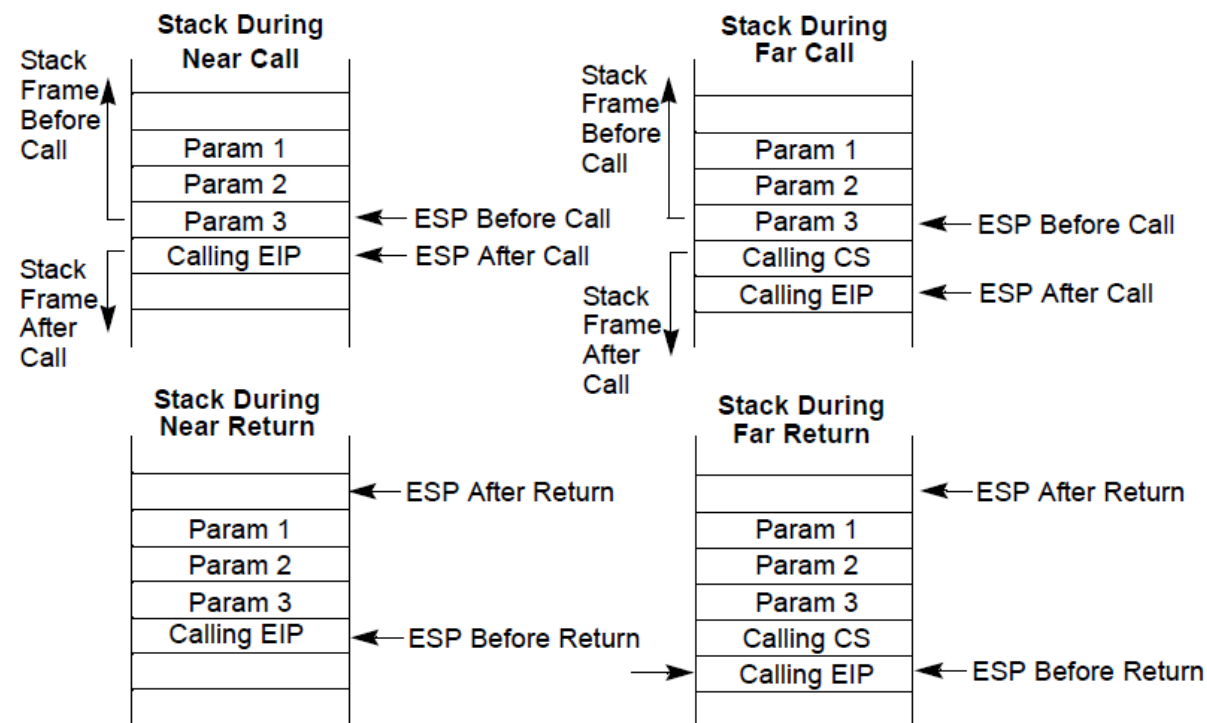
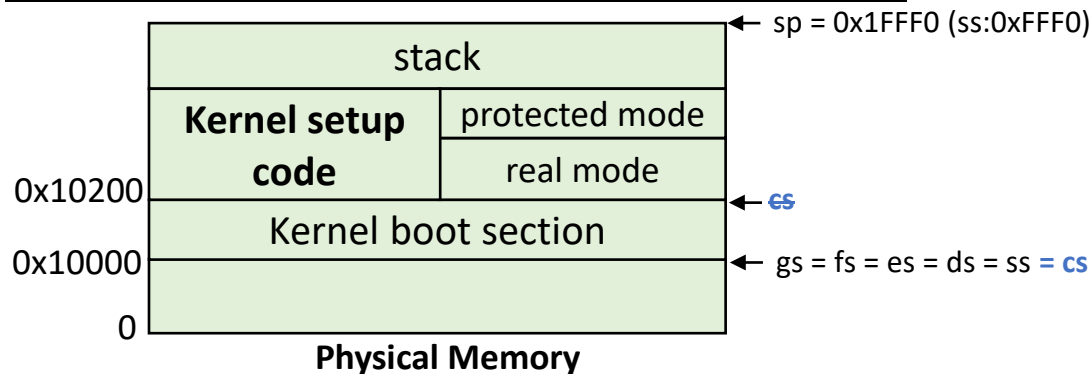
Entry Point of Linux: start_of_setup

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
    6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

lretw instruction: Far Return Operation
'l' prefix: far control transfer
'w' suffix: word (16 bits)

Boot loader loads 'setup.bin' at address 0x10000



Note: On a near or far return, parameters are released from the stack based on the optional n operand in the RET n instruction.

Figure 6-2. Stack on Near and Far Calls

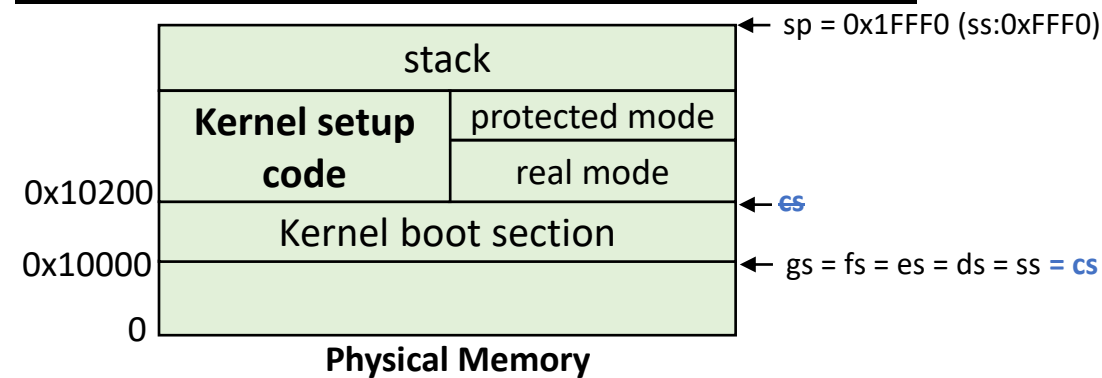
Entry Point of Linux: start_of_setup

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

lretw instruction: Far Return Operation
'l' prefix: far control transfer
'w' suffix: word (16 bits)

Boot loader loads 'setup.bin' at address 0x10000



```
$ objdump -M i8086 -D arch/x86/boot/setup.elf
0000026c <start_of_setup>:
...
1 29e: 1e                push    %ds
2 29f: 68 a3 02          push    $0x2a3
3 2a2: cb              lret
2a3: 66 81 3e d8 36 55 aa  cmpl    $0x5a5aaa55,0x36d8
2aa: 5a 5a
...
```

PUSH—Push Word, Doubleword or Quadword Onto the Stack

Opcode*	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
FF /6	PUSH <i>r/m16</i>	M	Valid	Valid	Push <i>r/m16</i> .
FF /6	PUSH <i>r/m32</i>	M	N.E.	Valid	Push <i>r/m32</i> .
FF /6	PUSH <i>r/m64</i>	M	Valid	N.E.	Push <i>r/m64</i> .
50+ <i>rw</i>	PUSH <i>r16</i>	0	Valid	Valid	Push <i>r16</i> .
50+ <i>rd</i>	PUSH <i>r32</i>	0	N.E.	Valid	Push <i>r32</i> .
50+ <i>rd</i>	PUSH <i>r64</i>	0	Valid	N.E.	Push <i>r64</i> .
6A <i>ib</i>	PUSH <i>imm8</i>	I	Valid	Valid	Push <i>imm8</i> .
2 68 <i>iw</i>	PUSH <i>imm16</i>	I	Valid	Valid	Push <i>imm16</i> .
68 <i>id</i>	PUSH <i>imm32</i>	I	Valid	Valid	Push <i>imm32</i> .
0E	PUSH CS	Z0	Invalid	Valid	Push CS.
16	PUSH SS	Z0	Invalid	Valid	Push SS.
1 1E	PUSH DS	Z0	Invalid	Valid	Push DS.
06	PUSH ES	Z0	Invalid	Valid	Push ES.
0F A0	PUSH FS	Z0	Valid	Valid	Push FS.
0F A8	PUSH GS	Z0	Valid	Valid	Push GS.

NOTES:

* See IA-32 Architecture Compatibility section below.

RET—Return from Procedure

Opcode*	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
C3	RET	Z0	Valid	Valid	Near return to calling procedure.
CB	RET	Z0	Valid	Valid	Far return to calling procedure.
3 C2 <i>iw</i>	RET <i>imm16</i>	I	Valid	Valid	Near return to calling procedure and pop <i>imm16</i> bytes from stack.
3 CA <i>iw</i>	RET <i>imm16</i>	I	Valid	Valid	Far return to calling procedure and pop <i>imm16</i> bytes from stack.

Entry Point of Linux: start_of_setup

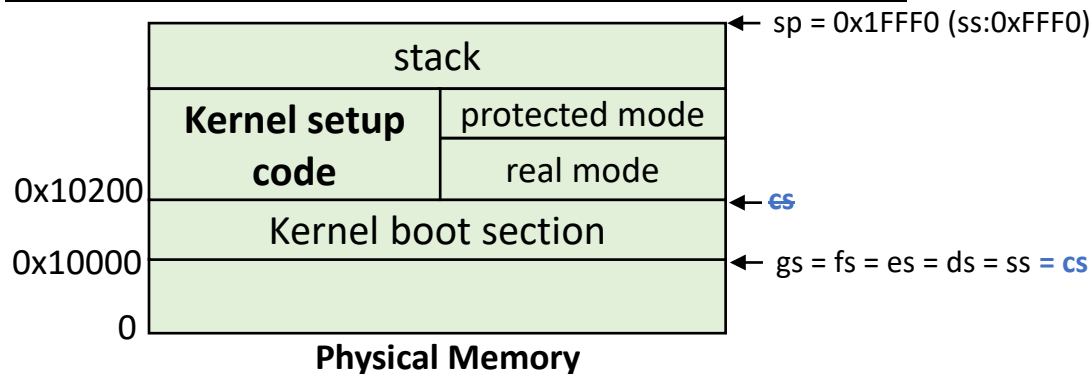
Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
    6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

lretw instruction: Far Return Operation

'l' prefix: far control transfer
'w' suffix: word (16 bits)

Boot loader loads 'setup.bin' at address 0x10000



After executing 'lretw'

```
(gdb) c
Continuing.

Thread 1 received signal SIGTRAP, Trace/breakpoint trap.
-----[ STACK ]-----
02A3 1000 0000 0000 0000 0000 0000 0000
0000 0000 6165 6C72 7079 6972 746E 3D6B
...
-----[ CPU ]-----
AX: 1000 BX: 0000 CX: 0000 DX: FFF0
SI: 0000 DI: 0000 SP: FFEC BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000
IP: 00A2 RIP:000000A2
CS:IP: 1020:00A2 (0x102A2)
SS:SP: 1000:FFEC (0x1FFEC)
SS:BP: 1000:0000 (0x10000)
-----[ CODE ]-----
0x102a2:    lret
0x102a3:    cmpw    $0x36d8, (%rsi)
...

(gdb) si
-----[ STACK ]-----
0000 0000 0000 0000 0000 0000 0000 0000
6165 6C72 7079 6972 746E 3D6B 6573 6972
...
-----[ CPU ]-----
AX: 1000 BX: 0000 CX: 0000 DX: FFF0
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1000 DS: 1000 ES: 1000 SS: 1000
IP: 02A3 RIP:000002A3
CS:IP: 1000:02A3 (0x102A3)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)
...
-----[ CODE ]-----
0x102a3:    cmpw    $0x36d8, (%rsi)
0x102a8:    push    %rbp
0x102a9:    stos     %al,%es:(%rdi)
0x102aa:    pop     %rdx
0x102ab:    pop     %rdx
0x102ac:    jne     0x102c5
0x102ae:    mov     $0x13b936e0,%edi
0x102b3:    rex.WX
0x102b4:    xor     %ax,%ax
0x102b7:    sub     %edi,%ecx
0x0000000000000002a3 in ?? ()
(gdb)
```

Entry Point of Linux: start_of_setup – Why to align CS?

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

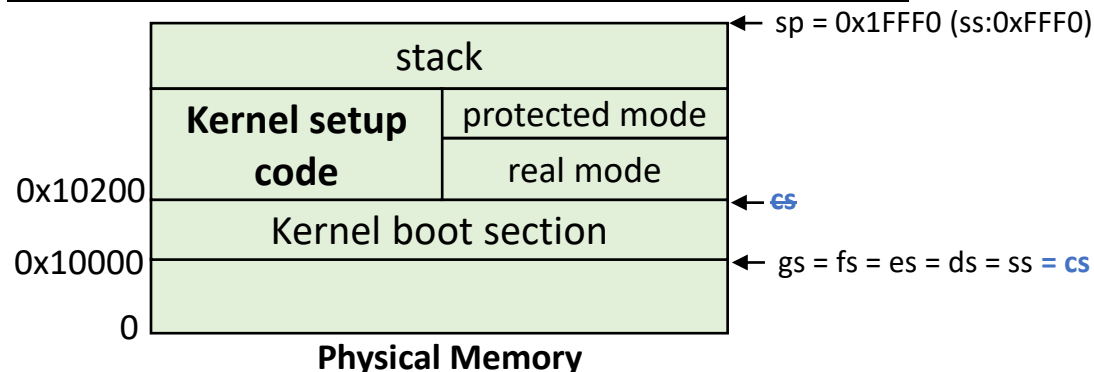
lretw instruction: Far Return Operation

'l' prefix: far control transfer
'w' suffix: word (16 bits)

```
.code16
.section ".inittext", "ax"
.globl  intcall
.type   intcall, @function
intcall:
...
/* Re-establish C environment invariants */
cld
movzwl  %sp, %esp
movw    %cs, %ax
movw    %ax, %ds
movw    %ax, %es
...
arch/x86/boot/bioscall.S
```

If cs is not align with ds, ds and es are incorrect after returning from 'intcall'.

Boot loader loads 'setup.bin' at address 0x10000



Entry Point of Linux: start_of_setup – data & bss section

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw %ds
    pushw $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

```
/*
 * setup.ld
 *
 * Linker script for the i386 setup code
 */
OUTPUT_FORMAT("elf32-i386")
OUTPUT_ARCH(i386)
ENTRY(_start)

SECTIONS
{
    . = 0;
    .bstext      : { *(.bstext) }
    .bsdata     : { *(.bsdata) }

    . = 495;
    .header     : { *(.header) }
    .entrytext  : { *(.entrytext) }
    .inittext   : { *(.inittext) }
    .initdata   : { *(.initdata) }
    __end_init = .;

    .text       : { *(.text .text.*) }
    .text32     : { *(.text32) }

    . = ALIGN(16);
    .rodata     : { *(.rodata*) }

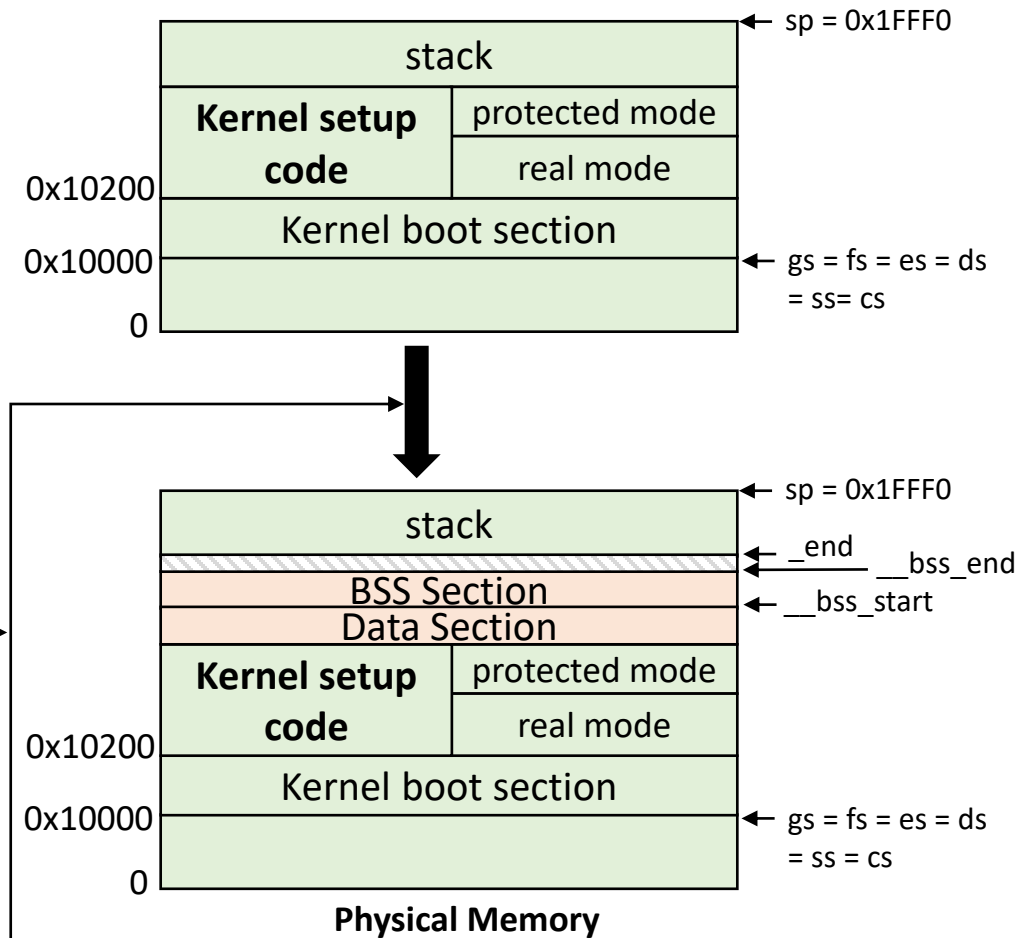
    ...

    . = ALIGN(16);
    .data       : { *(.data*) }

    .signature  : {
        setup_sig = .;
        LONG(0x5a5aaa55)
    }
}
```

```
. = ALIGN(16);
.bss      :
{
    __bss_start = .;
    *(.bss)
    __bss_end = .;
}
. = ALIGN(16);
__end = .;
```

Boot loader loads 'setup.bin' at address 0x10000



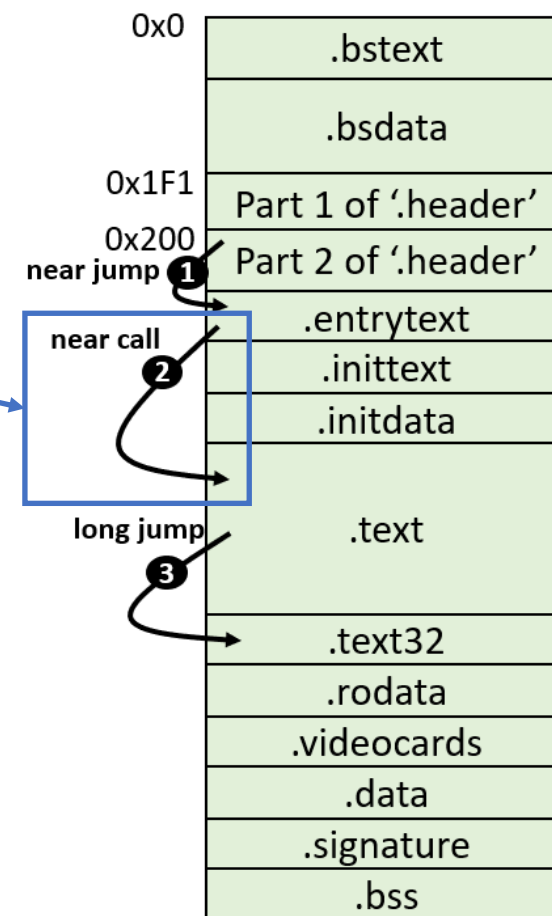
Entry Point of Linux: start_of_setup -> main()

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

```
    .section ".entrytext", "ax"
start_of_setup:
    ...

# Jump to C code (should not return)
    calll    main
```



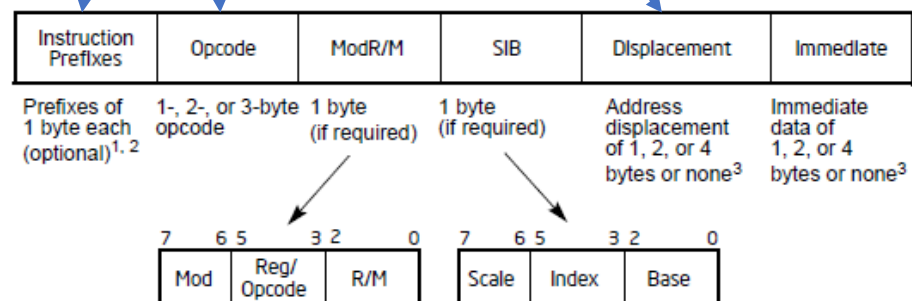
setup.bin

Entry Point of Linux: start_of_setup -> main()

```
.section ".entrytext", "ax"
start_of_setup:
...

# Jump to C code (should not return)
calll main
```

```
$ objdump -M i8086 -D arch/x86/boot/setup.elf
0000026c <start_of_setup>:
...
2bf: 66 e8 50 0d 00 00      calll 1015 <main>
000002c5 <setup_bad>:
2c5: 66 b8 e1 03 00 00      mov $0x3e1,%eax
2cb: 66 e8 f0 00 00 00      calll 3c1 <puts>
...
00001015 <main>:
1015: f3 0f 1e fb            endbr32
```



1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
3. Some rare instructions can take an 8B immediate or 8B displacement.

CALL—Call Procedure

Opcode	Instruction	Op/En	64-bit Mode	Compat/Leg Mode	Description
E8 cw	CALL <i>rel16</i>	D	N.S.	Valid	Call near, relative, displacement relative to next instruction.
E8 cd	CALL <i>rel32</i>	D	Valid	Valid	Call near, relative, displacement relative to next instruction. 32-bit displacement sign extended to 64-bits in 64-bit mode.
FF /2	CALL <i>r/m16</i>	M	N.E.	Valid	Call near, absolute indirect, address given in <i>r/m16</i> .
FF /2	CALL <i>r/m32</i>	M	N.E.	Valid	Call near, absolute indirect, address given in <i>r/m32</i> .
FF /2	CALL <i>r/m64</i>	M	Valid	N.E.	Call near, absolute indirect, address given in <i>r/m64</i> .
9A cd	CALL <i>ptr16:16</i>	D	Invalid	Valid	Call far, absolute, address given in operand.
9A cp	CALL <i>ptr16:32</i>	D	Invalid	Valid	Call far, absolute, address given in operand.
FF /3	CALL <i>m16:16</i>	M	Valid	Valid	Call far, absolute indirect address given in <i>m16:16</i> . In 32-bit mode: if selector points to a gate, then RIP = 32-bit zero extended displacement taken from gate; else RIP = zero extended 16-bit offset from far pointer referenced in the instruction.
FF /3	CALL <i>m16:32</i>	M	Valid	Valid	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = zero extended 32-bit offset from far pointer referenced in the instruction.
REX.W FF /3	CALL <i>m16:64</i>	M	Valid	N.E.	In 64-bit mode: If selector points to a gate, then RIP = 64-bit displacement taken from gate; else RIP = 64-bit offset from far pointer referenced in the instruction.

- **cb, cw, cd, cp, co, ct** — A 1-byte (cb), 2-byte (cw), 4-byte (cd), 6-byte (cp), 8-byte (co) or 10-byte (ct) value following the opcode. This value is used to specify a code offset and possibly a new value for the code segment register.

Figure 2-1. Intel 64 and IA-32 Architectures Instruction Format

Entry Point of Linux: start_of_setup -> main()

```
.section ".entrytext", "ax"
start_of_setup:
...

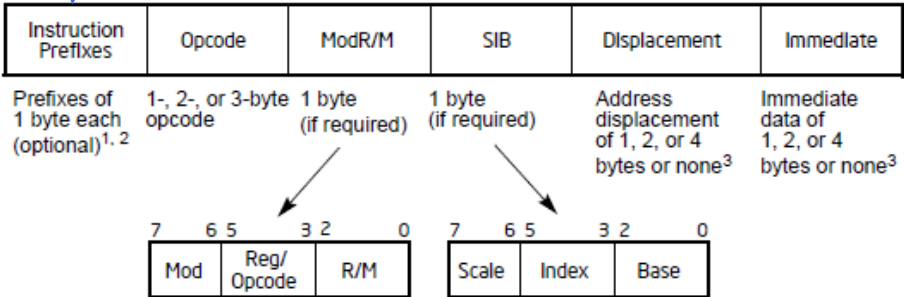
# Jump to C code (should not return)
calll main
```

```
$ objdump -M i8086 -D arch/x86/boot/setup.elf
0000026c <start_of_setup>:
...
2bf: 66 e8 50 0d 00 00      calll 1015 <main>

000002c5 <setup_bad>:
2c5: 66 b8 e1 03 00 00      mov $0x3e1,%eax
2cb: 66 e8 f0 00 00 00      calll 3c1 <puts>

...

00001015 <main>:
1015: f3 0f 1e fb            endbr32
```



1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
3. Some rare instructions can take an 8B immediate or 8B displacement.

Table 3-3. Effective Operand- and Address-Size Attributes

D Flag in Code Segment Descriptor	0	0	0	0	1	1	1	1
Operand-Size Prefix 66H	N	N	Y	Y	N	N	Y	Y
Address-Size Prefix 67H	N	Y	N	Y	N	Y	N	Y
Effective Operand Size	16	16	32	32	32	32	16	16
Effective Address Size	16	32	16	32	32	16	32	16

NOTES:

Y: Yes - this instruction prefix is present.
N: No - this instruction prefix is not present.

Figure 2-1. Intel 64 and IA-32 Architectures Instruction Format

Entry Point of Linux: start_of_setup -> main() -> copy_boot_params()

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
    %es = %ds (es for storing string -> es:di)
    setup %ss if %ds != %ss
    setup %sp if necessary
    set %cs = %ds by the following instructions:
        pushw    %ds
        pushw    $6f
        lretw
    6:
        xxx
    check signature of setup.bin
    clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
    |- copy_boot_params
    |- console_init
    |- init_heap
    |- validate_cpu
    |- detect_memory [e820 map]
    |- keyboard_init
    |- set_video [set video mode]
    |- go_to_protected_mode
```

- copy setup header into boot parameter block (*struct boot_params*: arch/x86/include/uapi/asm/bootparam.h)
 - `struct setup\_header hdr` in *boot_params*
 - Contain the same fields defined in Linux boot protocol. Those fields are configured by boot loader and kernel compile/build time

```
static void copy_boot_params(void)
{
    ...
    memcpy(&boot_params.hdr, &hdr, sizeof(hdr));
    ...
}
arch/x86/boot/main.c
```

```
        .globl    hdr
hdr:
setup_sects:    .byte 0                /* Filled in by build.c */
root_flags:    .word ROOT_RDONLY
syssize:       .long 0                /* Filled in by build.c */
ram_size:      .word 0                /* Obsolete */
vid_mode:      .word SVGA_MODE
root_dev:      .word 0                /* Filled in by build.c */
boot_flag:     .word 0xAA55

        # offset 512, entry point

        .globl    _start
_start:
        # Explicitly enter this as bytes, or the assembler
        # tries to generate a 3-byte jump here, which causes
        # everything else to push off to the wrong offset.
        .byte    0xeb                # short (2-byte) jump
        .byte    start_of_setup-1f

arch/x86/boot/header.S
```

Entry Point of Linux: start_of_setup -> main() -> console_init() – (1/2)

Call Path

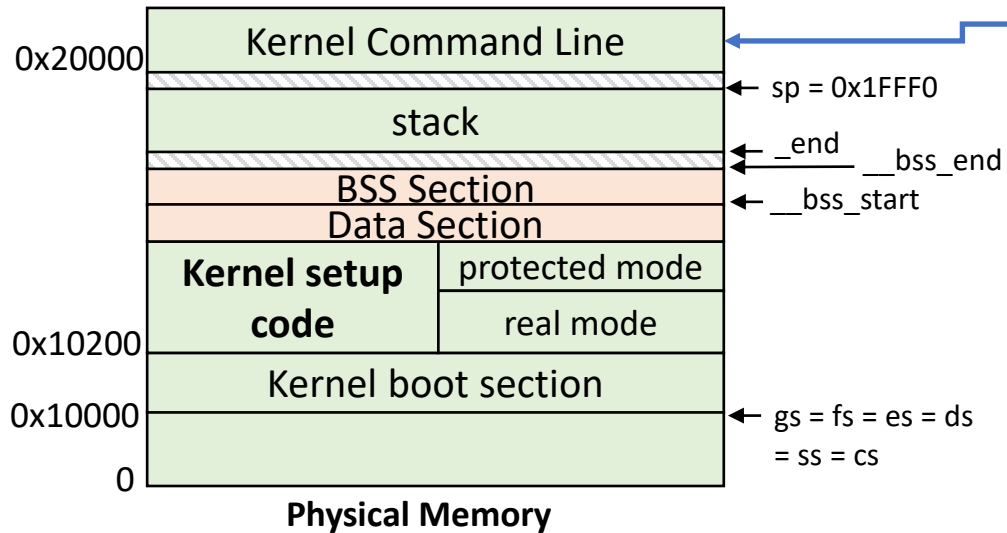
```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

- console_init()
 - Initialize the corresponding serial port if command line has 'earlyprintk' parameter

```
void x86_load_linux(X86MachineState *x86ms,
                   FWCfgState *fw_cfg,
                   int acpi_data_size,
                   bool pvh_enabled,
                   bool linuxboot_dma_enabled)
{
    ...
    if (protocol < 0x200 || !(header[0x211] & 0x01)) {
        /* Low kernel */
        real_addr    = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr    = 0x10000;
    } else if (protocol < 0x202) {
        /* High but ancient kernel */
        real_addr    = 0x90000;
        cmdline_addr = 0x9a000 - cmdline_size;
        prot_addr    = 0x10000;
    } else {
        /* High and recent kernel */
        real_addr    = 0x10000;
        cmdline_addr = 0x20000;
        prot_addr    = 0x10000;
    }

    ...
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_ADDR, prot_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_KERNEL_SIZE, kernel_size);

    fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_ADDR, real_addr);
    fw_cfg_add_i32(fw_cfg, FW_CFG_SETUP_SIZE, setup_size);
    ...
}
```



QEMU Loader

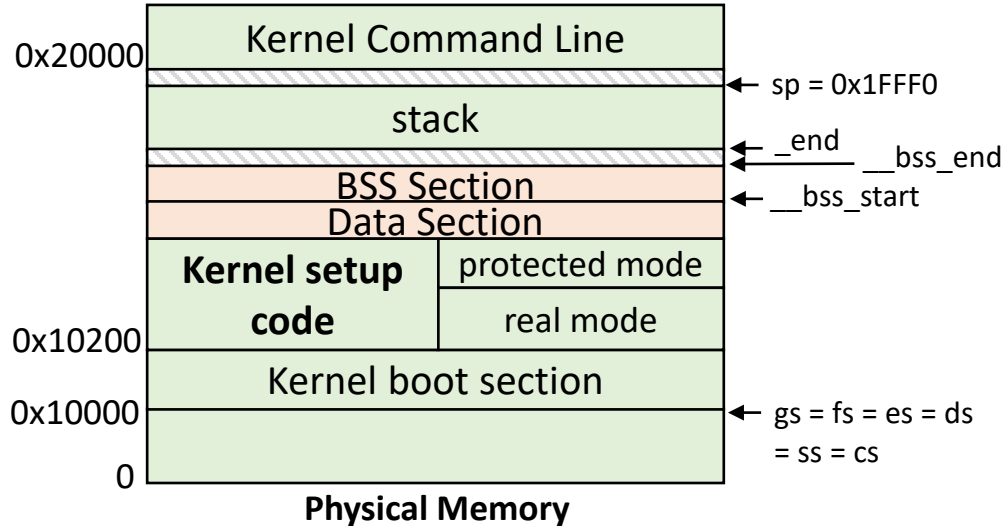
Entry Point of Linux: start_of_setup -> main() -> console_init() – (2/2)

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

- console_init()
 - Initialize the corresponding serial port if command line has 'earlyprintk' parameter

```
(gdb) info b
Num      Type          Disp Enb Address      What
4        breakpoint    keep y      0x0001109c in init_heap
                                                at /home/adrian/git-repo/gdb-linux-re
al-mode/src/linux-5.11/arch/x86/boot/main.c:145
(gdb) p /x boot_params.hdr
$1 = {setup_sects = 0x1b, root_flags = 0x1, syssize = 0x14a9b, ram_size = 0x0,
vid_mode = 0xffff, root_dev = 0x0, boot_flag = 0xaa55, jump = 0x6aeb,
header = 0x53726448, version = 0x20f, realmode_swch = 0x0,
start_sys_seg = 0x1000, kernel_version = 0x3360, type_of_loader = 0xb0,
loadflags = 0x81, setup_move_size = 0x8000, code32_start = 0x100000,
ramdisk_image = 0x7fd81000, ramdisk_size = 0x15e8f8, bootsect_kludge = 0x0,
heap_end_ptr = 0xfe00, ext_loader_ver = 0x0, ext_loader_type = 0x0,
cmd_line_ptr = 0x20000, initrd_addr_max = 0x7fffffff,
kernel_alignment = 0x200000, relocatable_kernel = 0x1, min_alignment = 0x15,
xloadflags = 0x63, cmdline_size = 0x7ff, hardware_subarch = 0x0,
hardware_subarch_data = 0x0, payload_offset = 0x2e0,
payload_length = 0x142f99, setup_data = 0x0, pref_address = 0x1000000,
init_size = 0xbb3000, handover_offset = 0x0, kernel_info_offset = 0x14a0b0}
```



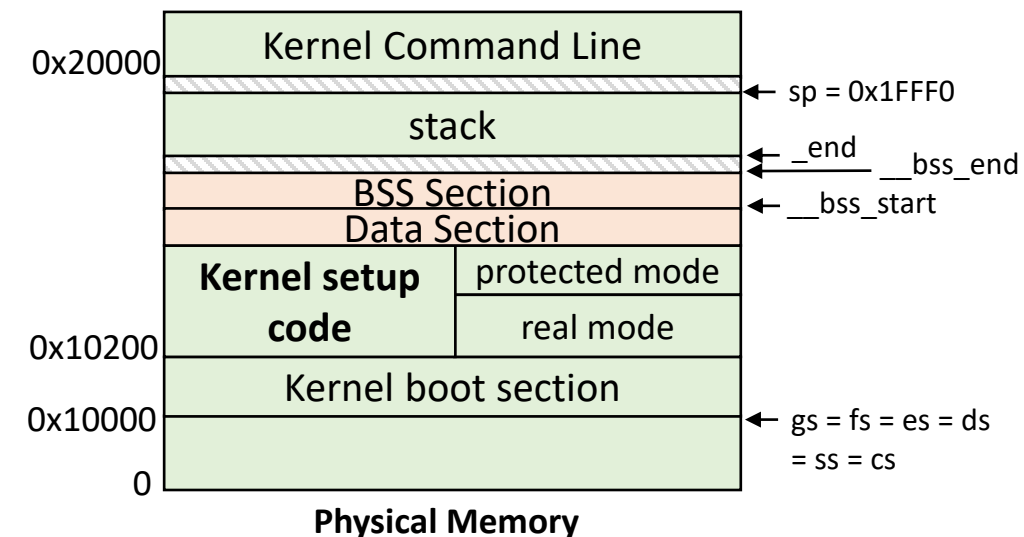
```
(gdb) x /s 0x20000
0x20000:      "earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr"
(gdb) x /s boot_params.hdr->cmd_line_ptr
0x20000:      "earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr"
```

Entry Point of Linux: start_of_setup -> main() -> validate_cpu() & detect_memory()

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw    %ds
    pushw    $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

- init_heap()
 - Discussion in the next few slides
- validate_cpu()
 - Check CPU flags
 - Check if long mode (x86_64) is available
 - [AMD – K7 Processor] Turn SSE+SSE2 on if they are missing in CPU flags
- detect_memory()
 - Use different program interfaces (0xe820, 0xe801 and 0x88) for memory detection
 - 0xe820
 - Fill boot_params.e820_table based on e820 map



Entry Point of Linux: start_of_setup -> main() -> init_heap() (1/2)

Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw %ds
    pushw $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
    |- copy_boot_params
    |- console_init
    |- init_heap
    |- validate_cpu
    |- detect_memory [e820 map]
    |- keyboard_init
    |- set_video [set video mode]
    |- go_to_protected_mode
```

- init_heap
 - Setup the heap space if the 'CAN_USE_HEAP' flag (0x80) is set in loadflags of the kernel setup header.

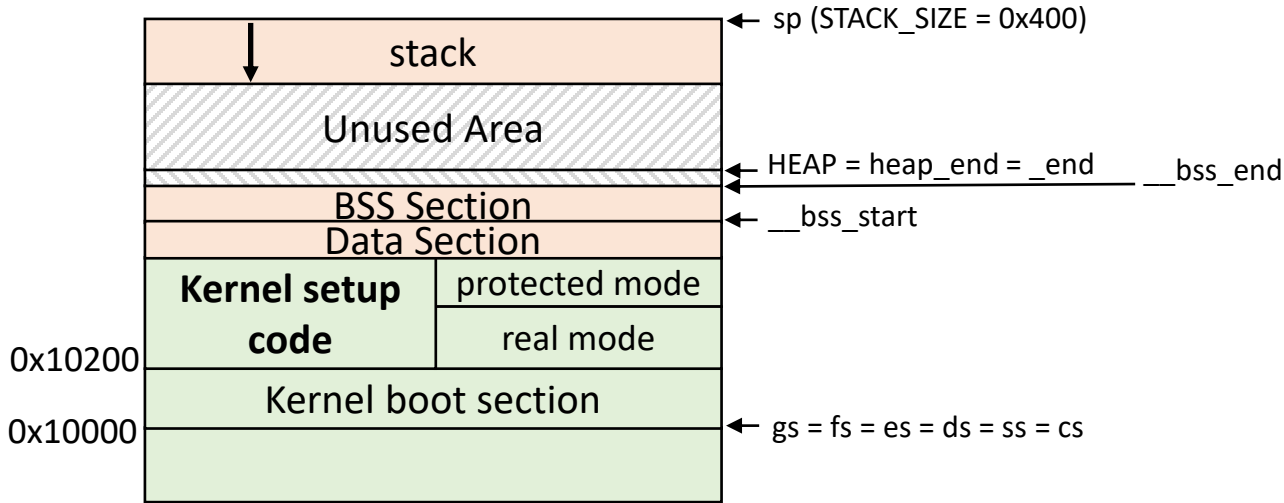
```
(gdb) info b
Num      Type      Disp Enb Address      What
4        breakpoint keep y   0x0001109c in init_heap
                                                at /home/adrian/git-repo/gdb-linux-re
al-mode/src/linux-5.11/arch/x86/boot/main.c:145
(gdb) p /x boot_params.hdr
$1 = {setup_sects = 0x1b, root_flags = 0x1, syssize = 0x14a9b, ram_size = 0x0,
      vid_mode = 0xffff, root_dev = 0x0, boot_flag = 0xaa55, jump = 0x6aeb,
      header = 0x53726448, version = 0x20f, realmode_swch = 0x0,
      start_sys_seg = 0x1000, kernel_version = 0x3360, type_of_loader = 0xb0,
      loadflags = 0x81, setup_move_size = 0x8000, code32_start = 0x100000,
      ramdisk_image = 0x7fd81000, ramdisk_size = 0x15e8f8, bootsect_kludge = 0x0,
      heap_end_ptr = 0xfe00, ext_loader_ver = 0x0, ext_loader_type = 0x0,
      cmd_line_ptr = 0x20000, initrd_addr_max = 0x7fffffff,
      kernel_alignment = 0x200000, relocatable_kernel = 0x1, min_alignment = 0x15,
      xloadflags = 0x63, cmdline_size = 0x7ff, hardware_subarch = 0x0,
      hardware_subarch_data = 0x0, payload_offset = 0x2e0,
      payload_length = 0x142f99, setup_data = 0x0, pref_address = 0x1000000,
      init_size = 0xbb3000, handover_offset = 0x0, kernel_info_offset = 0x14a0b0}
```

Entry Point of Linux: start_of_setup -> main() -> init_heap() (2/2)

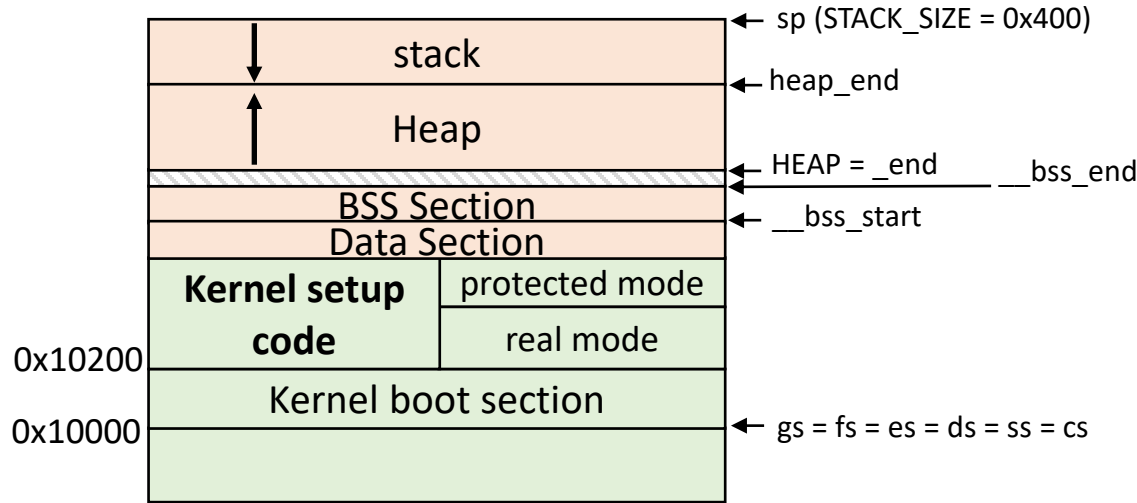
Call Path

```
start_of_setup [arch/x86/boot/header.S] -- ** real mode **
%es = %ds (es for storing string -> es:di)
setup %ss if %ds != %ss
setup %sp if necessary
set %cs = %ds by the following instructions:
    pushw %ds
    pushw $6f
    lretw
6:
    xxx
check signature of setup.bin
clear bss ('stosl' instruction: Store String)
|- main [arch/x86/boot/main.c] -- ** real mode **
|- copy_boot_params
|- console_init
|- init_heap
|- validate_cpu
|- detect_memory [e820 map]
|- keyboard_init
|- set_video [set video mode]
|- go_to_protected_mode
```

```
(gdb) info b
Num      Type      Disp Enb Address      What
4        breakpoint keep y   0x0001109c in init_heap
          at /home/adrian/git-repo/gdb-linux-re
al-mode/src/linux-5.11/arch/x86/boot/main.c:145
(gdb) p /x boot_params.hdr
$1 = {setup_sects = 0x1b, root_flags = 0x1, syssize = 0x14a9b, ram_size = 0x0,
      vid_mode = 0xffff, root_dev = 0x0, boot_flag = 0xaa55, jump = 0x6aeb,
      header = 0x53726448, version = 0x20f, realmode_swch = 0x0,
      start_sys_seg = 0x1000, kernel_version = 0x3360, type_of_loader = 0xb0,
      loadflags = 0x81, setup_move_size = 0x8000, code32_start = 0x100000,
      ramdisk_image = 0x7fd81000, ramdisk_size = 0x15e8f8, bootsect_kludge = 0x0,
      heap_end_ptr = 0xfe00, ext_loader_ver = 0x0, ext_loader_type = 0x0,
      cmd_line_ptr = 0x20000, initrd_addr_max = 0x7fffffff,
      kernel_alignment = 0x200000, relocatable_kernel = 0x1, min_alignment = 0x15,
      xloadflags = 0x63, cmdline_size = 0x7ff, hardware_subarch = 0x0,
      hardware_subarch_data = 0x0, payload_offset = 0x2e0,
      payload_length = 0x142f99, setup_data = 0x0, pref_address = 0x1000000,
      init_size = 0xbb3000, handover_offset = 0x0, kernel_info_offset = 0x14a0b0}
```



No heap



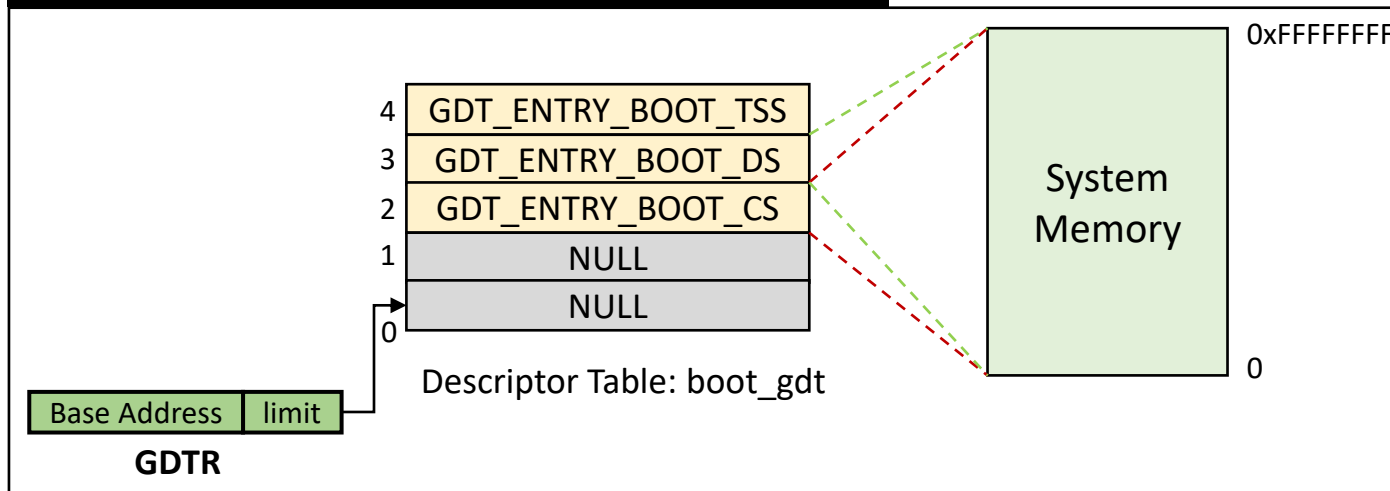
heap: allocate heap if CAN_USE_HEAP' flag (0x80) is set

go_to_protected_mode

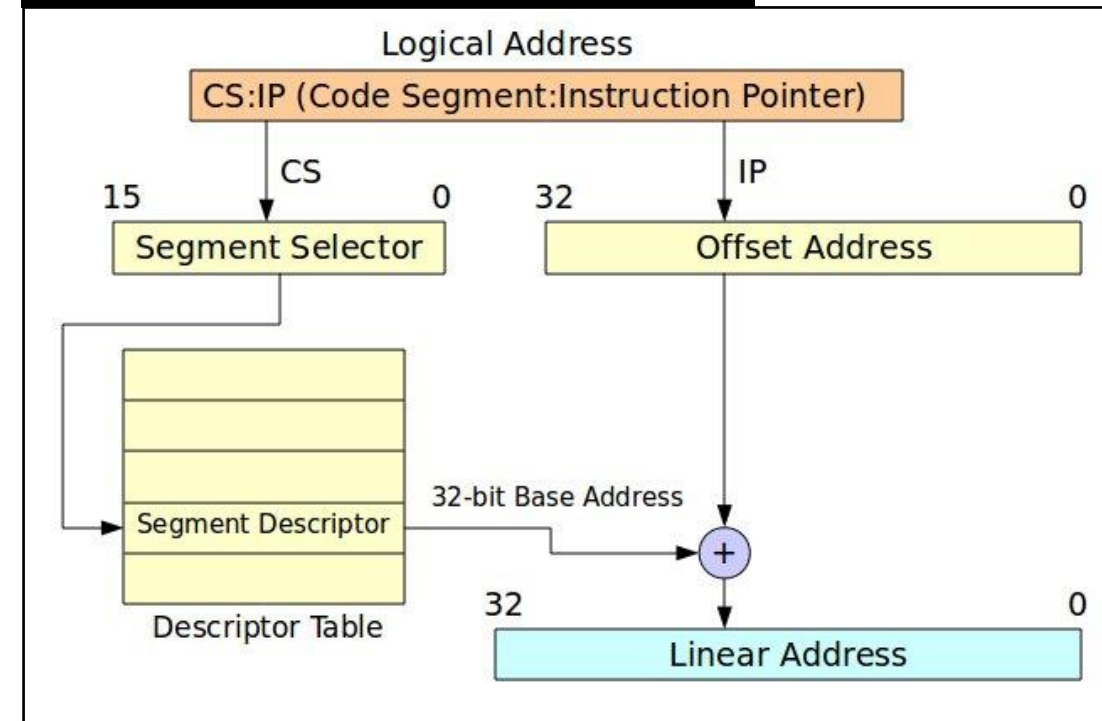
Call Path

```
go_to_protected_mode [arch/x86/boot/pm.c] -- ** real mode **  
|- realmode_switch_hook [disable local interrupt and NMI]  
|- enable_a20  
|- reset_coprocessor  
|- mask_all_interrupts [Disable all interrupts at the legacy PIC]  
|- setup_idt  
|- setup_gdt  
|- protected_mode_jump (boot_params.hdr.code32_start,  
                        (u32)&boot_params + (ds() << 4))
```

setup_gdt(): Setup 4G memory space for CS/DS



x86 Segmentation: Address Translation



protected_mode_jump (1/6)

```
protected_mode_jump [arch/x86/boot/pmjump.S] ** real mode **  
    Save cs to ebx in linear address (Ex: cs << 4)  
    Calculate relocation address for '.Lin_pm32' based on ebx  
    movw $__BOOT_DS, %cx  
    movw $__BOOT_TSS, %di  
    Enable protected mode by set bit 0 of cr0  
    ljmp1 $__BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```

protected_mode_jump – ljmp instruction: ignore '.Lin_pm32' relocation (2/6)

```
protected_mode_jump [arch/x86/boot/pmjump.S] ** real mode **  
Save cs to ebx in linear address (Ex: cs << 4)  
Calculate relocation address for '.Lin_pm32' based on ebx  
movw $__BOOT_DS, %cx  
movw $__BOOT_TSS, %di  
Enable protected mode by set bit 0 of cr0  
ljmpl $__BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```

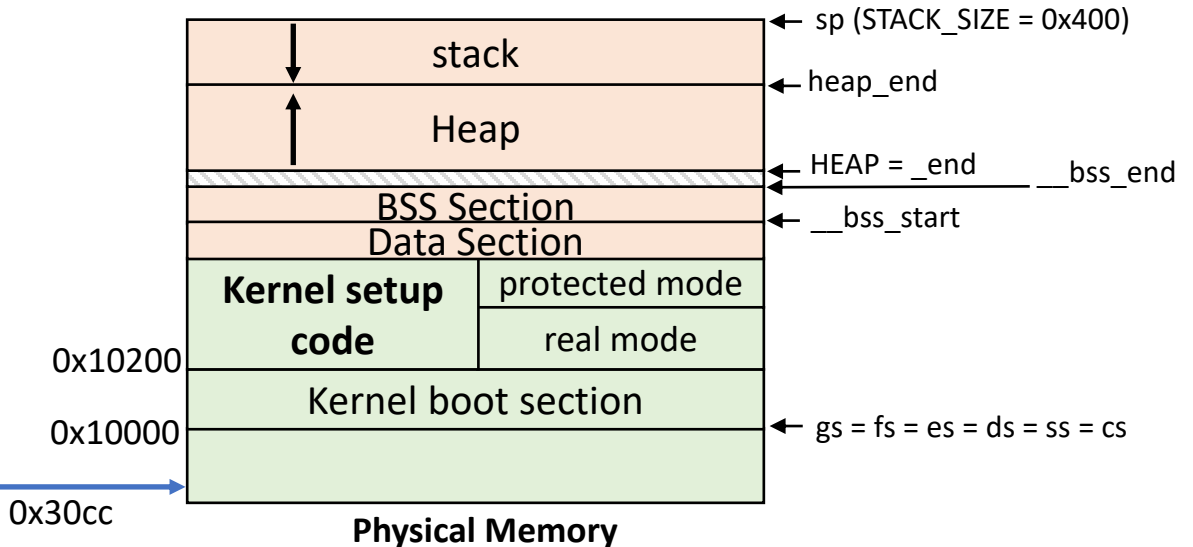
```
SYM_FUNC_START_NOALIGN(protected_mode_jump)  
    movl    %edx, %esi                # Pointer to boot_params table  
xorl    %ebx, %ebx  
movw    %cs, %bx  
shll    $4, %ebx  
addl    %ebx, 2f  
1:    jmp     1f                        # Short jump to serialize on 386/486  
  
    movw    $__BOOT_DS, %cx  
    movw    $__BOOT_TSS, %di  
  
    movl    %cr0, %edx  
    orb     $X86_CR0_PE, %dl          # Protected mode  
    movl    %edx, %cr0  
  
    # Transition to 32-bit mode  
    .byte   0x66, 0xea                # ljmp opcode  
    .long   $.Lin_pm32                # offset  
    .word   __BOOT_CS                 # segment  
2:    SYM_FUNC_END(protected_mode_jump)  
  
    .code32  
    .section ".text32", "ax"  
SYM_FUNC_START_LOCAL_NOALIGN(.Lin_pm32)  
    # Set up data segments for flat 32-bit mode  
    movl    %ecx, %ds
```

setup.bin generation

```
$ cat arch/x86/boot/.setup.bin.cmd  
cmd_arch/x86/boot/setup.bin := objcopy -O binary arch/x86/boot/setup.elf  
arch/x86/boot/setup.bin
```

```
$ objdump -M i8086 -D arch/x86/boot/setup.elf  
...  
000013aa <protected_mode_jump>:  
13aa: 66 89 d6                mov     %edx, %esi  
13ad: 66 31 db                xor     %ebx, %ebx  
13b0: 8c cb                  mov     %cs, %bx  
13b2: 66 c1 e3 04            shl     $0x4, %ebx  
13b6: 66 01 1e ce 13         add     %ebx, 0x13ce  
13bb: eb 00                  jmp     13bd <protected_mode_jump+0x13>  
13bd: b9 18 00              mov     $0x18, %cx  
13c0: bf 20 00              mov     $0x20, %di  
13c3: 0f 20 c2              mov     %cr0, %edx  
13c6: 80 ca 01              or      $0x1, %dl  
13c9: 0f 22 c2              mov     %edx, %cr0  
13cc: 66 ea cc 30 00 00 10    ljmpl   $0x10, $0x30cc  
13d3: 00  
...  
Disassembly of section .text32:  
  
000030cc <.text32>:  
30cc: 8e d9                mov     %cx, %ds  
30ce: 8e c1                mov     %cx, %es  
...
```

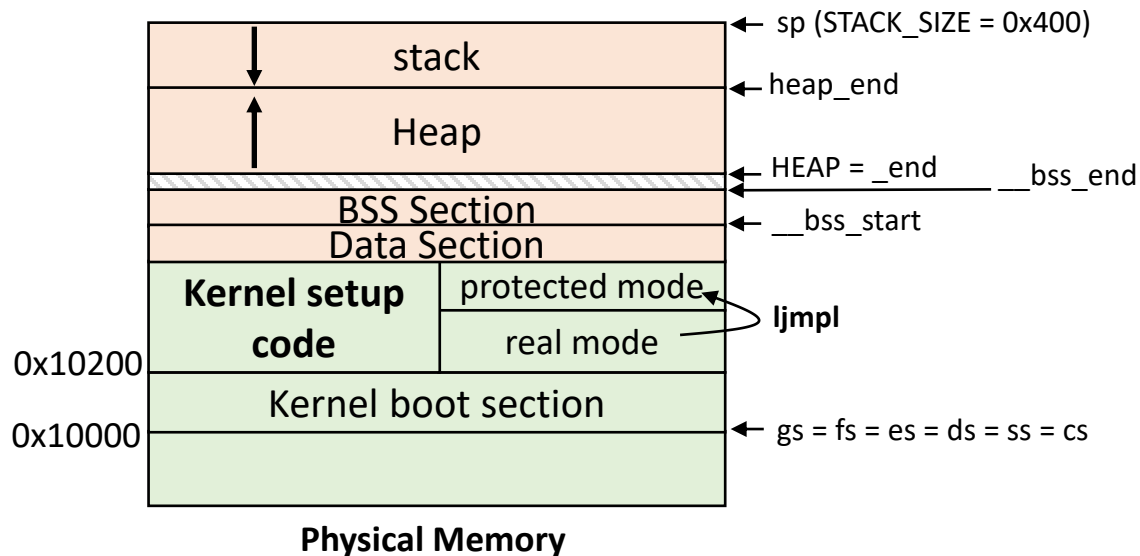
Jump (absolute address) to the wrong location



protected_mode_jump – ljmp instruction - relocation (3/6)

```
$ objdump -M i8086 -D arch/x86/boot/boots/setup.elf
...
000013aa <protected_mode_jump>:
    13aa: 66 89 d6          mov     %edx,%esi
    13ad: 66 31 db          xor     %ebx,%ebx
    13b0: 8c cb            mov     %cs,%bx
    13b2: 66 c1 e3 04       shl     $0x4,%ebx
    13b6: 66 01 1e ce 13    add     %ebx,0x13ce
    13bb: eb 00            jmp     13bd <protected_mode_jump+0x13>
    13bd: b9 18 00         mov     $0x18,%cx
    13c0: bf 20 00         mov     $0x20,%di
    13c3: 0f 20 c2         mov     %cr0,%edx
    13c6: 80 ca 01         or      $0x1,%dl
    13c9: 0f 22 c2         mov     %edx,%cr0
    13cc: 66 ea cc 30 00 00 10 1jmpl $0x10,$0x30cc
    13d3: 00
...
Disassembly of section .text32:

000030cc <.text32>:
    30cc: 8e d9            mov     %cx,%ds
    30ce: 8e c1            mov     %cx,%es
    ...
```



```

....
-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]
-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <0>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>
-----[ CODE ]-----
0x113b6:      add     %bx, (%rsi)
0x113b9:      (bad)
0x113ba:      adc     %ebx, %ebp

(gdb) disassemble /r 0x113b6,+5
Dump of assembler code from 0x113b6 to 0x113bb:
0x000000000000113b6: 66 01 1e          add     %bx, (%rsi)
0x000000000000113b9: ce              (bad)
0x000000000000113ba: 13 eb          adc     %ebx, %ebp

(gdb) x /2x 0x113ce
0x113ce:      0x000030cc      0x55660010

(gdb) si
...
-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]
-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <0>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>
-----[ CODE ]-----
0x113bb:      jmp     0x113bd
0x113bd:      mov     $0x20bf0018,%ecx
0x113c2:      add     %cl, (%rdi)

(gdb) x /2x 0x113ce
0x113ce:      0x000130cc      0x55660010

```

protected_mode_jump – ljmp instruction (4/6)

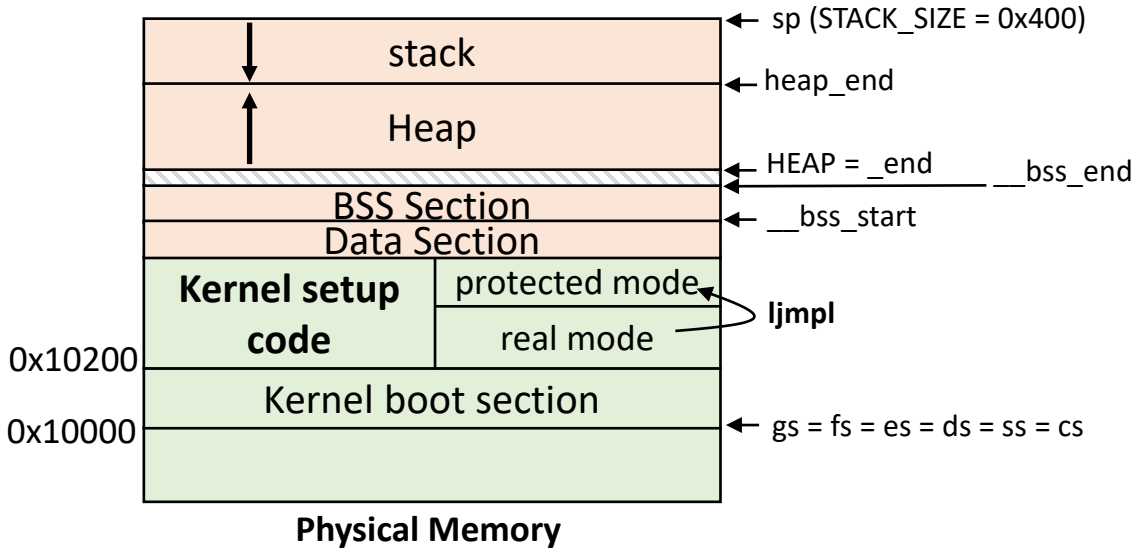
```
$ objdump -M i8086 -D arch/x86/boot/setup.elf
...
000013aa <protected_mode_jump>:
13aa: 66 89 d6      mov     %edx,%esi
13ad: 66 31 db      xor     %ebx,%ebx
13b0: 8c cb         mov     %cs,%bx
13b2: 66 c1 e3 04   shl     $0x4,%ebx
13b6: 66 01 1e ce 13 add     %ebx,0x13ce
13bb: eb 00        jmp     13bd <protected_mode_jump+0x13>
13bd: b9 18 00     mov     $0x18,%cx
13c0: bf 20 00     mov     $0x20,%di
13c3: 0f 20 c2     mov     %cr0,%edx
13c6: 80 ca 01     or      $0x1,%dl
13c9: 0f 22 c2     mov     %edx,%cr0
13cc: 66 ea cc 30 00 00 10 ljmp    $0x10,$0x30cc
13d3: 00
...
Disassembly of section .text32:
000030cc <.text32>:
30cc: 8e d9      mov     %cx,%ds
30ce: 8e c1      mov     %cx,%es
...
```

Relocation for absolute address of 'ljmpl'

mov %edx,%esi
xor %ebx,%ebx
mov %cs,%bx
shl \$0x4,%ebx
add %ebx,0x13ce

JMP—Jump

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
EB cb	JMP rel8	D	Valid	Valid	Jump short, RIP = RIP + 8-bit displacement sign extended to 64-bits
E9 cw	JMP rel16	D	N.S.	Valid	Jump near, relative, displacement relative to next instruction. Not supported in 64-bit mode.
E9 cd	JMP rel32	D	Valid	Valid	Jump near, relative, RIP = RIP + 32-bit displacement sign extended to 64-bits
FF /4	JMP r/m16	M	N.S.	Valid	Jump near, absolute indirect, address = zero-extended r/m16. Not supported in 64-bit mode.
FF /4	JMP r/m32	M	N.S.	Valid	Jump near, absolute indirect, address given in r/m32. Not supported in 64-bit mode.
FF /4	JMP r/m64	M	Valid	N.E.	Jump near, absolute indirect, RIP = 64-Bit offset from register or memory
EA cd	JMP ptr16:16	D	Inv.	Valid	Jump far, absolute, address given in operand
EA cp	JMP ptr16:32	D	Inv.	Valid	Jump far, absolute, address given in operand
FF /5	JMP m16:16	D	Valid	Valid	Jump far, absolute indirect, address given in m16:16
FF /5	JMP m16:32	D	Valid	Valid	Jump far, absolute indirect, address given in m16:32.
REX.W FF /5	JMP m16:64	D	Valid	N.E.	Jump far, absolute indirect, address given in m16:64.



- cb, cw, cd, cp, co, ct** — A 1-byte (cb), 2-byte (cw), 4-byte (cd), 6-byte (cp), 8-byte (co) or 10-byte (ct) value following the opcode. This value is used to specify a code offset and possibly a new value for the code segment register.

protected_mode_jump – ljmp instruction: instruction format (5/6)

```
protected_mode_jump [arch/x86/boot/pmjump.S] ** real mode **
Save cs to ebx in linear address (Ex: cs << 4)
Calculate relocation address for '.Lin_pm32' based on ebx
movw $__BOOT_DS, %cx
movw $__BOOT_TSS, %di
Enable protected mode by set bit 0 of cr0
ljmpl $__BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```

```
SYM_FUNC_START_NOALIGN(protected_mode_jump)
    movl    %edx, %esi           # Pointer to boot_params table

    xorl    %ebx, %ebx
    movw    %cs, %bx
    shll    $4, %ebx
    addl    %ebx, 2f
    jmp     1f                  # Short jump to serialize on 386/486

1:
    movw    $__BOOT_DS, %cx
    movw    $__BOOT_TSS, %di

    movl    %cr0, %edx
    orl     $X86_CR0_PE, %dl     # Protected mode
    movl    %edx, %cr0

    # Transition to 32-bit mode
    .byte   0x66, 0xea           # ljmp opcode
    .long   $.Lin_pm32          # offset
    .word   BOOT_CS              # segment

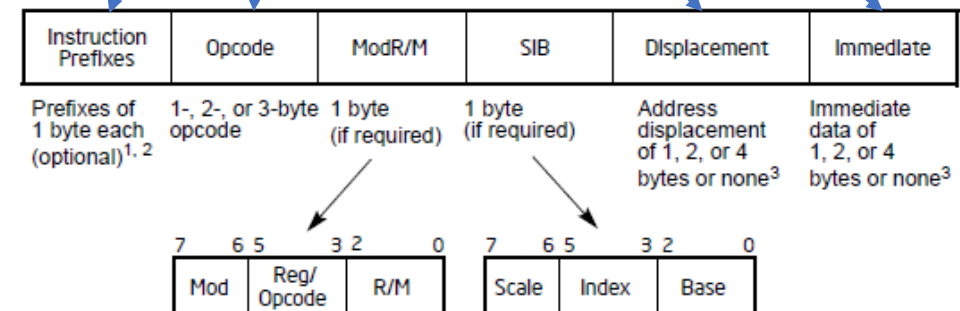
2:
SYM_FUNC_END(protected_mode_jump)

    .code32
    .section ".text32", "ax"
SYM_FUNC_START_LOCAL_NOALIGN(.Lin_pm32)
    # Set up data segments for flat 32-bit mode
    movl    %ecx, %ds
```

```
$ objdump -M i8086 -D arch/x86/boot/setup.elf
...
000013aa <protected_mode_jump>:
13aa: 66 89 d6                mov     %edx, %esi
13ad: 66 31 db                xor     %ebx, %ebx
13b0: 8c cb                  mov     %cs, %bx
13b2: 66 c1 e3 04            shl     $0x4, %ebx
13b6: 66 01 1e ce 13         add     %ebx, 0x13ce
13bb: eb 00                  jmp     13bd <protected_mode_jump+0x13>
13bd: b9 18 00              mov     $0x18, %cx
13c0: bf 20 00              mov     $0x20, %di
13c3: 0f 20 c2              mov     %cr0, %edx
13c6: 80 ca 01              or      $0x1, %dl
13c9: 0f 22 c2              mov     %edx, %cr0
13cc: 66 ea cc 30 00 00 10   ljmpl   $0x10, $0x30cc
13d3: 90                      ...

Disassembly of section .text32:

000030cc <.text32>:
30cc: 8e d9                mov     %cx, %ds
30ce: 8e c1                mov     %cx, %es
...
```

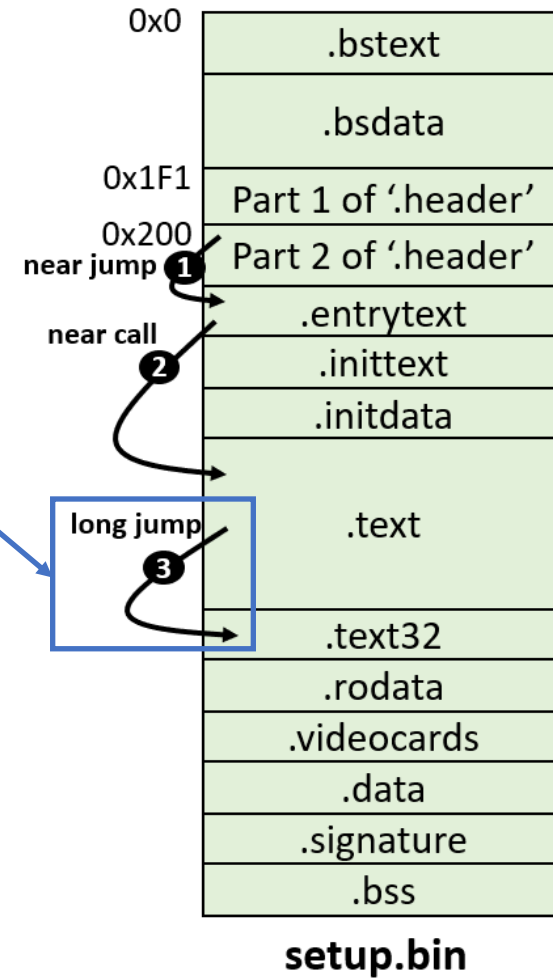


1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
3. Some rare instructions can take an 8B immediate or 8B displacement.

Figure 2-1. Intel 64 and IA-32 Architectures Instruction Format

protected_mode_jump – ljmp instruction: instruction format (6/6)

```
protected_mode_jump [arch/x86/boot/pmjump.S] ** real mode **  
Save cs to ebx in linear address (Ex: cs << 4)  
Calculate relocation address for '.Lin_pm32' based on ebx  
movw $__BOOT_DS, %cx  
movw $__BOOT_TSS, %di  
Enable protected mode by set bit 0 of cr0  
ljmpl $__BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```

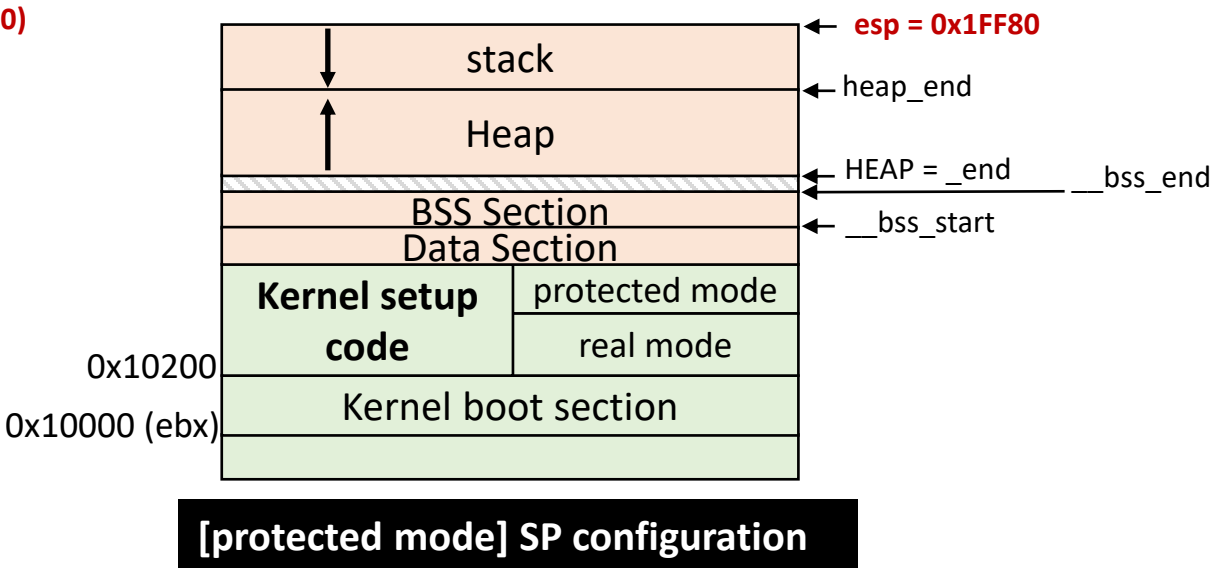
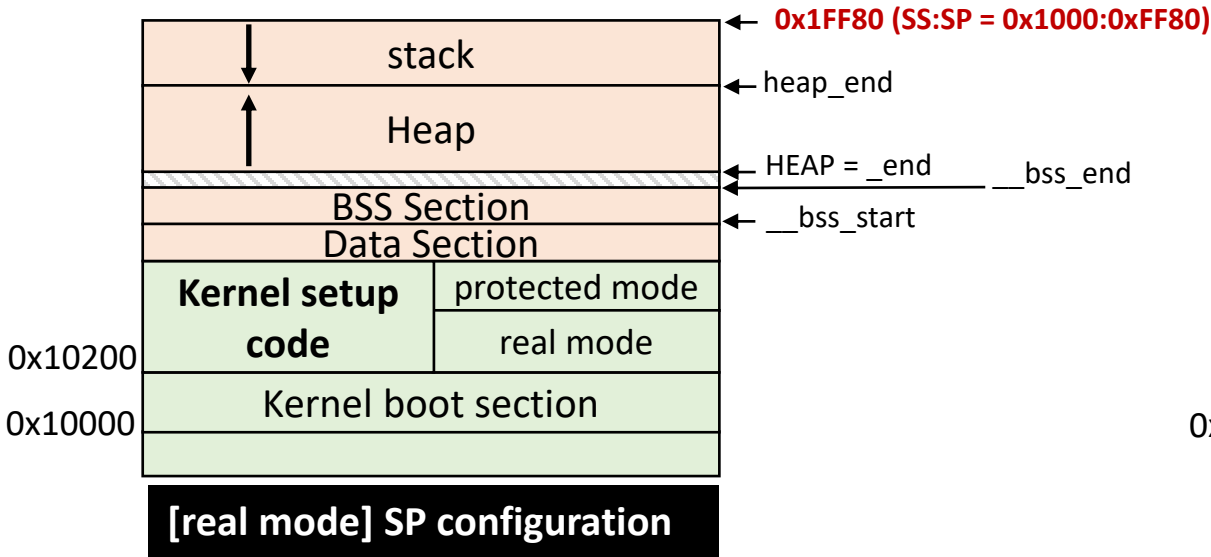
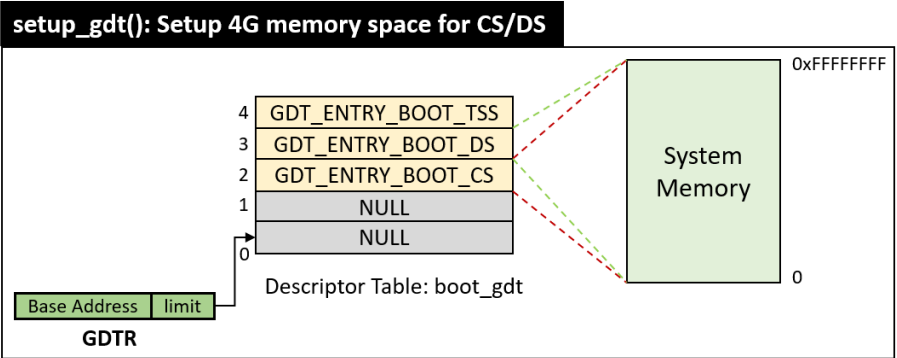


Protected mode: '.Lin_pm32' (1/2)

```
.Lin_pm32 [arch/x86/boot/pmjump.S] ** protected mode **
setup data segments (ds = es = fs = gs = ss = __BOOT_DS)
addl %ebx, %esp (setup esp: Use ebx to calculate correct
                esp in protected mode)
clear registers (ecx, edx, ebx, ebp and edi)
jmp1 *%eax (eax = code32_start = 0x100000)
```

'addl %ebx, %esp' in label ".Lin_pm32"

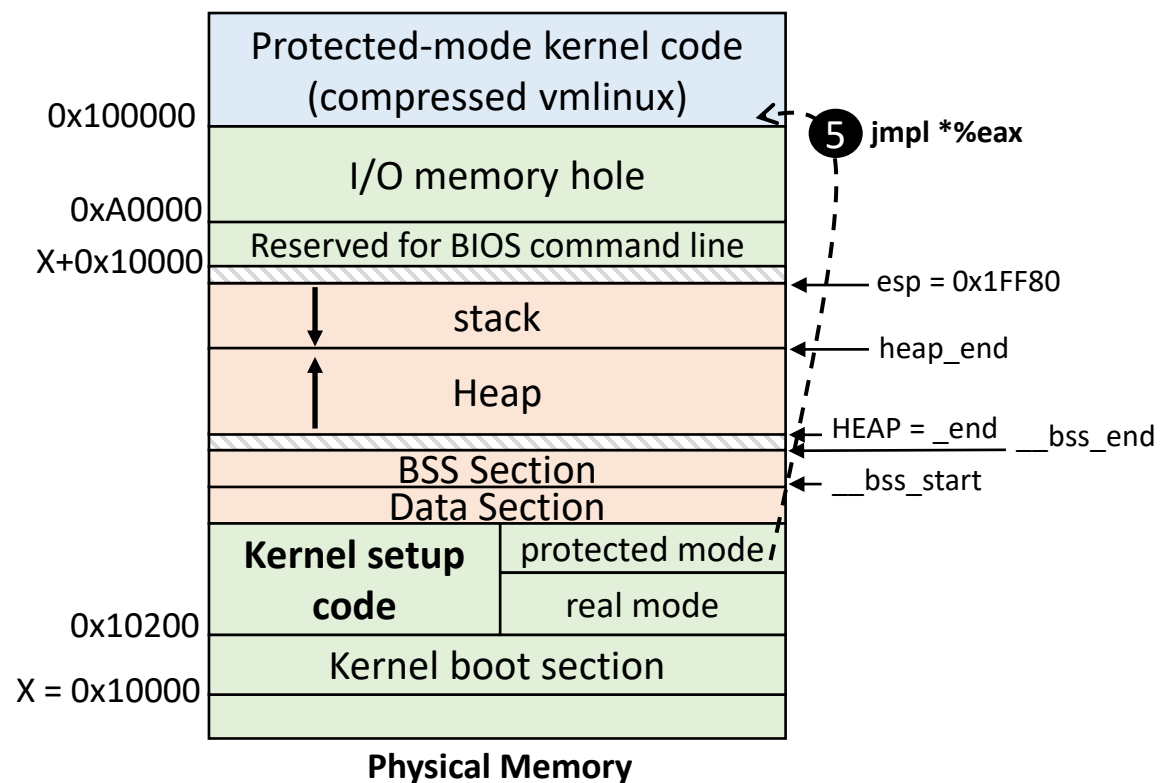
```
(gdb) si
58          addl    %ebx, %esp
(gdb) p /x $esp
$1 = 0xff80
(gdb) p /x %ebx
$2 = 0x10000
(gdb) si
?? ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/boot/
pmjump.S:61
61          ltr     %di
(gdb) p /x $esp
$3 = 0x1ff80
```



Protected mode: '.Lin_pm32' (2/2)

Call Path

```
go_to_protected_mode [arch/x86/boot/pm.c] -- ** real mode **
...
|- protected_mode_jump(boot_params.hdr.code32_start,
    (u32)&boot_params + (ds() << 4))
    ljmp $ __BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```



```
.Lin_pm32 [arch/x86/boot/pmjump.S] ** protected mode **
setup data segments (ds = es = fs = gs = ss = __BOOT_DS)
addl %ebx, %esp (setup esp: Use ebx to calculate correct
    esp in protected mode)
clear registers (ecx, edx, ebx, ebp and edi)
1 jmp $ __BOOT_CS, $.Lin_pm32 (Transition to 32-bit mode)
```

(gdb) si

[STACK]---			
13AA	0000	0000	0000
11D8	0000	0000	0000
0000	0000	0000	0000

[CPU]----			
rax	0x100000	1048576	
rbx	0x0	0	
rcx	0x0	0	
rdx	0x0	0	
rsi	0x139f0	80368	
rdi	0x0	0	
rbp	0x0	0x0	
rsp	0x1ff80	0x1ff80	
rip	0x130e8	0x130e8	
eflags	0x46	[IOPL=0 ZF PF]	
cs	0x10	16	
ss	0x18	24	
ds	0x18	24	
es	0x18	24	
fs	0x18	24	
gs	0x18	24	

[CPU Control Register]----			
cr0	0x11	[ET PE]	

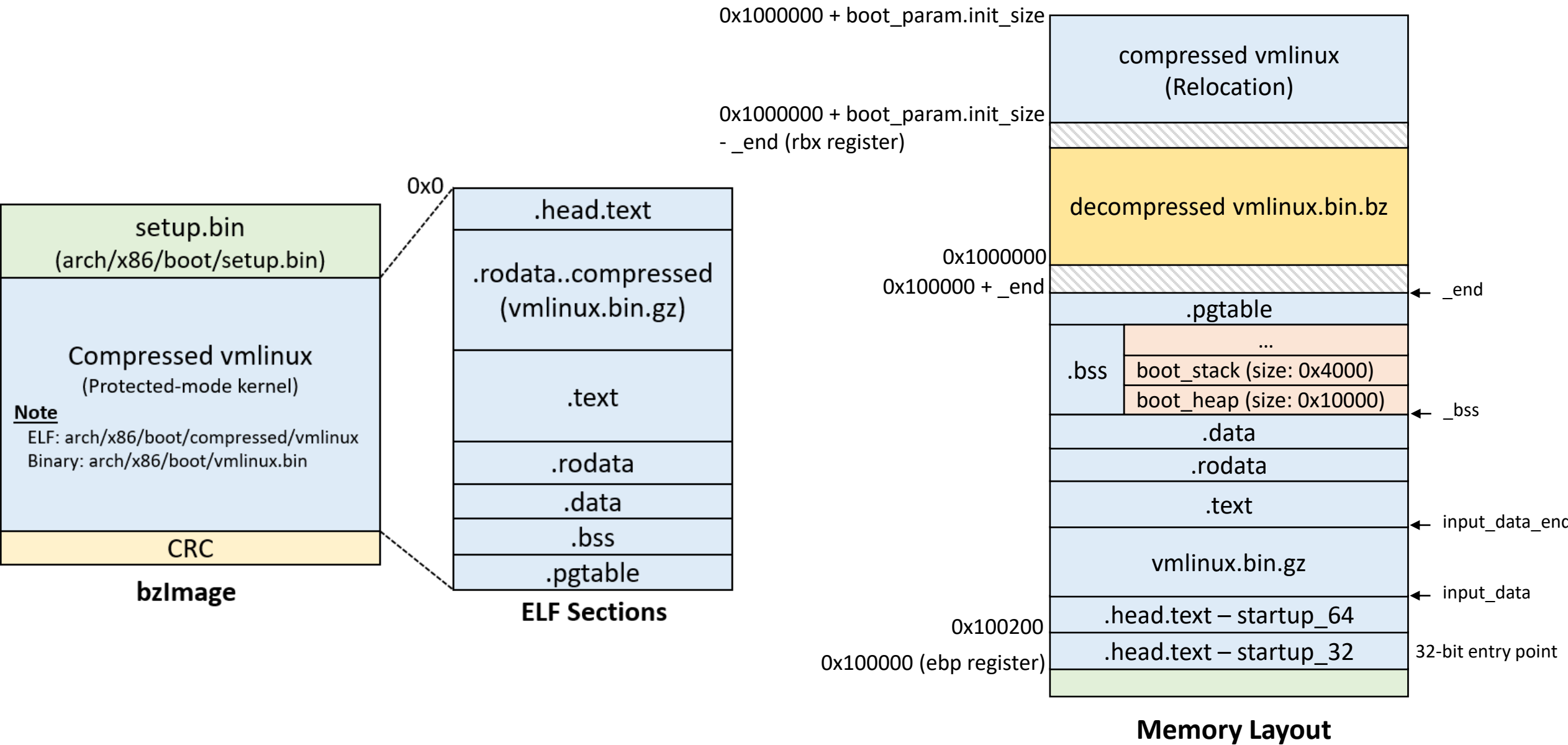
[eflags]----									
OF	<0>	DF	<0>	IF	<0>	TF	<0>	SF	<0>
ZF	<1>	AF	<0>	PF	<1>	CF	<0>		
ID	<0>	VIP	<0>	VIF	<0>	AC	<0>	VM	<0>
RF	<0>	NT	<0>	IOPL	<0>				

[CODE]----		
=> 0x130e8:	jmpq	*%rax
0x130ea:	add	%al, (%rax)
0x130ec:	add	%al, (%rax)
0x130ee:	add	%al, (%rax)
0x130f0:	add	%al, (%rax)
0x130f2:	add	%al, (%rax)
0x130f4:	add	%al, (%rax)
0x130f6:	add	%al, (%rax)
0x130f8:	add	%al, (%rax)
0x130fa:	add	%al, (%rax)
74	jmp	*%eax

Jump to the 32-bit entrypoint

(gdb)

Compressed vmlinux: memory layout (1/10)



Compressed vmlinux: boot_stack & boot_heap in .bss (2/10)

```
/*
 * Stack and heap for uncompression
 */
        .bss
        .balign 4
SYM_DATA_LOCAL(boot_heap,          .fill BOOT_HEAP_SIZE, 1, 0)

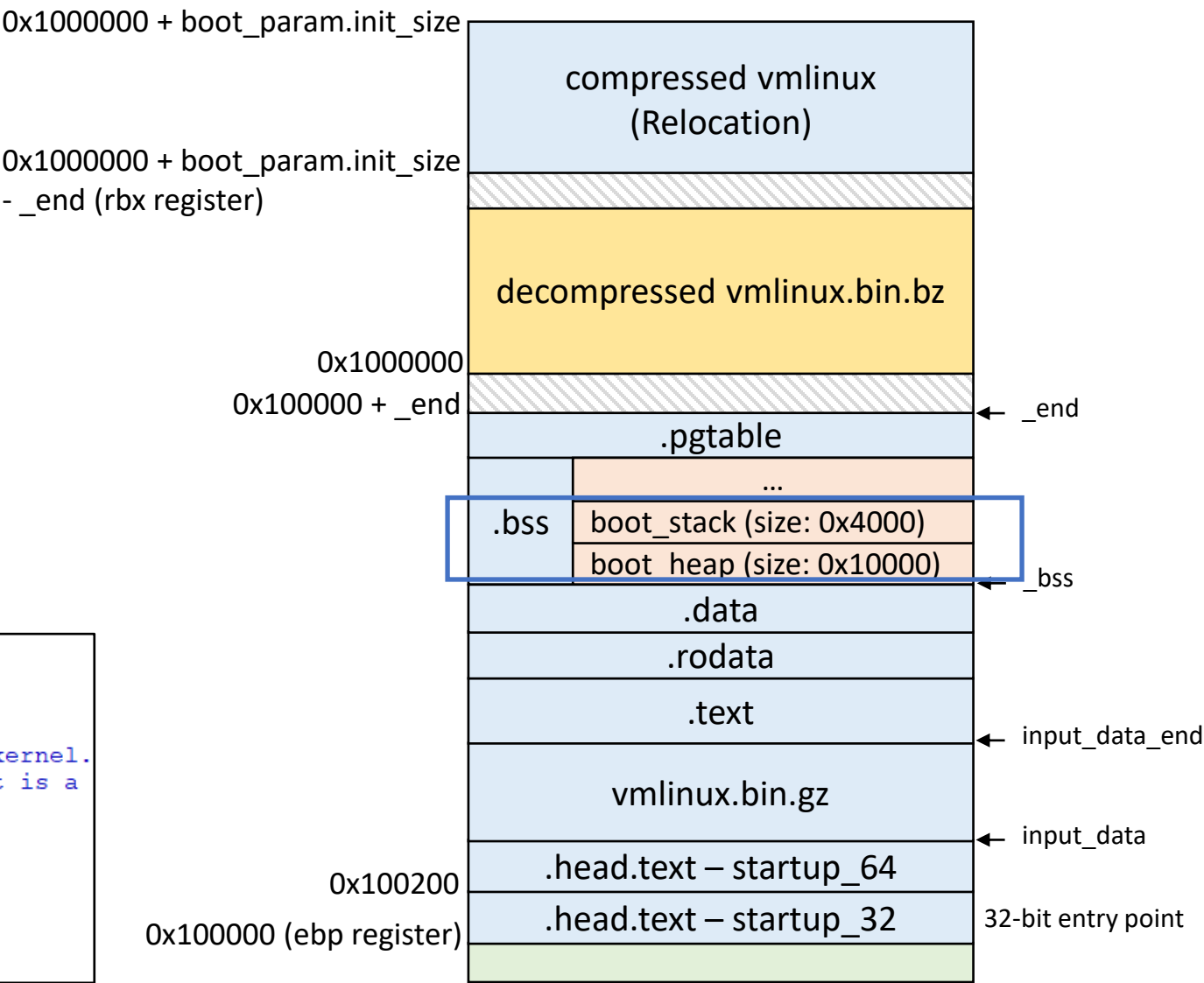
SYM_DATA_START_LOCAL(boot_stack)
        .fill BOOT_STACK_SIZE, 1, 0
        .balign 16
SYM_DATA_END_LABEL(boot_stack, SYM_L_LOCAL, boot_stack_end)

/*
 * Space for page tables (not in .bss so not zeroed)
 */
        .section ".pgtable", "aw", @nobits
        .balign 4096
SYM_DATA_LOCAL(pgtable,          .fill BOOT_PGT_SIZE, 1, 0)

arch/x86/boot/compressed/head 64.S
```

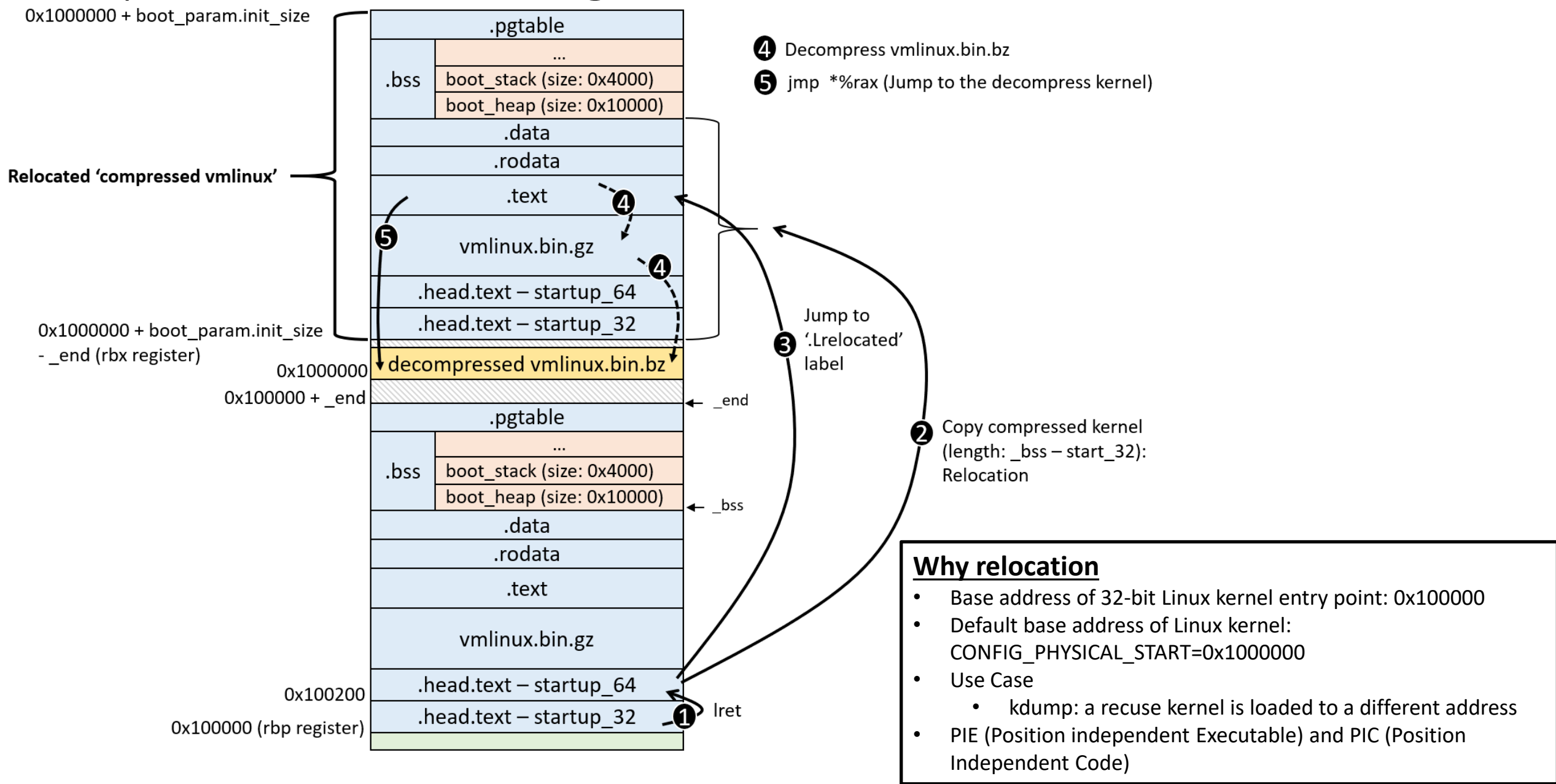
```
#if defined(CONFIG_KERNEL_BZIP2)
# define BOOT_HEAP_SIZE      0x400000
#elif defined(CONFIG_KERNEL_ZSTD)
/*
 * Zstd needs to allocate the ZSTD_DCtx in order to decompress the kernel.
 * The ZSTD_DCtx is ~160KB, so set the heap size to 192KB because it is a
 * round number and to allow some slack.
 */
# define BOOT_HEAP_SIZE      0x30000
#else
# define BOOT_HEAP_SIZE      0x10000
#endif

arch/x86/include/asm/boot.h      15,1
```



Memory Layout

Compressed vmlinux: High-level Overview (3/10)

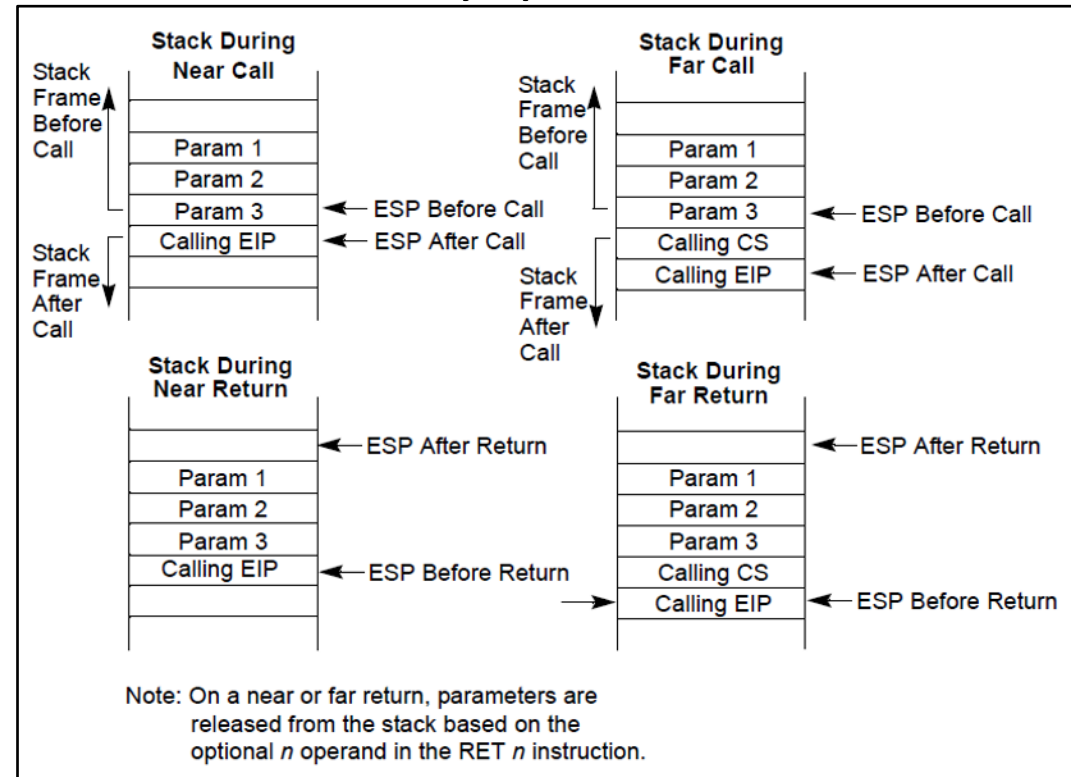


Why relocation

- Base address of 32-bit Linux kernel entry point: 0x100000
- Default base address of Linux kernel: CONFIG_PHYSICAL_START=0x1000000
- Use Case
 - kdump: a recuse kernel is loaded to a different address
- PIE (Position independent Executable) and PIC (Position Independent Code)

Compressed vmlinux: startup_32: 32-bit entry point (4/10)

```
startup_32 [arch/x86/boot/compressed/head_64.S] -- ** protected mode **
1 Invoke 'call' & 'popl' instructions to get current PC
  * Get the loading address of the compressed vmlinux.bin (ebp)
  * 32-bit EIP cannot be accessed directly by SW. It can be
    controlled implicitly control-transfer instruction (JMP, Jcc, CALL
    and RET), interrupts and exceptions.
  * 64-bit RIP: Support RIP-relative addressing
Load new gdt with the 64-bit segment
Load segment registers (ds, es, fs, gs and ss) with __BOOT_DS
Setup stack (16KB stack for x86_64, 4KB stack for x86)
  * This stack space is between 0x100000 and LOAD_PHYSICAL_ADDR (0x1000000)
Verify CPU if it supports long mode (64-bit mode)
Calculate the address for relocated 'compressed vmlinux' (stored in ebx)
Enable PAE mode
|- get_sev_encryption_bit
Init 4-level page table: identity mapping for 0-4GB (relocated space)
Load boot page table to CR3
Enable long mode (64-bit mode) - EFER (Extended Feature Enable Register)
Jump to 64-bit mode
    leal    rva(startup_64)(%ebp), %eax
    pushl   $__KERNEL_CS # __KERNEL_CS: 64-bit code segment
    pushl   %eax
    movl    $(X86_CR0_PG | X86_CR0_PE), %eax
    movl    %eax, %cr0
    lret
```



```
1
#define rva(X) ((X) - startup_32)

.code32
SYM_FUNC_START(startup_32)
    cld
    cli

    leal    (BP_scratch+4)(%esi), %esp
    call    1f
1:    popl   %ebp
    subl    $ rva(1b), %ebp
```

```
$ objdump -D arch/x86/boot/compressed/vmlinux

arch/x86/boot/compressed/vmlinux:      file format elf64-x86-64

Disassembly of section .head.text:

0000000000000000 <startup_32>:
0:    fc                                cld
1:    fa                                cli
2:    8d a6 e8 01 00 00                lea    0x1e8(%rsi),%esp
8:    e8 00 00 00 00                    callq  d <startup_32+0xd>
d:    5d                                pop     %rbp
e:    83 ed 0d                          sub     $0xd,%ebp
...
```

Compressed vmlinux: startup_32 (5/10)

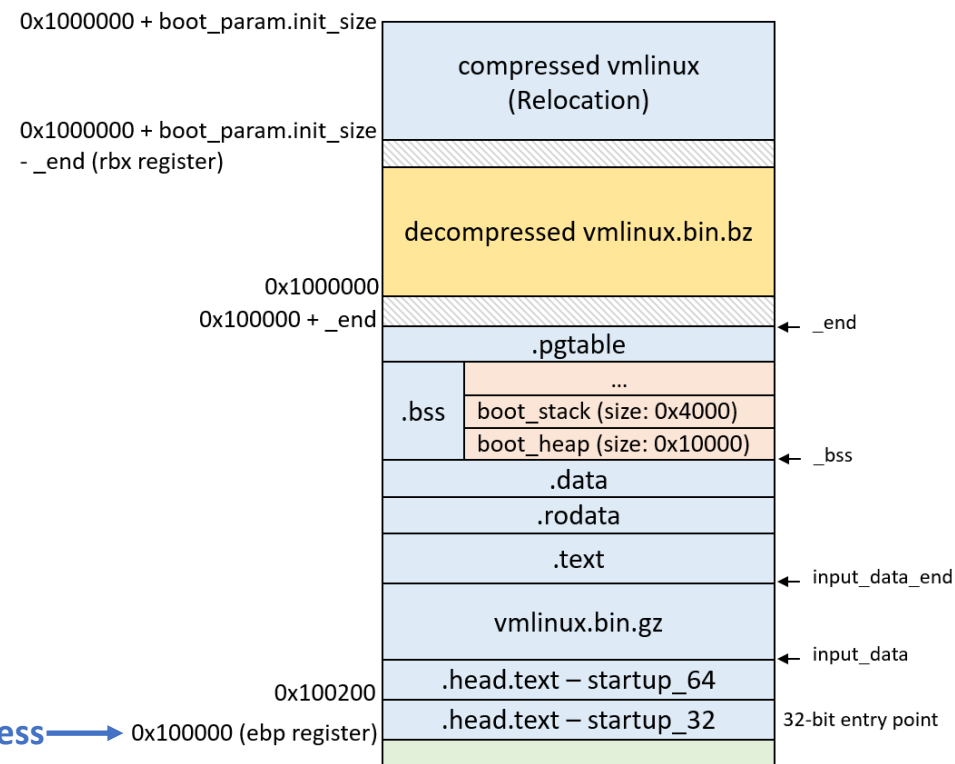
```
1 startup_32 [arch/x86/boot/compressed/head_64.S] -- ** protected mode **
* Invoke 'call' & 'popl' instructions to get current PC
* Get the loading address of the compressed vmlinux.bin (ebp)
* 32-bit EIP cannot be accessed directly by SW. It can be
  controlled implicitly control-transfer instruction (JMP, Jcc, CALL
  and RET), interrupts and exceptions.
* 64-bit RIP: Support RIP-relative addressing
Load new gdt with the 64-bit segment
Load segment registers (ds, es, fs, gs and ss) with __BOOT_DS
Setup stack (16KB stack for x86_64, 4KB stack for x86)
* This stack space is between 0x100000 and LOAD_PHYSICAL_ADDR (0x1000000)
Verify CPU if it supports long mode (64-bit mode)
Calculate the address for relocated 'compressed vmlinux' (stored in ebx)
Enable PAE mode
|- get_sev_encryption_bit
Init 4-level page table: identity mapping for 0-4GB (relocated space)
Load boot page table to CR3
Enable long mode (64-bit mode) - EFER (Extended Feature Enable Register)
Jump to 64-bit mode
    leal    rva(startup_64)(%ebp), %eax
    pushl   $__KERNEL_CS # __KERNEL_CS: 64-bit code segment
    pushl   %eax
    movl    $(X86_CR0_PG | X86_CR0_PE), %eax
    movl    %eax, %cr0
    lret
```

```
1 #define rva(X) ((X) - startup_32)

        .code32
SYM_FUNC_START(startup_32)
    cld
    cli

    leal    (BP_scratch+4)(%esi), %esp
    call    1f
1:    popl    %ebp
    subl    $ rva(1b), %ebp
```

Get the loading address → 0x100000 (ebp register)



```
adrian@adrian-ubuntu:linux$ readelf -l arch/x86/boot/compressed/vmlinux

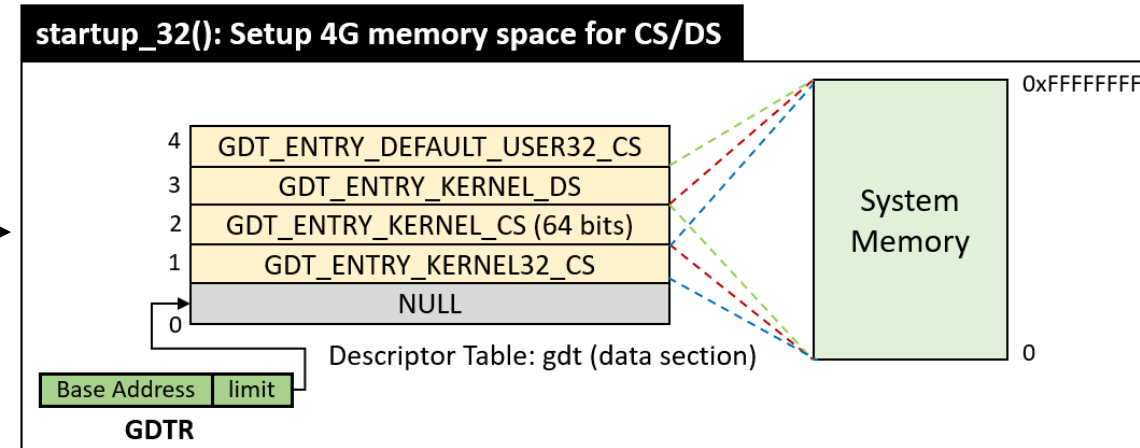
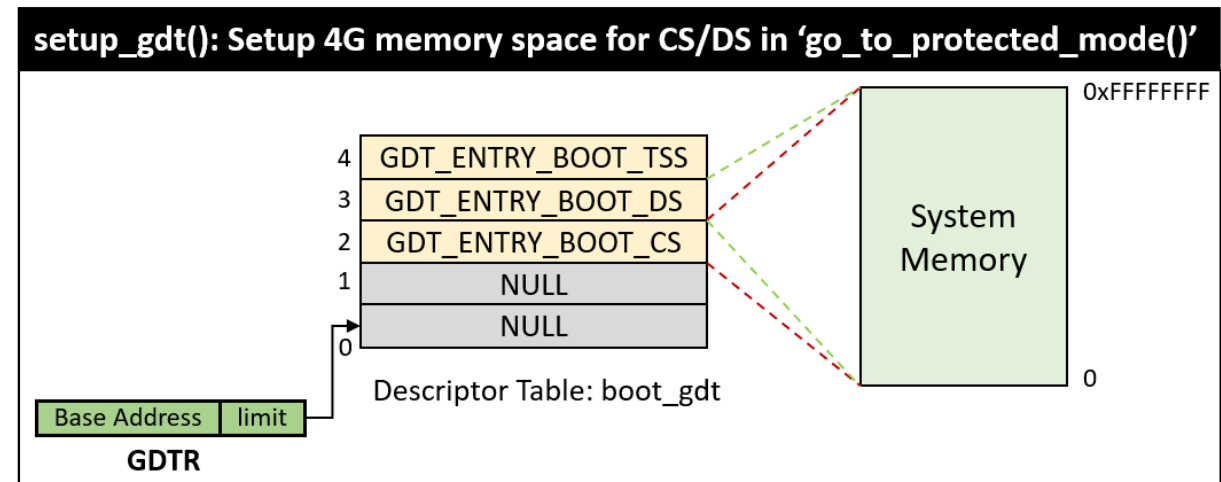
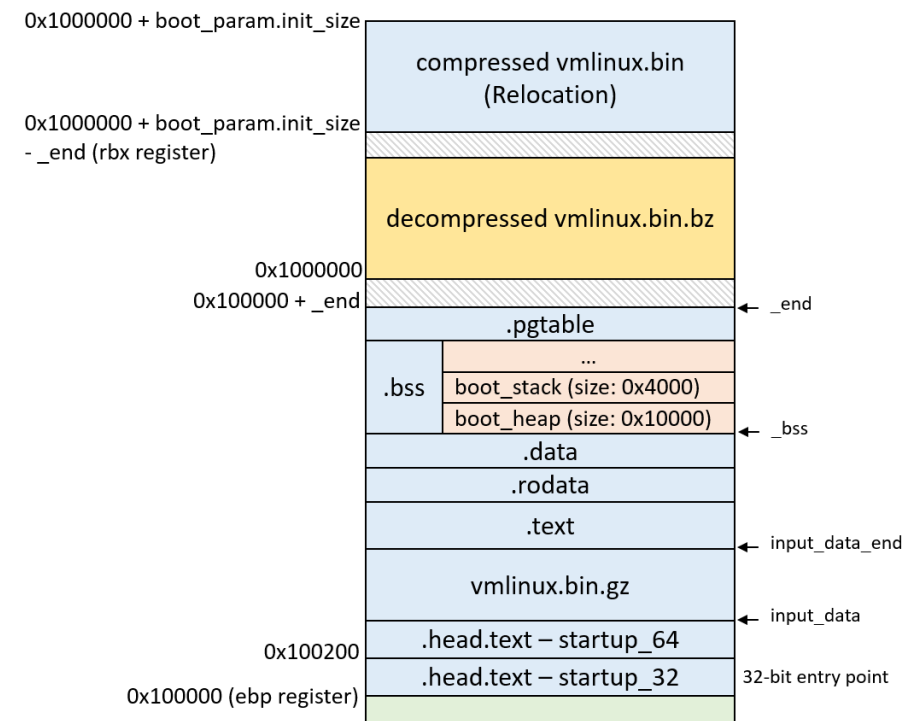
Elf file type is DYN (Shared object file)
Entry point 0x200
There are 3 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
     FileSiz      MemSiz          Flags             Align
LOAD            0x0000000000000100 0x0000000000000000 0x0000000000000000
LOAD            0x0000000000012188 0x00000000000138968 RWE               0x1000
LOAD            0x0000000000000000 0x00000000000139000 0x00000000000139000
                                0x0000000000007000  RW               0x1000
GNU_STACK       0x0000000000000000 0x0000000000000000 0x0000000000000000
                                0x0000000000000000  RWE               0x10

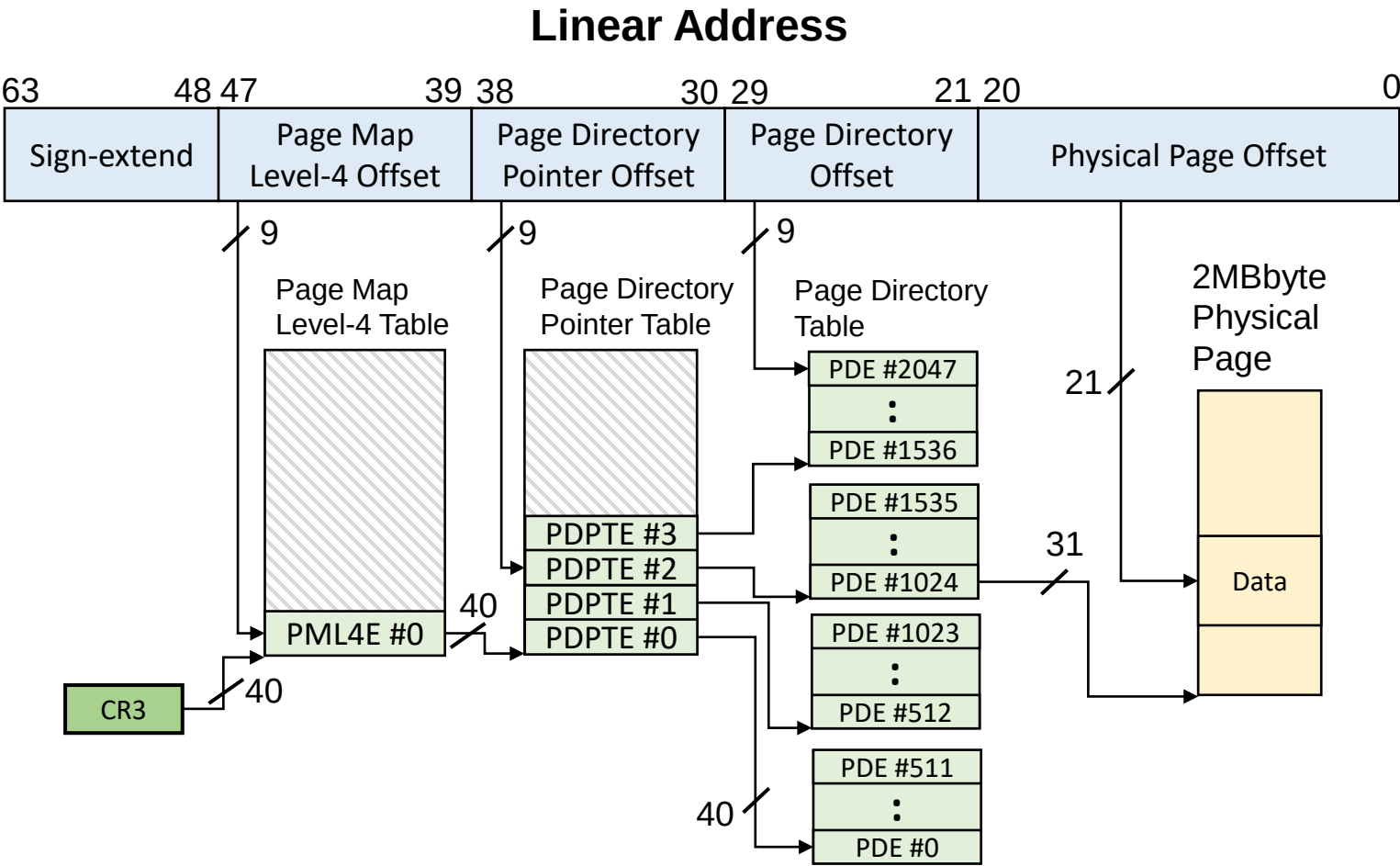
Section to Segment mapping:
Segment Sections...
 00  .head.text .rodata..compressed .text .rodata .data .bss
 01  .pgtable
 02
```

Compressed vmlinux: startup_32 (6/10)

```
startup_32 [arch/x86/boot/compressed/head_64.S] -- ** protected mode **
Invoke 'call' & 'popl' instructions to get current PC
* Get the loading address of the compressed vmlinux.bin (ebp)
* 32-bit EIP cannot be accessed directly by SW. It can be
  controlled implicitly control-transfer instruction (JMP, Jcc, CALL
  and RET), interrupts and exceptions.
* 64-bit RIP: Support RIP-relative addressing
2 Load new gdt with the 64-bit segment
Load segment registers (ds, es, fs, gs and ss) with __BOOT_DS
Setup stack (16KB stack for x86_64, 4KB stack for x86)
* This stack space is between 0x100000 and LOAD_PHYSICAL_ADDR (0x1000000)
Verify CPU if it supports long mode (64-bit mode)
Calculate the address for relocated 'compressed vmlinux' (stored in ebx)
Enable PAE mode
|- get_sev_encryption_bit
Init 4-level page table: identity mapping for 0-4GB (relocated space)
Load boot page table to CR3
Enable long mode (64-bit mode) - EFER (Extended Feature Enable Register)
Jump to 64-bit mode
    leal    rva(startup_64)(%ebp), %eax
    pushl   $__KERNEL_CS # __KERNEL_CS: 64-bit code segment
    pushl   %eax
    movl    $(X86_CR0_PG | X86_CR0_PE), %eax
    movl    %eax, %cr0
    lret
```



Compressed vmlinux: startup_32: Init 4-level page table (7/10)



[Paging] Identity mapping for 0-4GB memory space

Compressed vmlinux: startup_32: Init 4-level page table (8/10)

Table 4-1. Properties of Different Paging Modes

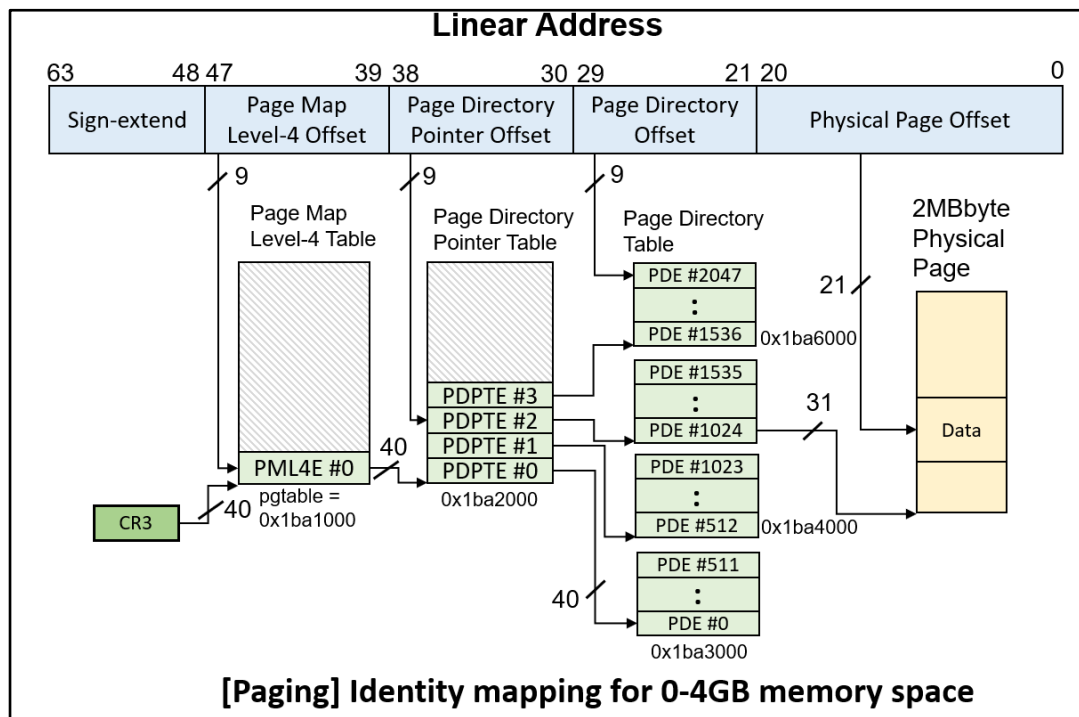
Paging Mode	PG in CR0	PAE in CR4	LME in IA32_EFER	Lin.-Addr. Width	Phys.-Addr. Width ¹	Page Sizes	Supports Execute-Disable?	Supports PCIDs and protection keys?
None	0	N/A	N/A	32	32	N/A	No	No
32-bit	1	0	0 ²	32	Up to 40 ³	4 KB 4 MB ⁴	No	No
PAE	1	1	0	32	Up to 52	4 KB 2 MB	Yes ⁵	No
4-level	1	1	1	48	Up to 52	4 KB 2 MB 1 GB ⁶	Yes ⁵	Yes ⁷

NOTES:

1. The physical-address width is always bounded by MAXPHYADDR; see Section 4.1.4.
2. The processor ensures that IA32_EFER.LME must be 0 if CR0.PG = 1 and CR4.PAE = 0.
3. 32-bit paging supports physical-address widths of more than 32 bits only for 4-MByte pages and only if the PSE-36 mechanism is supported; see Section 4.1.4 and Section 4.3.
4. 4-MByte pages are used with 32-bit paging only if CR4.PSE = 1; see Section 4.3.
5. Execute-disable access rights are applied only if IA32_EFER.NXE = 1; see Section 4.6.
6. Not all processors that support 4-level paging support 1-GByte pages; see Section 4.1.4.
7. PCIDs are used only if CR4.PCIDE = 1; see Section 4.10.1. Protection keys are used only if certain conditions hold; see Section 4.6.2.

Reference: Section 4.1 “PAGING MODES AND CONTROL BITS”, Intel® 64 and IA-32 Architectures Software Developer’s Manual Volume 3 (3A, 3B, 3C & 3D): System Programming Guide

Compressed vmlinux: startup_32: Init 4-level page table (9/10)



```
$ objdump -D arch/x86/boot/compressed/vmlinux | less
Disassembly of section .pgtable:

0000000000163000 <pgtable>:
...

0000000000176000 <top_pgtable>:
...
```

```
(gdb) info registers ebx
ebx          0x1a3e000          27516928
(gdb) x /4x 0x1ba1000
0x1ba1000:    0x01ba2027    0x00000000    0x00000000    0x00000000
(gdb) x /12x 0x1ba2000
0x1ba2000:    0x01ba3027    0x00000000    0x01ba4007    0x00000000
0x1ba2010:    0x01ba5007    0x00000000    0x01ba6007    0x00000000
0x1ba2020:    0x00000000    0x00000000    0x00000000    0x00000000
(gdb)
```

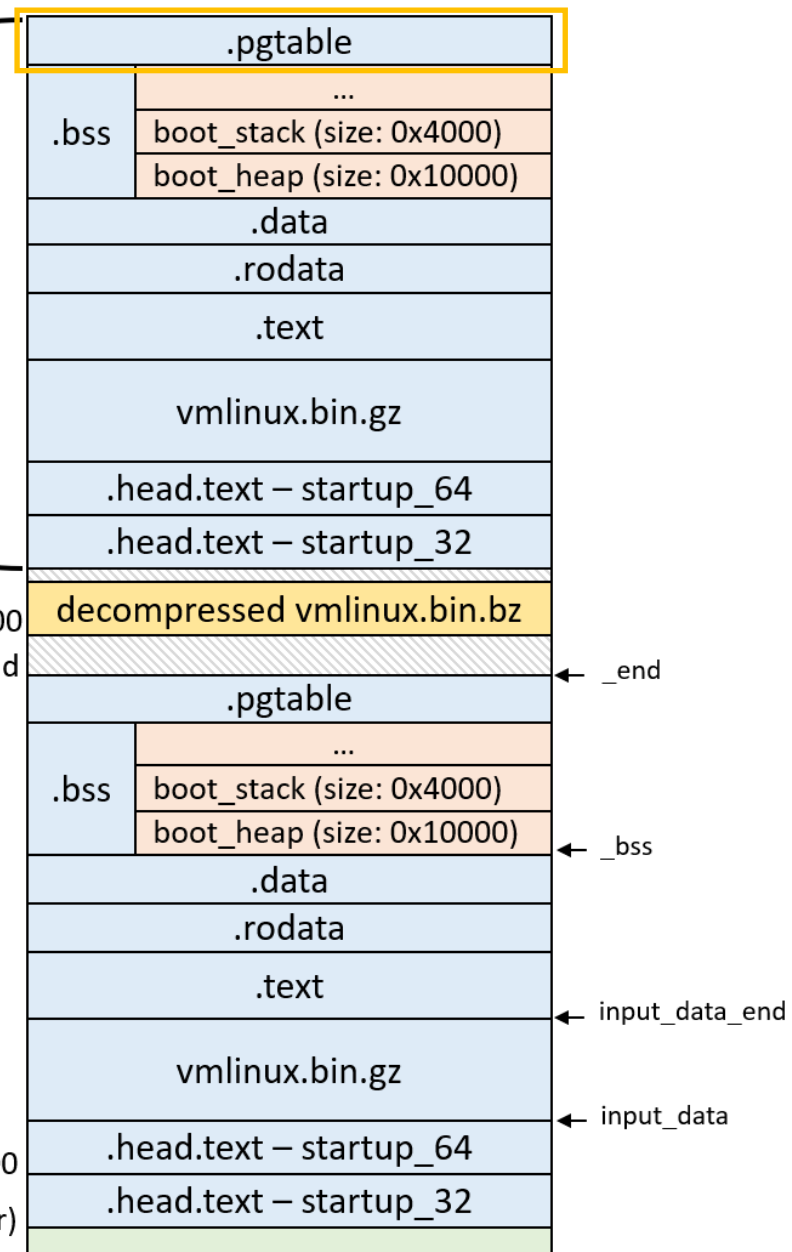
0x1000000 + boot_param.init_size

Relocated 'compressed vmlinux'

0x1000000 + boot_param.init_size
- _end (rbx register)

0x1000000
0x100000 + _end

0x100200
0x100000 (rbp register)



Compressed vmlinux: far return to startup_64 (10/10)

```
startup_32 [arch/x86/boot/compressed/head_64.S] -- ** protected mode **
Invoke 'call' & 'popl' instructions to get current PC
* Get the loading address of the compressed vmlinux.bin (ebp)
* 32-bit EIP cannot be accessed directly by SW. It can be
  controlled implicitly control-transfer instruction (JMP, Jcc, CALL
  and RET), interrupts and exceptions.
* 64-bit RIP: Support RIP-relative addressing
Load new gdt with the 64-bit segment
Load segment registers (ds, es, fs, gs and ss) with __BOOT_DS
Setup stack (16KB stack for x86_64, 4KB stack for x86)
* This stack space is between 0x100000 and LOAD_PHYSICAL_ADDR (0x1000000)
Verify CPU if it supports long mode (64-bit mode)
Calculate the address for relocated 'compressed vmlinux' (stored in ebx)
Enable PAE mode
|- get_sev_encryption_bit
Init 4-level page table: identity mapping for 0-4GB (relocated space)
Load boot page table to CR3
Enable long mode (64-bit mode) - EFER (Extended Feature Enable Register)
Jump to 64-bit mode
    leal    rva(startup_64)(%ebp), %eax
    pushl   $__KERNEL_CS # __KERNEL_CS: 64-bit code segment
    pushl   %eax
    movl    $(X86_CR0_PG | X86_CR0_PE), %eax
    movl    %eax, %cr0
    lret
```

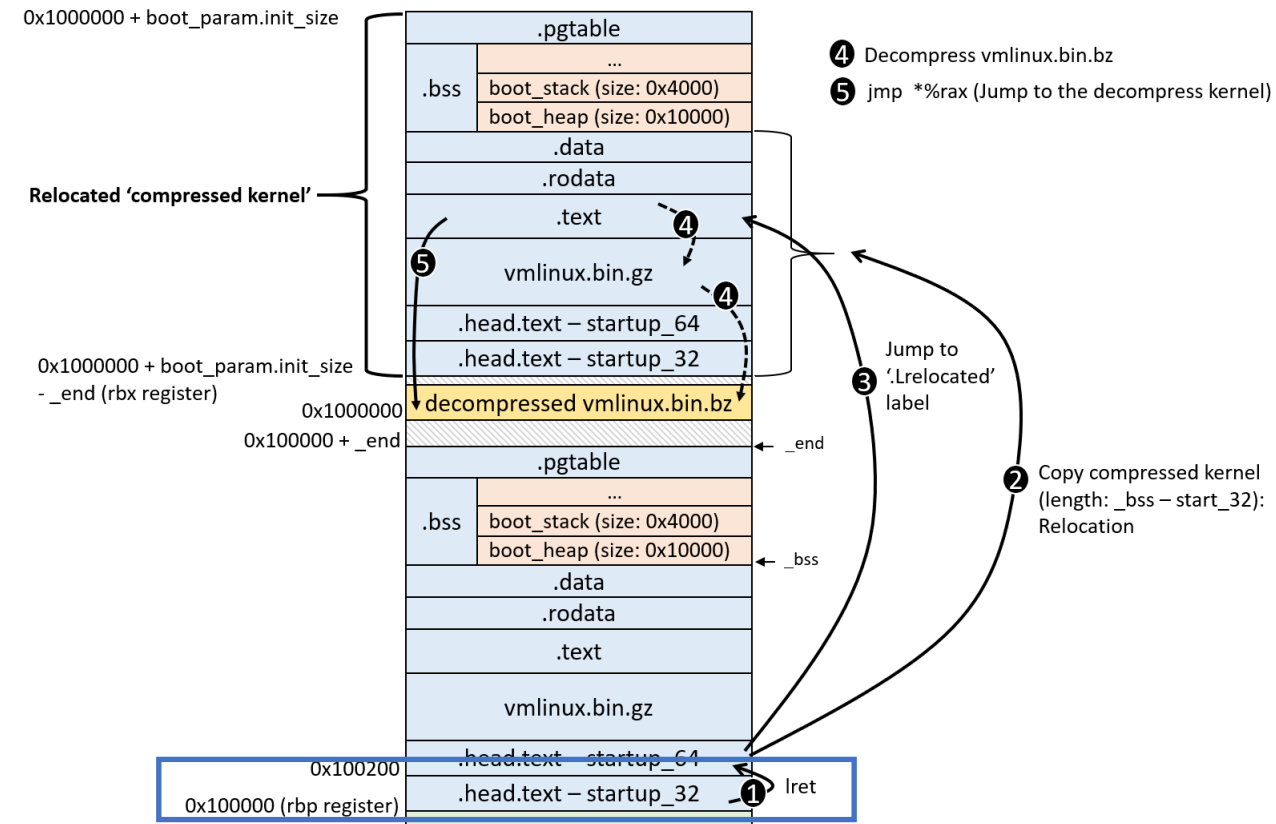
```
$ objdump -D arch/x86/boot/compressed/vmlinux | less

arch/x86/boot/compressed/vmlinux:      file format elf64-x86-64

Disassembly of section .head.text:

0000000000000000 <startup_32>:
 0:  fc          cld
 1:  fa          cli
 2:  8d a6 e8 01 00 00    lea    0x1e8(%rsi),%esp
 8:  e8 00 00 00 00    callq d <startup_32+0xd>
 d:  5d          pop    %rbp
 ...

0000000000000200 <startup_64>:
200:  fc          cld
201:  fa          cli
 ...
```



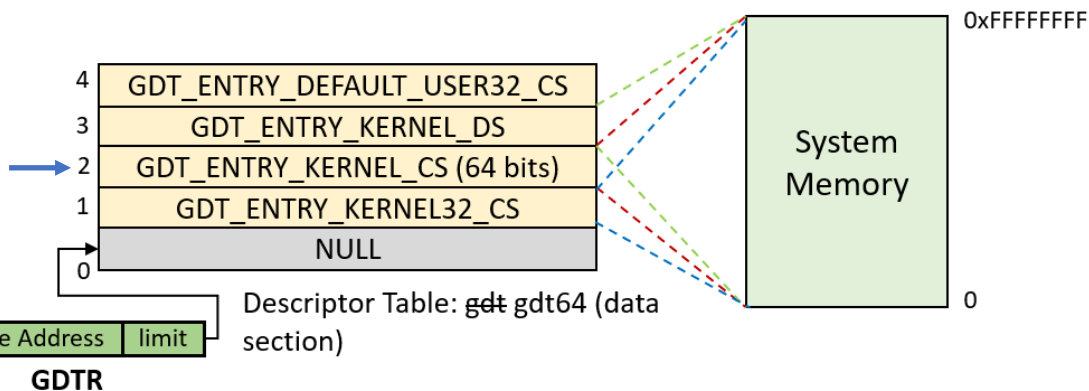
```
#define rva(X) ((X) - startup_32)
arch/x86/boot/compressed/head_64.S
```

rva(startup_64) = 0x200
ebp = 0x100000
eax = 0x100000 + 0x200 = 0x100200

Compressed vmlinux: startup_64

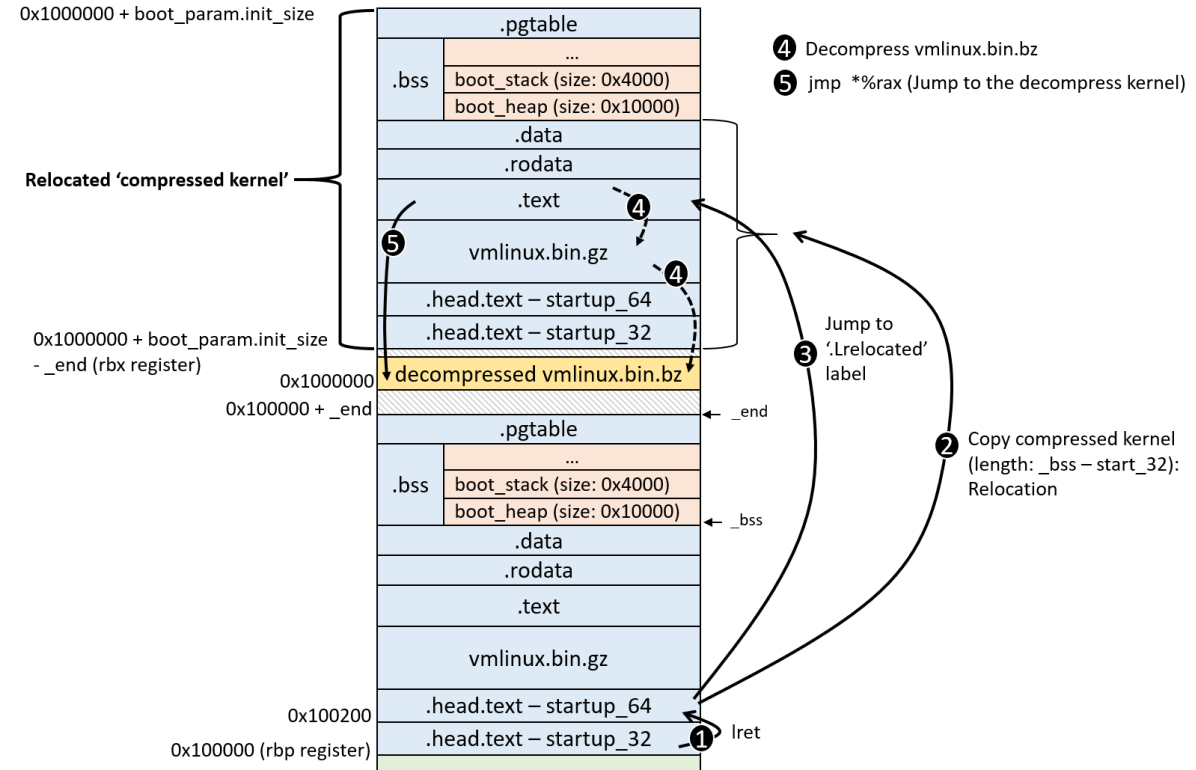
```
startup_64 [arch/x86/boot/compressed/head_64.S] -- * long mode + paging *
Clear all segment registers except for cs
Calculate the address for relocated 'compressed vmlinux' (stored in rbx)
Setup stack (16KB stack for x86_64, 4KB stack for x86)
* This stack space is above LOAD_PHYSICAL_ADDR (0x1000000)
Load 64-bit GDT to GDTR (GDT Register) - with 32-bit code segment
Reload CS
    pushq    $_KERNEL_CS
    leaq     .Lon_kernel_cs(%rip), %rax
    pushq    %rax
    lretq
.Lon_kernel_cs
|- load_stagel_idt
|- paging_prepare (check if 5-level paging needs to be enabled)
2 Copy vmlinux.bin to the address above LOAD_PHYSICAL_ADDR
3 Jump to .Lrelocated
    leaq     rva(.Lrelocated)(%rbx), %rax
    jmp     *%rax
```

startup_32(): Setup 4G memory space for CS/DS



Why to reload CS? (Commit "34bb49229f19")

When the pre-decompression code loads its first GDT in startup_64, it is still running on the CS value of the previous GDT. In the case of SEV-ES this is the EFI GDT. It can be anything depending on what has loaded the kernel (EFI, legacy boot code, container runtime, etc.)



System Table Registers				
	47(79)	16	15	0
GDTR	32(64)-bit Linear Base Address		16-Bit Table Limit	
IDTR	32(64)-bit Linear Base Address		16-Bit Table Limit	

System Segment Registers		Segment Descriptor Registers (Automatically Loaded)			
Task Register	15	0		Attributes	
LDTR	Seg. Sel.	32(64)-bit Linear Base Address	Segment Limit		
	Seg. Sel.	32(64)-bit Linear Base Address	Segment Limit		

Compressed vmlinux: [.text] .Lrelocated (1/5)

```
.Lrelocated [arch/x86/boot/compressed/head_64.S] -- * protected mode + paging *
Clear bss in relocated space
|- set_sev_encryption_mask
|- load_stage2_idt
|- initialize_identity_maps
|- extract_kernel
|- Init heap 'start' and 'end' pointer
|- choose_random_location (KASLR feature)
|- __decompress
|- parse_elf
|- handle_relocations
5 Jump to the decompressed kernel
  jmp    *%rax
```

Why to call initialize_identity_maps()?

commit ca0e22d4f011a56e974fa3a712d76e86a791559d

Author: Joerg Roedel <jroedel@suse.de>

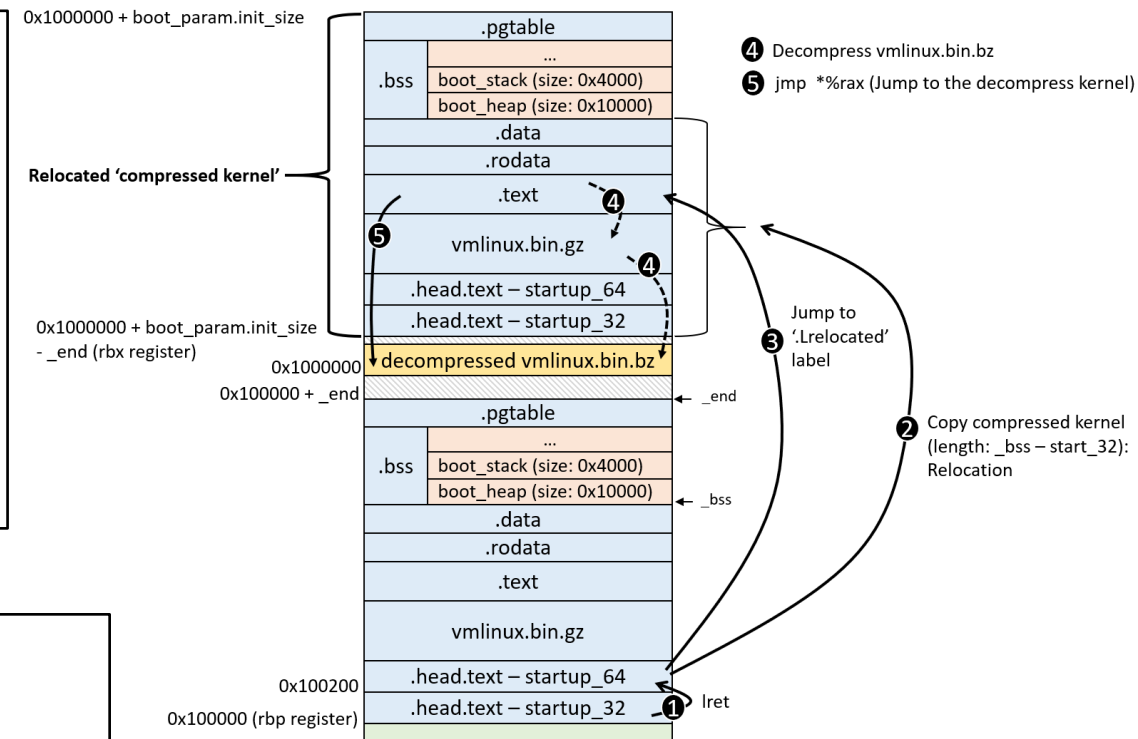
Date: Mon Sep 7 15:15:17 2020 +0200

x86/boot/compressed/64: Always switch to own page table

When booted through startup_64(), the kernel keeps running on the EFI page table until the KASLR code sets up its own page table. Without KASLR, the pre-decompression boot code never switches off the EFI page table. Change that by unconditionally switching to a kernel-controlled page table after relocation.

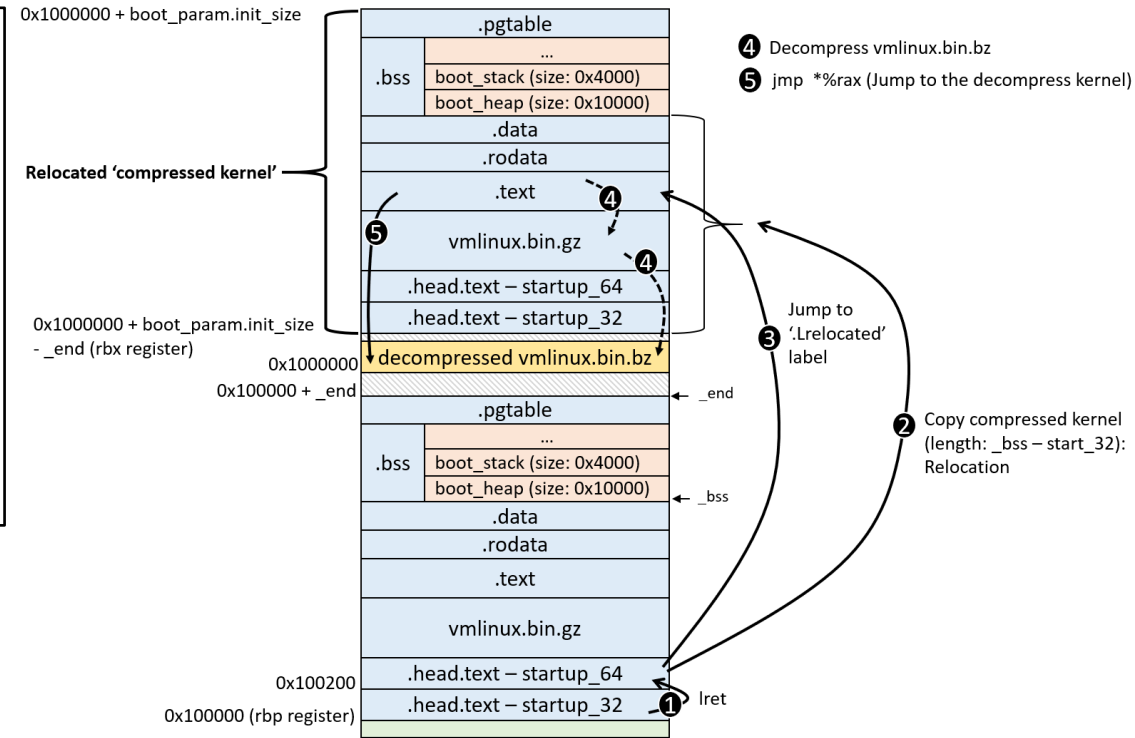
This makes sure the kernel can make changes to the mapping when necessary, for example map pages unencrypted in SEV and SEV-ES guests.

Also, remove the debug_putstr() calls in initialize_identity_maps() because the function now runs before console_init() is called.



Compressed vmlinux: [.text] .Lrelocated (2/5)

```
.Lrelocated [arch/x86/boot/compressed/head_64.S] -- * protected mode + paging *
Clear bss in relocated space
|- set_sev_encryption_mask
|- load_stage2_idt
|- initialize_identity_maps
|- add_identity_map(_head, _end)
|- add_identity_map(boot_params, boot_params + 1)
|- add_identity_map(cmdline, cmdline + COMMAND_LINE_SIZE)
4|- extract_kernel
   |- Init heap 'start' and 'end' pointer
   |- choose_random_location (KASLR feature)
   |- __decompress
   |- parse_elf
   |- handle_relocations
5 Jump to the decompressed kernel
   jmp    *%rax
```



Why to map boot_params and command line?

commit b17a45b6e53f6613118b2e5cfc4a992cc50deb2c
Author: Arvind Sankar <nivedita@alum.mit.edu>
Date: Fri Oct 16 16:04:01 2020 -0400

x86/boot/64: Explicitly map boot_params and command line

Commits

```
ca0e22d4f011 ("x86/boot/compressed/64: Always switch to own page table")
8570978ea030 ("x86/boot/compressed/64: Don't pre-map memory in KASLR code")
```

set up a new page table in the decompressor stub, but without explicit mappings for boot_params and the kernel command line, relying on the #PF handler instead.

This is fragile, as boot_params and the command line mappings are required for the main kernel. If EARLY_PRINTK and RANDOMIZE_BASE are disabled, a QEMU/OVMF boot never accesses the command line in the decompressor stub, and so it never gets mapped. The main kernel accesses it from the identity mapping if AMD_MEM_ENCRYPT is enabled, and will crash.

Fix this by adding back the explicit mapping of boot_params and the command line.

Compressed vmlinux: parse_elf (3/5)

```
.Lrelocated [arch/x86/boot/compressed/head_64.S] -- * protected mode + paging *
Clear bss in relocated space
|- set_sev_encryption_mask
|- load_stage2_idt
|- initialize_identity_maps
4|- extract_kernel
   |- Init heap 'start' and 'end' pointer
   |- choose_random_location (KASLR feature)
   |- __decompress
   |- parse_elf ←
   |- handle_relocations
Jump to the decompressed kernel
    jmp    *%rax
```

```
adrian@adrian-ubuntu:linux$ readelf -l arch/x86/boot/compressed/vmlinux.bin
```

```
Elf file type is EXEC (Executable file)
Entry point 0x1000000
There are 4 program headers, starting at offset 64
```

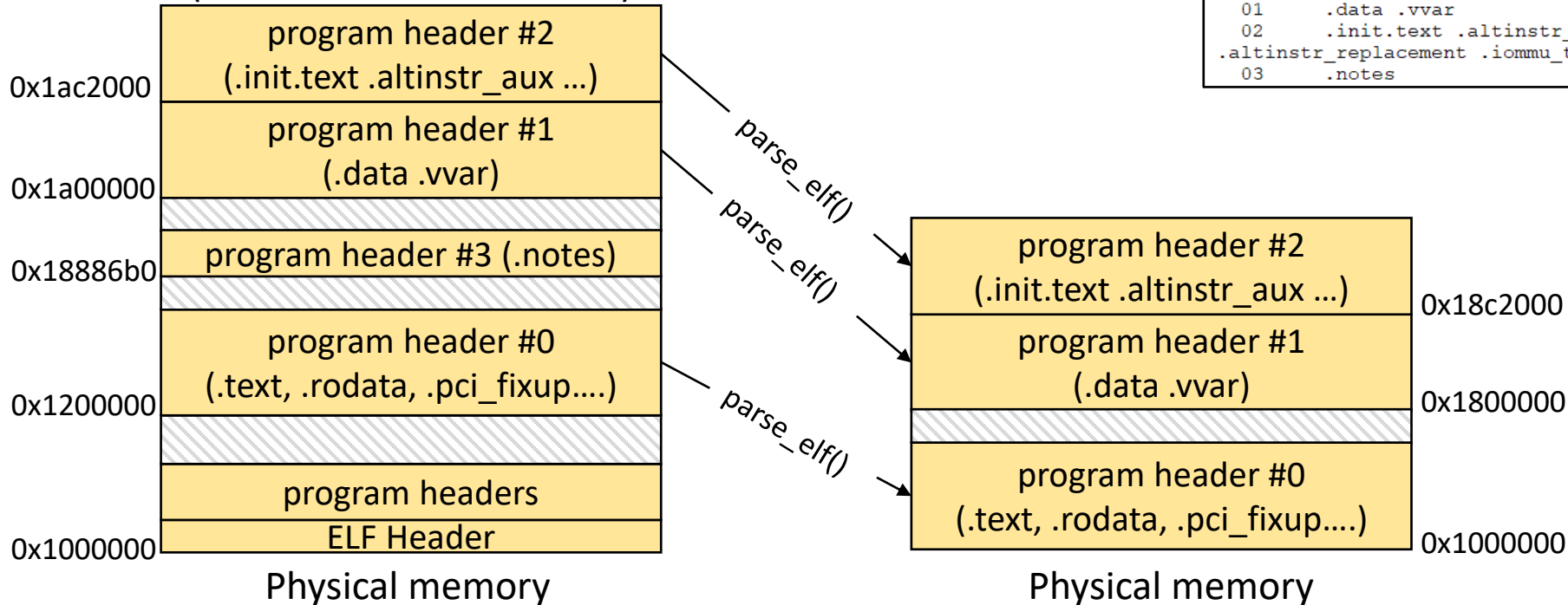
Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
LOAD	0x0000000000200000	0xfffffffff81000000	0x0000000001000000
	0x00000000006886ec	0x00000000006886ec	R E 0x200000
LOAD	0x0000000000a00000	0xfffffffff81800000	0x0000000001800000
	0x0000000000c2000	0x0000000000c2000	RW 0x200000
LOAD	0x0000000000ac2000	0xfffffffff818c2000	0x00000000018c2000
	0x00000000007b657	0x000000000015a000	RWE 0x200000
NOTE	0x00000000008886b0	0xfffffffff816886b0	0x00000000016886b0
	0x000000000000003c	0x000000000000003c	0x4

Section to Segment mapping:

```
Segment Sections...
00 .text .rodata .pci_fixup __param __modver __ex_table .notes
01 .data .vvar
02 .init.text .altinstr_aux .init.data .x86_cpu_dev.init .altinstructions
.altinstr_replacement .iommu_table .apicdrivers .exit.text .bss .brk
03 .notes
```

decompressed vmlinux.bin.bz
(vmlinux.bin – ELF format)



Compressed vmlinux: handle_relocations (4/5)

```
.Lrelocated [arch/x86/boot/compressed/head_64.S] -- * protected mode + paging *
Clear bss in relocated space
|- set_sev_encryption_mask
|- load_stage2_idt
|- initialize_identity_maps
4|- extract_kernel
   |- Init heap 'start' and 'end' pointer
   |- choose_random_location (KASLR feature)
   |- __decompress
   |- parse_elf
   |- handle_relocations ←
Jump to the decompressed kernel
    jmp    *%rax
```

CONFIG_RELOCATABLE

- Retain relocation information (generate .rel.* or rela.* sections) when building a kernel image, so it can be loaded someplace besides the default address (CONFIG_PHYSICAL_START = 16MB).
- Use case: kdump kernel (recovery kernel)

handle_relocations() - Relocation if CONFIG_X86_NEED_RELOCS is set

- Depend on RANDOMIZE_BASE || (X86_32 && RELOCATABLE)
- Scan relocation tables (.rel.* or .rela.* sections) for symbol relocation

```
adrian@adrian-ubuntu:linux$ readelf -S vmlinux
There are 58 section headers, starting at offset 0x83c0ce0:

Section Headers:
 [Nr] Name              Type              Address            Offset
      Size              EntSize          Flags Link Info  Align
-----
[ 0]                      NULL              0000000000000000   00000000
      0000000000000000   0000000000000000           0  0    0
[ 1] .text              PROGBITS          ffffffff81000000   00200000
      000000000004007c0   0000000000000000    AX    0  0    64
→[ 2] .rela.text         RELA              0000000000000000   03ef03b0
      000000000000e9328   0000000000000018    I     55  1    8
[ 3] .rodata            PROGBITS          ffffffff81600000   00800000
      00000000000085c90   0000000000000000    WA    0  0   4096
→[ 4] .rela.rodata       RELA              0000000000000000   03fd96d8
      0000000000002d990   0000000000000018    I     55  3    8
[ 5] .pci_fixup         PROGBITS          ffffffff81685c90   00885c90
      0000000000000220   0000000000000000    A     0  0    16
→[ 6] .rela.pci_fixup    RELA              0000000000000000   04007068
      0000000000000330   0000000000000018    I     55  5    8
[ 7] __param            PROGBITS          ffffffff81685eb0   00885eb0
      00000000000000a78   0000000000000000    A     0  0    8
→[ 8] .rela__param       RELA              0000000000000000   04007398
      00000000000012d8   0000000000000018    I     55  7    8
...
[14] .data              PROGBITS          ffffffff81800000   00a00000
      000000000000c0c00   0000000000000000    WA    0  0   16384
→[15] .rela.data         RELA              0000000000000000   04013650
      00000000000013500   0000000000000018    I     55  14   8
[16] .vvar             PROGBITS          ffffffff818c1000   00ac1000
      00000000000001000   0000000000000000    WA    0  0    16
[17] .init.text         PROGBITS          ffffffff818c2000   00ac2000
      00000000000020059   0000000000000000    AX    0  0    16
→[18] .rela.init.text    RELA              0000000000000000   04026b50
      00000000000031c20   0000000000000018    I     55  17    8
...
```

Compressed vmlinux: handle_relocations (5/5)

```
.Lrelocated [arch/x86/boot/compressed/head_64.S] -- * protected mode + paging *
```

```
Clear bss in relocated space
|- set_sev_encryption_mask
|- load_stage2_idt
|- initialize_identity_maps
4|- extract_kernel
   |- Init heap 'start' and 'end' pointer
   |- choose_random_location (KASLR feature)
   |- __decompress
   |- parse_elf
   |- handle_relocations ←
Jump to the decompressed kernel
    jmp    *%rax
```

```
adrian@adrian-ubuntu:linux$ readelf -r vmlinux
```

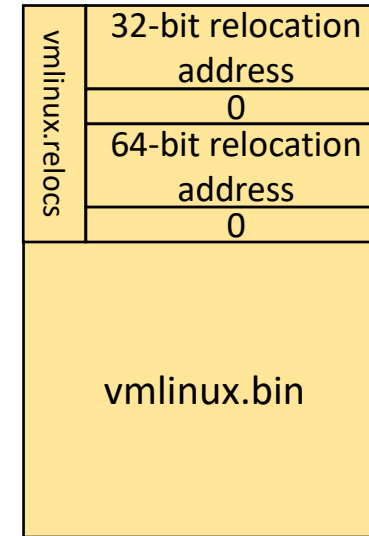
```
Relocation section '.rela.text' at offset 0x3ef03a0 contains 39799 entries:
```

Offset	Info	Type	Sym. Value	Sym. Name + Addend
ffffffff81000003	357a00000002	R_X86_64_PC32	ffffffff81804000	__end_init_task - ac
ffffffff8100000a	3a2400000002	R_X86_64_PC32	ffffffff81000000	_text - 4
ffffffff81000010	2ccf00000004	R_X86_64_PLT32	ffffffff810005d0	startup_64_setup_env - 4
ffffffff81000029	3a2400000002	R_X86_64_PC32	ffffffff81000000	_text - 4
ffffffff8100002f	324a00000004	R_X86_64_PLT32	ffffffff81000220	__startup_64 - 4
ffffffff81000036	23020000000b	R_X86_64_32S	ffffffff818e4000	early_top_pgt + 80000000
ffffffff81000047	32f200000004	R_X86_64_PLT32	ffffffff81000620	__startup_secondary_64 - 4
ffffffff8100004e	2cce0000000b	R_X86_64_32S	ffffffff8181a000	init_top_pgt + 80000000
....				

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.bin.gz.cmd
cmd_arch/x86/boot/compressed/vmlinux.bin.gz := cat arch/x86/boot/compressed/vmlinux.bin arch/x86/boot/compressed/vmlinux.relocs | gzip -n -f -9 > arch/x86/boot/compressed/vmlinux.bin.gz
```

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.bin.cmd
cmd_arch/x86/boot/compressed/vmlinux.bin := objcopy -R .comment -S vmlinux arch/x86/boot/compressed/vmlinux.bin
```

```
adrian@adrian-ubuntu:linux$ cat arch/x86/boot/compressed/.vmlinux.relocs.cmd
cmd_arch/x86/boot/compressed/vmlinux.relocs := arch/x86/tools/relocs vmlinux > arch/x86/boot/compressed/vmlinux.relocs; arch/x86/tools/relocs --abs-relocs vmlinux
```



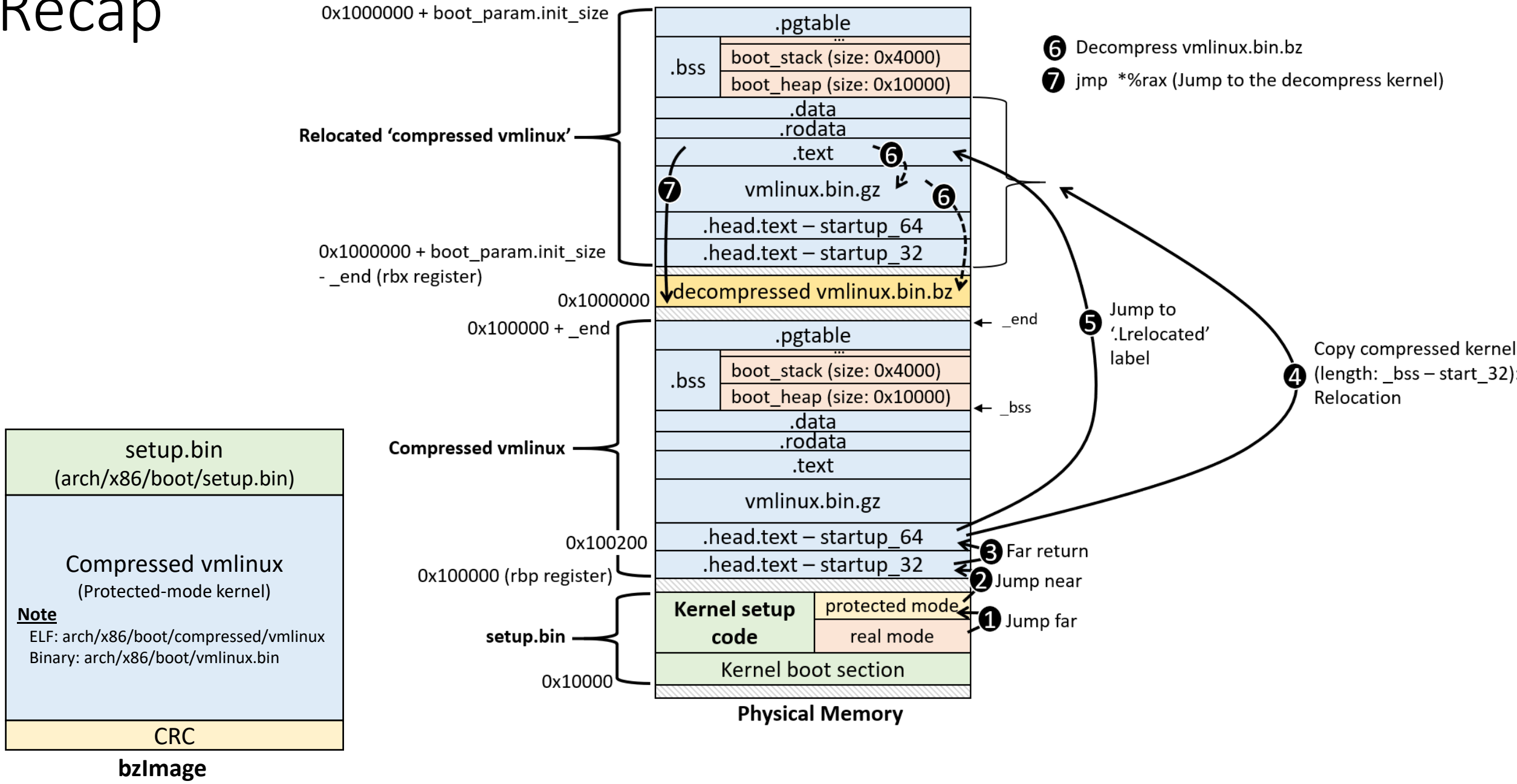
handle_relocations():
Perform relocation
backwards from the end
of the decompressed
vmlinux

vmlinux.bin.bz

objdump options

- R section_name: Remove any section matching section_name
- S or strip-all: Do not copy relocation and symbol information from the source file

Recap



[More info] bzImage = vmlinuz

```
[root@rh82 linux]# md5sum arch/x86/boot/bzImage
8f6edd8760ab29fad6754f5ffdb1b947  arch/x86/boot/bzImage
[root@rh82 linux]# md5sum /boot/vmlinuz-5.11.0-rc5+
8f6edd8760ab29fad6754f5ffdb1b947  /boot/vmlinuz-5.11.0-rc5+
```

On a physical machine

Source code: arch/x86/boot/Makefile, arch/x86/boot/install.sh

Reference

- The Linux/x86 Boot Protocol, Documentation/x86/boot.rst
- Intel® 64 and IA-32 Architectures Software Developer's Manual
- <https://wdv4758h.github.io/notes/blog/linux-kernel-boot.html>
- Linux insides, <https://0xax.gitbooks.io/linux-insides/content/>

Appendix

gdb: Preparation for debugging real-mode of Linux kernel (1/2)

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ vimcat scripts/launch-vm.sh
#!/bin/bash

source `find ./ -name config.sh | head -n1`

# Since we target at debugging real-mode (32 bit), we need to use
# the tool 'qemu-system-i386' instead of qemu-system-x86_64
# If you want to run x86_64 kernel image to boot into shell prompt,
# please use the tool 'qemu-system-x86_64'
qemu_bin=qemu-system-i386

${qemu_bin} -nographic -smp 4 -m 2047M \
    -kernel $KERNEL_OBJ/arch/x86/boot/bzImage \
    -initrd $INITRAMFS_IGZ \
    -append "earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr" \
    -s -S
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-vm.sh
```

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ vimcat scripts/launch-gdb.sh
#!/bin/bash

source `find ./ -name config.sh | head -n1`

GDB_FILES_FOLDER=$ROOT/gdb-files
GDB_LINUX_CFG=$GDB_FILES_FOLDER/gdb-linux-kernel-real-mode.txt
SETUP_ELF=$OUT/obj/linux/arch/x86/boot/setup.elf
SETUP_ELF_BASE=0x10000

generate_gdb_cfg() {
    elf_sections=( ".btext" ".bssdata" ".header" ".entrytext" )
    elf_sections_addr=( 0, 0, 0, 0 )

    for ((i=0;i<${#elf_sections[@]};i++)); do
        elf_sections_addr[$i]=`readelf -S $SETUP_ELF | grep -w ${elf_sections[$i]} | awk '{print $5}'`
        elf_sections_addr[$i]=`printf "0x%x\n" $((16#${elf_sections_addr[$i]} + $SETUP_ELF_BASE))`
    done

    text_section_addr=`readelf -S $SETUP_ELF | grep -w .text | awk '{print $5}'`
    text_section_addr=`printf "0x%x" $((16#${text_section_addr} + $SETUP_ELF_BASE))`

    # This is a quite tricky way to run 'tee' with EOF in a bash function.
    # The file content 'GDB_LINUX_CFG' cannot have the indentation for
    # the utility command 'tee file << EOF'
    tee $GDB_LINUX_CFG << EOF
# debug real-mode code of Linux kernel
add-symbol-file $SETUP_ELF 0x103f7 \\\
    -s .btext ${elf_sections_addr[0]} \\\
    -s .bssdata ${elf_sections_addr[1]} \\\
    -s .header ${elf_sections_addr[2]} \\\
    -s .entrytext ${elf_sections_addr[3]}
target remote :1234
b start_of_setup
c
EOF
}

if [ ! -f $GDB_LINUX_CFG ]; then
    generate_gdb_cfg
fi

cd $GDB_FILES_FOLDER
gdb -ix $GDB_FILES_FOLDER/gdb-init-real-mode.txt -ix $GDB_LINUX_CFG
```

Github: <https://github.com/AdrianHuang/gdb-linux-real-mode>

gdb: Preparation for debugging real-mode of Linux kernel (2/2)

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ ./scripts/launch-gdb.sh
...
Breakpoint 1 at 0x10200

Thread 1 received signal SIGTRAP, Trace/breakpoint trap.
-----[ STACK ]-----
0000 0000 0000 0000 0000 0000 0000 0000
6165 6C72 7079 6972 746E 3D6B 6573 6972
-----[ DS:SI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D8 8E C0 8E D0 31 E4 FB .....1..
00010010: FC BE 2D 00 AC 20 C0 74 09 B4 0E BB 07 00 CD 10 ..-....t.....
00010020: EB F2 31 C0 CD 16 CD 19 EA F0 FF 00 F0 55 73 65 ..1.....Use
00010030: 20 61 20 62 6F 6F 74 20 6C 6F 61 64 65 72 2E 0D .a.boot.loader..
-----[ ES:DI ]-----
00010000: EA 05 00 C0 07 8C C8 8E D8 8E C0 8E D0 31 E4 FB .....1..
00010010: FC BE 2D 00 AC 20 C0 74 09 B4 0E BB 07 00 CD 10 ..-....t.....
00010020: EB F2 31 C0 CD 16 CD 19 EA F0 FF 00 F0 55 73 65 ..1.....Use
00010030: 20 61 20 62 6F 6F 74 20 6C 6F 61 64 65 72 2E 0D .a.boot.loader..
-----[ CPU ]-----
AX: 1020 BX: 0000 CX: 0000 DX: 0000
SI: 0000 DI: 0000 SP: FFF0 BP: 0000
CS: 1020 DS: 1000 ES: 1000 SS: 1000

IP: 0000 RIP:00000000
CS:IP: 1020:0000 (0x10200)
SS:SP: 1000:FFF0 (0x1FFF0)
SS:BP: 1000:0000 (0x10000)
-----[ CPU Control Register]-----
cr0          0x10          [ ET ]
cr3          0x0          [ PDBR=0 PCID=0 ]
efer         0x0          [ ]
-----[ eflags]-----
OF <0>  DF <0>  IF <0>  TF <0>  SF <0>  ZF <1>  AF <0>  PF <1>  CF <0>
ID <0>  VIP <0>  VIF <0>  AC <0>  VM <0>  RF <0>  NT <0>  IOPL <0>
-----[ CODE ]-----
0x10200:  jmp     0x1026c
0x10202:  rex.W
0x10203:  fs jnb  0x10259
0x10206:  lar     (%rax),%eax
0x10209:  add     %al, (%rax)
0x1020b:  add     %al, (%rax)
0x1020d:  adc     %ah, 0x33(%rax)
0x10210:  mov     $0x81, %al
0x10212:  add     %al, 0x100000(%rax)
0x10218:  add     %dl, (%rax)
0x0000000000000000 in ?? ()
(gdb)
```

Github: <https://github.com/AdrianHuang/gdb-linux-real-mode>

initialize_identity_maps

x86_mapping_info
void *(*alloc_pgt_page)(void *)
void *context
unsigned long page_flag
unsigned long offset
bool direct_gbpages
unsigned long kernpg_flag

```
static void *alloc_pgt_page(void *context)
{
    struct alloc_pgt_data *pages = (struct alloc_pgt_data *)context;
    unsigned char *entry;

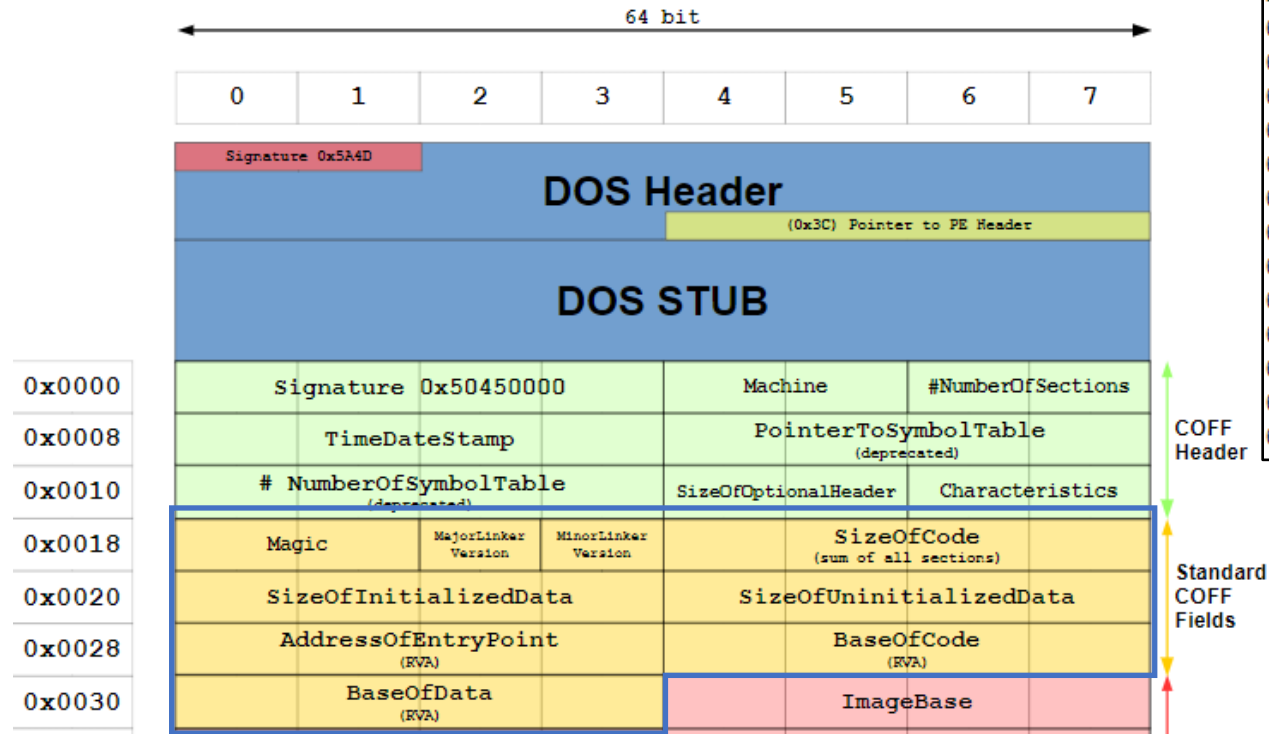
    /* Validate there is space available for a new page. */
    if (pages->pgt_buf_offset >= pages->pgt_buf_size) {
        debug_putstr("out of pgt_buf in " __FILE__ "!\n");
        debug_putstr(pages->pgt_buf_offset);
        debug_putstr(pages->pgt_buf_size);
        return NULL;
    }

    entry = pages->pgt_buf + pages->pgt_buf_offset;
    pages->pgt_buf_offset += PAGE_SIZE;

    return entry;
}
```

alloc_pgt_data
unsigned char *pgt_buf
unsigned long pgt_buf_size
unsigned long pgt_buf_offset

UEFI booting flow – EFI boot stub: Entry point



```
$ xxd arch/x86/boot/bzImage
00000000: 4d5a ea07 00c0 078c c88e d88e c08e d031  MZ.....1
00000010: e4fb fcbe 4000 ac20 c074 09b4 0ebb 0700  ....@.. .t.....
00000020: cd10 ebf2 31c0 cd16 cd19 eaf0 ff00 f000  ....1.....
00000030: 0000 0000 0000 0000 0000 0000 8200 0000  .....
00000040: 5573 6520 6120 626f 6f74 206c 6f61 6465  Use a boot load
00000050: 722e 0d0a 0a52 656d 6f76 6520 6469 736b  r....Remove disk
00000060: 2061 6e64 2070 7265 7373 2061 6e79 206b  and press any k
00000070: 6579 2074 6f20 7265 626f 6f74 2e2e 2e0d  ey to reboot....
00000080: 0a00 5045 0000 6486 0400 0000 0000 0000  ..PE..d.....
00000090: 0000 0100 0000 a000 0602 0b02 0214 003e  .....>
000000a0: dd00 0000 0000 0000 0000 4ad8 1800 0002  .....J.....
000000b0: 0000 0000 0001 0000 0000 2000 0000 2000  .....
000000c0: 0000 0000 0000 0100 0000 0000 0000 0000  .....
```

AddressOfEntryPoint (efi_pe_entry): 0x18d84a

```
$ nm arch/x86/boot/compressed/vmlinux | grep efi_pe_entry
000000000018a04a T efi_pe_entry
```

ImageBase = 0x1000000

Physical address of AddressOfEntryPoint = 0x1000000 +
0x18d84a = 0x118d84a

```
static void update_pecoff_text(unsigned int text_start, unsigned int file_sz,
                               unsigned int init_sz)
{
    ...

    /*
     * Address of entry point for PE/COFF executable
     */
    put_unaligned_le32(text_start + efi_pe_entry, &buf[pe_header + 0x28]);

    update_pecoff_section_header_fields(".text", text_start, text_sz + bss_s
z,
                                     text_sz, text_start);
}
arch/x86/boot/tools/build.c 221,2-9 48%
```

UEFI booting flow – EFI Handover protocol

```
=====
Field name:    handover_offset
Type:         read
Offset/size:   0x264/4
=====

This field is the offset from the beginning of the kernel image to
the EFI handover protocol entry point. Boot loaders using the EFI
handover protocol to boot the kernel should jump to this offset.

See EFI HANDOVER PROTOCOL below for more details.

Documentation/x86/boot.rst 888,1
```

```
static void efi_stub_entry_update(void)
{
    unsigned long addr = efi32_stub_entry;

#ifdef CONFIG_X86_64
    /* Yes, this is really how we defined it :( */
    addr = efi64_stub_entry - 0x200;
#endif

#ifdef CONFIG_EFI_MIXED
    if (efi32_stub_entry != addr)
        die("32-bit and 64-bit EFI entry points do not match\n");
#endif

    put_unaligned_le32(addr, &buf[0x264]);
}

arch/x86/boot/tools/build.c 278,8
```

```
#ifdef CONFIG_EFI_MIXED
    .org 0x190
SYM_FUNC_START(efi32_stub_entry)
    add    $0x4, %esp          /* Discard return address */
    popl   %ecx
arch/x86/boot/compressed/head_64.S 255,9-16
```

```
#ifdef CONFIG_EFI_STUB
    .org 0x390
SYM_FUNC_START(efi64_stub_entry)
SYM_FUNC_START_ALIAS(efi_stub_entry)
    and    $~0xf, %rsp        /* realign the stack */
    movq    %rdx, %rbx        /* save boot_params pointer */
    call    efi_main
arch/x86/boot/compressed/head_64.S 489,6-16 62%
```

UEFI booting flow – EFI Handover protocol

```
=====
Field name:      handover_offset
Type:            read
Offset/size:     0x264/4
=====

This field is the offset from the beginning of the kernel image to
the EFI handover protocol entry point. Boot loaders using the EFI
handover protocol to boot the kernel should jump to this offset.

See EFI HANDOVER PROTOCOL below for more details.

Documentation/x86/boot.rst 888,1
```

```
#ifdef CONFIG_EFI_MIXED
.org 0x190
SYM_FUNC_START(efi32_stub_entry)
    add     $0x4, %esp          /* Discard return address */
    popl    %ecx
arch/x86/boot/compressed/head_64.S 255,9-16
```

```
$ xxd -e arch/x86/boot/bzImage
00000000: 07ea5a4d 8c07c000 8ed88ec8 31d08ec0 MZ.....1
00000010: befcbfe4 20ac0040 b40974c0 0007bb0e ....@... .t.....
00000020: f2eb10cd 16cdc031 f0ea19cd 00f000ff ....1.....
00000030: 00000000 00000000 00000000 00000082 .....
00000040: 20657355 6f622061 6c20746f 6564616f Use a boot loader
00000050: 0a0d2e72 6d65520a 2065766f 6b736964 r....Remove disk
00000060: 646e6120 65727020 61207373 6b20796e and press any k
00000070: 74207965 6572206f 746f6662 0d2e2e2e ey to reboot....
00000080: 4550000a 86640000 00000004 00000000 ..PE..d.....
00000090: 00010000 00a00000 020b0206 3e001402 .....>
000000a0: 000000dd 00000000 d84a0000 02000018 .....J.....
000000b0: 00000000 00000100 00200000 00200000 .....
000000c0: 00000000 00010000 00000000 00000000 .....
000000d0: 40000000 020000dd 00000000 000a0000 ...@.....
000000e0: 00000000 00000000 00000000 00000000 .....
000000f0: 00000000 00000000 00000000 00000000 .....
00000100: 00000000 00060000 00000000 00000000 .....
00000110: 00000000 00000000 00000000 00000000 .....
00000120: 00000000 00000000 00000000 00000000 .....
00000130: 00000000 00000000 732e0000 70757465 .....setup
00000140: 35c00000 02000000 35c00000 02000000 ...5.....5....
00000150: 00000000 00000000 00000000 00200000 .....
00000160: 722e6050 636f6c65 00200000 37c00000 P`.reloc.. ....7
00000170: 00200000 37c00000 00000000 00000000 .. ....7.....
00000180: 00000000 00400000 632e4210 61706d6f .....@...B.compa
00000190: 00200074 37e00000 00200000 37e00000 t. ....7.. ....7
000001a0: 00000000 00000000 00000000 00400000 .....@.
000001b0: 742e4210 00747865 08000000 380000dd .B.text.....8
000001c0: 15000000 38000019 00000000 00000000 .....8.....
000001d0: 00000000 00200000 00006050 00000000 .....P`.....
000001e0: 00000000 00000000 00000000 ff000000 .....
000001f0: 00011bff 00019150 ffff0000 aa550000 ....P.....U.
00000200: 64486aeb 020f5372 00000000 33601000 .jHdrS.....`3
00000210: 80000100 00100000 00000000 00000000 .....
00000220: 00000000 00004c10 00000000 7fffffff .....L.....
00000230: 00200000 006f1501 000007ff 00000000 .. ...o.....
00000240: 00000000 00000000 0000040d 00183301 .....3..
00000250: 00000000 00000000 01000000 00000000 .....
00000260: 00bd4000 00000190 001900a0 c08ed88c .@.....
```

UEFI booting flow – EFI Handover protocol

```
=====
Field name:      handover_offset
Type:            read
Offset/size:     0x264/4
=====

This field is the offset from the beginning of the kernel image to
the EFI handover protocol entry point. Boot loaders using the EFI
handover protocol to boot the kernel should jump to this offset.

See EFI HANDOVER PROTOCOL below for more details.

Documentation/x86/boot.rst 888,1
```

```
#ifdef CONFIG_EFI_MIXED
.org 0x190
SYM_FUNC_START(efi32_stub_entry)
    add     $0x4, %esp          /* Discard return address */
    popl    %ecx
arch/x86/boot/compressed/head_64.S 255,9-16
```

```
$ xxd -e arch/x86/boot/bzImage
00000000: 07ea5a4d 8c07c000 8ed88ec8 31d08ec0 MZ.....1
00000010: befcbfe4 20ac0040 b40974c0 0007bb0e ....@... .t.....
00000020: f2eb10cd 16cdc031 f0ea19cd 00f000ff ....1.....
00000030: 00000000 00000000 00000000 00000082 .....
00000040: 20657355 6f622061 6c20746f 6564616f Use a boot loader
00000050: 0a0d2e72 6d65520a 2065766f 6b736964 r....Remove disk
00000060: 646e6120 65727020 61207373 6b20796e and press any k
00000070: 74207965 6572206f 746f6662 0d2e2e2e ey to reboot....
00000080: 4550000a 86640000 00000004 00000000 ..PE..d.....
00000090: 00010000 00a00000 020b0206 3e001402 .....>
000000a0: 000000dd 00000000 d84a0000 02000018 .....J.....
000000b0: 00000000 00000100 00200000 00200000 .....
000000c0: 00000000 00010000 00000000 00000000 .....
000000d0: 40000000 020000dd 00000000 000a0000 ...@.....
000000e0: 00000000 00000000 00000000 00000000 .....
000000f0: 00000000 00000000 00000000 00000000 .....
00000100: 00000000 00060000 00000000 00000000 .....
00000110: 00000000 00000000 00000000 00000000 .....
00000120: 00000000 00000000 00000000 00000000 .....
00000130: 00000000 00000000 732e0000 70757465 .....setup
00000140: 35c00000 02000000 35c00000 02000000 ...5.....5....
00000150: 00000000 00000000 00000000 00200000 .....
00000160: 722e6050 636f6c65 00200000 37c00000 P`.reloc.. ....7
00000170: 00200000 37c00000 00000000 00000000 .. ....7.....
00000180: 00000000 00400000 632e4210 61706d6f .....@...B.compa
00000190: 00200074 37e00000 00200000 37e00000 t. ....7.. ....7
000001a0: 00000000 00000000 00000000 00400000 .....@.
000001b0: 742e4210 00747865 08000000 380000dd .B.text.....8
000001c0: 15000000 38000019 00000000 00000000 .....8.....
000001d0: 00000000 00200000 00006050 00000000 .....P`.....
000001e0: 00000000 00000000 00000000 ff000000 .....
000001f0: 00011bff 00019150 ffff0000 aa550000 ....P.....U.
00000200: 64486aeb 020f5372 00000000 33601000 .jHdrS.....`3
00000210: 80000100 00100000 00000000 00000000 .....
00000220: 00000000 00004c10 00000000 7fffffff ....L.....
00000230: 00200000 006f1501 000007ff 00000000 .. ...o.....
.....3..
.....
..@.....
```

Where is the address of bzimage loaded by boot loader?

UEFI booting: call path

```
adrian@adrian-ubuntu:gdb-linux-real-mode$ vimcat scripts/launch-vm.sh
#!/bin/bash

source `find ./ -name config.sh | head -n1`

qemu_bin=qemu-system-x86_64

${qemu_bin} -nographic -smp 4 -m 2047M -bios /usr/share/qemu/OVMF.fd \
    -kernel $KERNEL_OBJ/arch/x86/boot/bzImage \
    -initrd $INITRAMFS_IGZ \
    -append "earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr" \
    -s -S
```

```
---- efi linux + qemu: code flow ----
efi64_stub_entry [arch/x86/boot/compressed/head_64.S] -- ** long mode **
|- efi_main
  jmp *%rax (jmp startup_64)
```